Zephyr RTOS

Zephyr RTOS

The build, the kernel and a driver, on a board that does not exist

Erik Pihl

QRTECH ACADEMY
20 September 2026

Zephyr RTOS
Copyright © 2026 QRTECH Academy.

The text and figures of this book, and of the course it is typeset from, are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). You may copy and redistribute them, and adapt them, for any non-commercial purpose, provided you give appropriate credit and share what you make under the same license. To view a copy of the license, visit `https://creativecommons.org/licenses/by-nc-sa/4.0/`.

The code in this book may also be used under the MIT License, which is the license of all the code in the companion repository. The repository holds the lecture material, the example programs, the exercises and their test suites:

`https://github.com/qrtech-academy/zephyr`

Every program in this book is built for Zephyr 4.4.2 and the board `native_sim/native/64`, with the host's GCC, on Linux. Zephyr itself is licensed under Apache 2.0 and is not part of this book; §1.1 fetches it. Zephyr, Linux and the other product names in this book are trademarks of their respective owners.

Set in TeX Gyre Pagella and DejaVu Sans Mono with LuaLaTeX.

Typeset from the six lectures of the course and their exercises. This edition: 20 September 2026.

Contents

Preface **vii**

1 Zephyr, west and the Build **1**
1.1 Setting Up the Workspace . . . 1
1.2 Zephyr, west and the Build . . . 4
1.3 Review . . . 17
1.4 Exercises . . . 18

2 Devicetree **25**
2.1 Devicetree . . . 25
2.2 Review . . . 41
2.3 Exercises . . . 42

3 Kernel I: Threads, Time and Work Queues **53**
3.1 Threads, Time and Work Queues . . . 54
3.2 Review . . . 65
3.3 Exercises . . . 66

4 Kernel II: Communication and Application Architecture **77**
4.1 Kernel Objects: Synchronization, Data and Memory . . . 77
4.2 Application Architecture: zbus and the State Machine Framework . . . 87
4.3 Review . . . 94
4.4 Exercises . . . 95

5 Writing a Driver **107**
5.1 Device Drivers . . . 108
5.2 Emulators and Testing . . . 118
5.3 Review . . . 125
5.4 Exercises . . . 126

6 Subsystems and Debugging **135**
6.1 Subsystems: the Shell, Settings and Networking . . . 135

6.2 Debugging: Fatal Errors, Sanitizers, GDB and Coverage 145
6.3 Review 152
6.4 Exercises 153

A The Self-Assessment Papers **165**

B Self-Assessment Paper A **169**

C Self-Assessment Paper B **179**

D Model Answers to Paper A **185**
Question 1: The workspace and the build 185
Question 2: Kconfig and logging 187
Question 3: Devicetree, bindings and overlays 189
Question 4: Threads, scheduling and time 190
Question 5: Interrupt context, work queues and simulated time 192
Question 6: Synchronization and communication 193
Question 7: Writing a driver 195
Question 8: Subsystems and debugging 197

E Model Answers to Paper B **199**
Question 1: The workspace and the build 199
Question 2: Configuring an application 202
Question 3: Describing hardware 204
Question 4: Threads and time 207
Question 5: Interrupts, work and the simulator 209
Question 6: Threads that must agree 211
Question 7: A driver of your own 214
Question 8: Making it usable 219

Preface

This book is for embedded C developers who want to build firmware with Zephyr. Every program in it is built and run on Linux, on Zephyr's `native_sim` board, so no hardware is needed: not a development board, not a debug probe, not a single wire.

The hard part of Zephyr is not the kernel. An engineer who has used FreeRTOS, or written a scheduler, already knows what a thread, a mutex and a message queue are. What stops people is everything around them: a workspace that west manages, a build that is CMake plus Kconfig plus devicetree, and a driver model held together by macros. When Zephyr goes wrong, it is rarely `k_thread_create()` at fault. It is `undefined reference to __device_dts_ord_42`, or a build aborted by a Kconfig warning, or a misspelled `CONFIG_` macro that C quietly treats as off.

So this book spends its first two chapters on the build and on devicetree, teaches the kernel as *the Zephyr way to do what you already know*, and finishes by writing a driver for a sensor that does not exist, then testing and debugging it. Along the way it covers:

- The west workspace, the manifest, and pinning a Zephyr release.
- The build: CMake, `prj.conf`, Kconfig, and what to do when the build rejects an option.
- Logging.
- Devicetree: nodes, properties, bindings, overlays, and the macros that read them at compile time.
- Getting devices from the devicetree, and GPIO through Zephyr's emulated GPIO controller.
- Threads, priorities, cooperative and preemptive scheduling, and time.
- Interrupt context, and deferring work out of it with work queues.
- Semaphores, mutexes with priority inheritance, message queues, events and polling.
- Application architecture with zbus and the State Machine Framework.
- The device driver model: API structs, instances, init levels and bindings.
- Emulators, which stand in for hardware in drivers and tests.
- Testing with ztest and Twister.
- The shell, settings stored on a simulated flash, and networking through host sockets.
- Debugging on the host: GDB, sanitizers, fatal error handling and coverage.

Why native_sim

`native_sim` compiles Zephyr, kernel and application together, into an ordinary Linux executable. There is no instruction set simulator: your code runs natively on the host CPU, and the hardware around it, a timer, an interrupt controller, a GPIO controller, an I2C bus, a flash chip, is simulated. The book uses it for three reasons.

Everything builds and runs on a laptop. No boards to hand out, no probes, no drivers for a USB debugger in WSL. A build takes seconds and a run takes less.

It is deterministic. Time on `native_sim` is simulated. A program that prints the uptime prints the same numbers on every run and on every machine, which is what lets the exercises compare your output with the expected output line by line.

The host's tools work on it. GDB, AddressSanitizer, UndefinedBehaviorSanitizer, gcov and Valgrind all work on a `native_sim` program as on any other Linux program.

The counterweight belongs on the first page: **`native_sim` is not a microcontroller.** Code runs in zero simulated time, so an interrupt cannot preempt a running thread until that thread calls into the kernel. A busy loop waiting for an interrupt never ends. Nothing here teaches pin multiplexing, clocks, power or flashing. Chapter 3 shows the time model on purpose, because knowing what the simulator does *not* do is as important as knowing what it does.

What you will do

Along the way you will write code, not just read it. You will:

- Set up a pinned Zephyr workspace and build and run applications on `native_sim`.
- Configure applications with Kconfig and describe hardware with devicetree overlays.
- Write multi-threaded applications with the kernel's scheduling and IPC primitives.
- Structure an application around zbus channels and a state machine.
- Write a devicetree-driven sensor driver, and an emulator to test it against.
- Test with ztest and Twister, and debug with GDB and the sanitizers.

The examples and exercises stay close to typical embedded problems: blinking an LED and reading a button through an emulated GPIO controller, periodic sampling with timers and work queues, producer and consumer threads, priority inversion produced and then fixed, a driver for an I2C temperature sensor, and a shell command and a setting that survives a restart.

What you should be able to do at the end

Having worked through the book, you should be able to:

- Set up a Zephyr workspace and explain what west, CMake, Kconfig and devicetree each contribute.

- Find out why a configuration option or devicetree node did not take effect.
- Read a devicetree and binding, and access devices and properties from C.
- Choose between cooperative and preemptive threads, and between an ISR, a work item and a thread.
- Use Zephyr's IPC primitives, and explain when each one is the right tool.
- Write a devicetree-instantiated driver for Zephyr's device model.
- Test drivers and applications with ztest, emulators and Twister.
- Explain what `native_sim` simulates faithfully and what it does not.

What you are assumed to know

The subject of this book is **Zephyr**, not embedded programming in general. You are expected to arrive already comfortable with:

- C programming: functions, pointers, structs and the rest of the language.
- A Linux terminal, `make` and, at the level of reading it, CMake.

Basic RTOS concepts, threads, priorities and blocking, help, but Chapters 3 and 4 cover them again in Zephyr's terms. What is underneath them, the context switch itself, is the subject of a different course, *Real-Time Kernel Design*.

How the book is organised

The book is typeset from a six-lecture course, and each chapter is one lecture. Its sections are the lecture's appendices, in the same order and with the same subsections, so a chapter here and a lecture directory in the repository can be read side by side:

Chapter	Lecture	Topic
1	`L01`	Zephyr, west and the build
2	`L02`	Devicetree
3	`L03`	Kernel I: threads, time and work queues
4	`L04`	Kernel II: communication and application architecture
5	`L05`	Writing a driver
6	`L06`	Subsystems and debugging

Every chapter opens with what is in it, works through the material, closes with a review of what you should now be able to explain, and ends with its exercises.

The live-coding examples are not printed. Each chapter is taught from a handful of small applications, which live in the companion repository under `lectures/LNN/examples`; every one has a README saying how to build and run it and what to look at. The book names them where they belong, and the code in the text is the code that makes a point, not a listing of every file.

What the exercises look like

The exercises come in **sets**. A set builds one small application, or one part of a larger one, and each exercise in a set adds a piece of it. Every exercise is labelled with its kind:

Kind	What it asks of you
CODE	Write C, build it for `native_sim`, and check it with `make test`, which also compares what it printed with the output the exercise gives.
REFLECTION	Answer in prose: explain what your program does, or why the build behaved as it did.

The solutions are not in this book. They are published in the companion repository, under `lectures/LNN/appendix/solutions`, a few days after each lecture. Try each exercise before you read them.

Two written papers, and what they are not

Nothing in the course this book comes from is marked. Assessment is the exercise set after every chapter, each with a published solution, and the questions in every chapter's review.

Appendices A to E hold two three-hour papers with worked solutions, and they check something else: **what you can reconstruct on paper, with nothing in front of you.** Eight questions each, mixing what a build decided and why with the artifacts a Zephyr application is made of, and no build in the room to tell you whether you are right.

They exist purely so that you can test your own knowledge after working through the book. They gate nothing, they are not a qualification, and no part of the book requires them.

Take one after the last chapter. Both papers draw on all six chapters, so sitting one partway through examines material you have not read yet.

Setting up

Everything in this book runs in a Linux terminal: on Linux itself, or on Windows through WSL 2. §1.1 installs the packages, sets up the workspace and checks that it works; do that first, because fetching Zephyr takes a few minutes. In short, it is:

```
git clone https://github.com/qrtech-academy/zephyr.git
cd zephyr
make setup
```

`make setup` creates a Python environment in `.venv`, installs west into it, and fetches the Zephyr release the manifest pins into `deps/`. Nothing is installed outside the repository. Then, in every new terminal:

```
source env.sh
```

Three more commands cover everything the book asks of the repository:

```
# Build an example, and run it.
west build -p -b native_sim/native/64 lectures/L01/examples/hello
west build -t run

# Test your exercises.
make test
```

You write your exercise solutions in the repository, in `lectures/LNN/exercises`, where each lecture's README says which file goes where. `make test` then runs every lecture's tests against them: a set you have not started is reported as `SKIP`, with the file its tests are waiting for, and a set you have written is built, run and compared with the output the exercise gives.

CHAPTER 1

Zephyr, west and the Build

What Zephyr is, and how an application becomes a program.

IN THIS CHAPTER

- What Zephyr is: a kernel, drivers, subsystems, and a build system that ties them together.
- `native_sim`: Zephyr as a Linux program, and why this book runs on it.
- The west workspace: the manifest, `west update`, and pinning a Zephyr release.
- Anatomy of an application: `CMakeLists.txt`, `prj.conf` and `src/`.
- Building and running: `west build`, the build directory, and `west build -t run`.
- Kconfig: options, dependencies, where values come from, and how to read what the build chose.
- Application-specific Kconfig options, used from C and from CMake.
- `printk()` and the logging subsystem: modules, levels and compile-time filtering.

This chapter is about everything that happens before a line of your code is compiled. Zephyr decides what a build contains, and what the hardware in it is, long before the compiler runs, and nearly every question a newcomer has is really a question about that: where Zephyr lives, what west does with it, which options a build ended up with, and why an option that was set was not the one used.

Work the chapter in order. §1.1 sets up the workspace: do it before the first lecture, because fetching Zephyr takes a few minutes. §1.2 is the substance of the chapter, and the exercises in §1.4 are where you write the code yourself. The examples the lecture is taught from are `lectures/L01/examples/hello` and `lectures/L01/examples/kconfig` in the companion repository.

1.1 Setting Up the Workspace

Every program in this book is built and run on Linux. This section installs what the workspace needs, sets it up, and checks that it works. Do it before the first lecture.

1.1.1 Linux

Any recent x86-64 Linux works. On Windows, use **WSL 2** with **Ubuntu 24.04**, as in the other courses in this track. In **Windows PowerShell as Administrator**:

```
wsl --install -d Ubuntu-24.04
```

Restart if asked, start *Ubuntu* from the Start menu, and choose a user name and password. `native_sim` needs WSL **2**: the old WSL 1 cannot run the programs it builds. Check with `wsl --status` in PowerShell, which must say `Default Version: 2`.

Keep the repository in the Linux file system, for example in `~/qacademy`, and not under `/mnt/c`. A Zephyr build touches thousands of files, and through `/mnt/c` it is many times slower.

1.1.2 Packages

In the Ubuntu terminal:

```
sudo apt -y update
sudo apt -y install git make cmake gcc g++ python3 python3-venv python3-pip clang-format gdb
```

What each one is for:

Package	Used for
`git`	Fetching the companion repository, and Zephyr through west.
`make`	The repository's `make setup`, `make test` and friends.
`cmake`	Zephyr's build system. Version 3.20 or newer; Ubuntu 22.04 and 24.04 both have it.
`gcc`, `g++`	The compiler. `native_sim` programs are Linux programs, built by the host's own GCC.
`python3`, `python3-venv`, `python3-pip`	west, Twister, and the scripts that process Kconfig and devicetree.
`clang-format`	`make format`, which formats C files the way Zephyr does.
`gdb`	Debugging in Chapter 6.

No Zephyr SDK is needed. The SDK holds cross-compilers for real microcontrollers; `native_sim` uses the host compiler, which `source env.sh` (below) tells Zephyr to do.

Two packages are optional:

- `device-tree-compiler` (`dtc`). Zephyr processes devicetree with its own Python scripts and uses `dtc` only for extra warnings. Without it, every build prints `Could NOT find Dtc (missing: DTC)`, which is harmless.

- `gcc-multilib g++-multilib`. This book builds for `native_sim/native/64`, a 64-bit program. The plain `native_sim` board is 32-bit, and building for it without these packages fails with `fatal error: bits/libc-header-start.h: No such file or directory`.

1.1.3 The workspace

Clone the repository and set it up:

```
git clone https://github.com/qrtech-academy/zephyr.git
cd zephyr
make setup
```

`make setup` does four things, all inside the repository:

1. Creates a Python environment in `.venv` and installs west into it.
2. Makes the repository a west workspace, with `west.yml` as its manifest.
3. Runs `west update`, which fetches the Zephyr release that `west.yml` pins into `deps/zephyr`.
4. Installs Zephyr's Python requirements, Twister's among them, into `.venv`.

It takes a few minutes and about 1 GB of disk. Nothing is installed outside the repository, and running it again is safe.

Then, **in every new terminal**:

```
source env.sh
```

That activates the Python environment, so that `west` is found, and sets two variables: `ZEPHYR_BASE`, which says where Zephyr is, and `ZEPHYR_TOOLCHAIN_VARIANT=host`, which says to build with the host's GCC. Your prompt starts with `(.venv)` while it is active.

1.1.4 Checking that it works

From the repository root:

```
west build -p -b native_sim/native/64 lectures/L01/examples/hello
west build -t run
```

The first command configures and builds the example into `build/`, which takes about half a minute the first time. The second runs it:

```
*** Booting Zephyr OS build v4.4.2 ***
Hello from native_sim/native/64
[00:00:00.000,000] <inf> hello: Tick 1 at 0 ms
[00:00:00.510,000] <inf> hello: Tick 2 at 510 ms
```

```
[00:00:01.020,000] <inf> hello: Tick 3 at 1020 ms
[00:00:01.530,000] <wrn> hello: A warning, in yellow
```

The program keeps running after that, because a Zephyr system does not exit when `main()` returns. Stop it with `Ctrl+C`. Finally, check that the test suites run:

```
make test
```

With no exercises written yet, every exercise set is reported as `SKIP`, and the command succeeds.

1.1.5 Visual Studio Code

With the **WSL** extension, run `code .` from the repository root in the Ubuntu terminal. For code navigation that understands Zephyr's include paths and `CONFIG_` macros, install the **C/C++** extension and point it at the build's compilation database, which every build writes to `build/compile_commands.json`. In `.vscode/settings.json`:

```
{
    "C_Cpp.default.compileCommands": "${workspaceFolder}/build/compile_commands.↪
    ↪ json"
}
```

1.1.6 When something goes wrong

Message	Cause
`west: command not found`	`source env.sh` has not been run in this terminal. Also `source env.sh`: without it, Zephyr looks for the SDK.
`ZEPHYR_TOOLCHAIN_VARIANT not set, trying to locate Zephyr SDK` followed by `Could not find a package configuration file provided by "Zephyr-sdk"`	
`error: the workspace is not set up`	`make setup` has not been run, or did not finish. Run it again.
`fatal error: bits/libc-header-start.h: No such file or directory`	Building for the 32-bit `native_sim` without `gcc-multilib`. Use `native_sim/native/64`.
A build that fails for no reason you can see after changing boards or options	A stale build directory. Build again with `-p` (pristine).

1.2 Zephyr, west and the Build

This section covers what Zephyr is, the board this book runs on, the workspace it lives in, and the build that turns an application into a program. By the end you should be able

to explain every file in the `lectures/L01/examples/hello` and `lectures/L01/examples/kconfig` examples, and every line of output their builds print that matters.

1.2.1 What Zephyr is

Zephyr is an open-source real-time operating system for microcontrollers, hosted by the Linux Foundation and licensed under Apache 2.0. It runs on ARM Cortex-M and Cortex-A, RISC-V, x86, Xtensa, ARC and more, and ships support for well over a thousand boards.

The word *operating system* is meant more literally than in "FreeRTOS". FreeRTOS gives you a scheduler and a handful of kernel objects, and you bring everything else: the drivers, the logging, the build. Zephyr ships all of it:

Part	What it gives you
The kernel	Threads, scheduling, interrupts, timers, and kernel objects for synchronization and communication. Chapters 3 and 4.
The device driver model	One API per kind of device (GPIO, I2C, sensor, flash, ...), implemented by drivers for many chips. Chapters 2 and 5.
Subsystems	Logging, a shell, settings storage, file systems, networking, Bluetooth, USB, and more. This chapter and Chapter 6.
The build system	CMake, Kconfig and devicetree, which select and configure all of the above at compile time. This chapter and Chapter 2.
Test tooling	The ztest framework and the Twister test runner. Chapter 5.

Everything is decided at compile time. A Zephyr image holds only the drivers, subsystems and kernel features the application asked for, sized for the board it was built for. There is no loading of drivers at run time and nothing to discover: the build has already worked out what hardware exists and which code serves it. That is what makes Zephyr fit into a few kilobytes, and it is also why most of what is hard about Zephyr is in the build. If you have met Linux's `make menuconfig` or its `.dts` files, those are the same two ideas, Kconfig and devicetree, used the same way.

Zephyr releases roughly three times a year, with a long-term support (LTS) release, maintained with fixes for years, every few releases. Projects pin a release and move deliberately; so does this book.

1.2.2 `native_sim`

A *board*, in Zephyr, is what an application is built for: a processor, the peripherals around it, and how they are wired. `native_sim` is a board that does not exist. It is Zephyr's **POSIX architecture**: the kernel, drivers and application compiled together into an ordinary Linux program, which runs on the host CPU.

```
west build -b native_sim/native/64 lectures/L01/examples/hello
./build/zephyr/zephyr.exe
```

Nothing is emulated at the instruction level, as QEMU would. Each Zephyr thread is backed by a Linux thread, and the kernel lets exactly one of them run at a time, so Zephyr's scheduler is in charge exactly as on a microcontroller. The hardware is what is simulated: a system timer, an interrupt controller, and a set of peripherals with emulated backends, among them a GPIO controller, an I2C bus, a flash chip and an EEPROM. Chapters 2 and 5 use those.

Time on `native_sim` is simulated. The system timer counts simulated microseconds, and the simulator advances that clock only when every Zephyr thread is waiting for something. By default a program slows its clock down to real time, so that a one-second sleep takes a second, which is what you want when you watch it. Two command-line options change that:

Option	Effect
`-no-rt`	Advance simulated time as fast as the host can. Ten simulated minutes take a moment.
`-stop_at=<seconds>`	Stop the program after that much simulated time.

`./build/zephyr/zephyr.exe --help` lists all of them. Because time is simulated, a program's behaviour does not depend on how busy the host is: it prints the same timestamps on every run, on every machine. The exercises rely on that.

The same property is `native_sim`'s biggest limitation. Code runs in zero simulated time, so no interrupt, not even the timer's, can arrive while a thread is running; only when it waits. A loop such as `while (!flag) {}`, waiting for an interrupt to set `flag`, never ends, and because time cannot advance, nothing else runs either. On a microcontroller that loop is merely wasteful. Chapter 3 shows it on purpose.

A few kernel features are also missing on `native_sim`: memory protection between threads (userspace), and the stack overflow checks, since each thread really runs on its Linux thread's stack. The Zephyr documentation lists them under *Native simulator: Important limitations*.

1.2.3 The west workspace

Zephyr is not one repository. The kernel and core live in `zephyrproject-rtos/zephyr`, and the vendor HALs, cryptography libraries, file systems and other *modules* each live in repositories of their own, about eighty in all. **west** is Zephyr's tool for managing that: a multi-repository manager in the spirit of Git submodules or Google's `repo`, plus a set of commands for building, flashing and testing.

Workspaces and manifests

A **west workspace** is a directory with a `.west/config` file in it. The config names a **manifest**, a `west.yml` file listing the Git repositories (*projects*) that belong to the workspace, where each one goes, and which revision of each one to check out.

The companion repository is its own workspace. `make setup` writes a `.west/config` that points at the repository's `west.yml`:

```
manifest:
  remotes:
    - name: zephyrproject-rtos
      url-base: https://github.com/zephyrproject-rtos

  projects:
    - name: zephyr
      remote: zephyrproject-rtos
      revision: v4.4.2
      path: deps/zephyr
      clone-depth: 1
      west-commands: scripts/west-commands.yml

  self:
    path: .
```

Line by line:

- **`remotes`** give a name to a base URL. A project's repository URL is its remote's `url-base` followed by the project's name: `https://github.com/zephyrproject-rtos/zephyr`.
- **`revision: v4.4.2`** pins Zephyr to a release tag. It could as well be a branch or a commit hash, and in a product it should be a tag or a hash, so that the same manifest checks out the same code a year later.
- **`path: deps/zephyr`** is where the project is checked out, relative to the workspace.
- **`clone-depth: 1`** fetches only the pinned commit, not Zephyr's decade of history, which makes the first `west update` much faster, at the cost of `git log` showing one commit.
- **`west-commands`** tells west that Zephyr provides extra commands, `west build` and `west twister` among them. Without that line, west knows only its built-in commands.
- **`self: path: .`** says that the manifest repository is the workspace directory itself.

What the manifest does **not** do is import Zephyr's own `west.yml`, which lists all of Zephyr's modules. `native_sim` needs none of them, and together they are several gigabytes. A product built for a real microcontroller would import the few it needs, such as the vendor's HAL, by name.

The commands

West's own commands manage the workspace:

```
west update                  # Check out every project at the revision the ↩
    ↪ manifest pins.
west list                    # Show every project, its path and its revision.
west manifest --resolve      # Show the whole manifest, with imports expanded.
west config build.board      # Read a configuration value from .west/config.
```

`west list` in this workspace:

```
manifest     .                          HEAD                                    ↩
    ↪ N/A
zephyr       deps/zephyr                v4.4.2                                  ↩
    ↪ https://github.com/zephyrproject-rtos/zephyr
```

Moving to another Zephyr release is two steps: change `revision` in `west.yml`, and run `west update`. West never updates a project on its own; `git pull` in the companion repository changes the manifest, and `west update` is what makes the workspace match it.

The extension commands that Zephyr adds are the ones used every day: `west build`, and, in Chapter 5, `west twister`. On real hardware there would also be `west flash` and `west debug`.

1.2.4 Anatomy of an application

A Zephyr application is a directory with at least three files:

```
hello/
    CMakeLists.txt      How to build it.
    prj.conf            How to configure Zephyr for it.
    src/main.c          The code.
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(hello)

target_sources(app PRIVATE src/main.c)
```

`find_package(Zephyr)` is where Zephyr's build system takes over. It finds Zephyr through `ZEPHYR_BASE`, which `env.sh` sets, and then does most of the work described in *The build* below: it reads the board, processes the devicetree, runs Kconfig, and sets up the whole of Zephyr as a set of CMake libraries. It also creates a library target named **`app`**, which is where the application's own sources go. `project()` must come after it, because Zephyr selects the compiler before CMake's `project()` looks for one.

prj.conf

A list of Kconfig settings, one per line, applied on top of the board's defaults:

```
CONFIG_LOG=y
```

The file must exist, even if it is empty. An application without one fails to configure with `No prj.conf file(s) was found`.

src/main.c

`main()` is called by the kernel after it has initialized itself and every driver, in a thread called `main`. Returning from it ends that thread, **not the program**: the kernel keeps running, along with every other thread, and if nothing else has work to do, the idle thread runs forever. On `native_sim` that is why a program whose `main()` has returned keeps running until `Ctrl+C` or `-stop_at`.

Optional files

Applications grow more files as they need them. Those you will meet in this book:

File	What it is	Chapter
`Kconfig`	Configuration options of the application's own.	This one
`*.conf`	Extra Kconfig settings, merged when asked for with `-DEXTRA_CONF_FILE`.	This one
`app.overlay`	Changes to the board's devicetree for this application.	Chapter 2
`dts/bindings/`	Devicetree bindings of the application's own.	Chapter 2
`boards/`	Kconfig settings and overlays that apply to one board only.	Chapter 2
`sample.yaml`, `testcase.yaml`	How Twister builds, runs and checks the application.	Chapter 5

1.2.5 The build

west build

```
west build -b native_sim/native/64 lectures/L01/examples/hello
```

`west build` is a wrapper around CMake and Ninja. It configures the application for the board given with `-b`, into a build directory, `build/` unless `-d` says otherwise, and then builds it. The options you will use most:

Option	Effect
`-b <board>`	The board to build for. The repository sets `native_sim/native/64` as the workspace default, so it may be left out.
`-d <dir>`	The build directory. Defaults to `build`.
`-p`	*Pristine*: delete the build directory first. By default (`-p auto`) west does so only when the application directory has changed.
`-t <target>`	Build a particular target, such as `run` or `menuconfig`.
`-- <args>`	Pass the rest on to CMake, e.g. `-- -DEXTRA_CONF_FILE=shout.conf`.

A build directory remembers its board and application, so a second build of the same application needs no arguments, and `west build -t run` from the repository root runs what `build/` holds.

What happens when

A Zephyr build has two phases, and the first is the one that is different.

Configuration, which CMake runs, happens once per build directory, and again when an input changes. In order:

1. **The board** is found, and with it the processor architecture, the SoC, and the board's default Kconfig settings and devicetree.
2. **Devicetree.** The board's devicetree and any overlays are combined into one tree, checked against their bindings, and turned into a C header of macros.
3. **Kconfig.** Every Kconfig option in Zephyr and the application is resolved, from its defaults, the board's settings, `prj.conf` and the rest, into one final configuration. Kconfig runs after devicetree because options may depend on it: once GPIO is enabled at all, the driver for a GPIO controller is enabled by default exactly when the devicetree has an enabled node for it (`depends on DT_HAS_ZEPHYR_GPIO_EMUL_ENABLED`, for `native_sim`'s).
4. **CMake** generates the build: which of Zephyr's source files to compile, decided by the Kconfig options, with which flags.

Compilation, which Ninja runs, then builds the libraries Zephyr is made of and links them with the application into one image. On `native_sim` that image is a Linux executable.

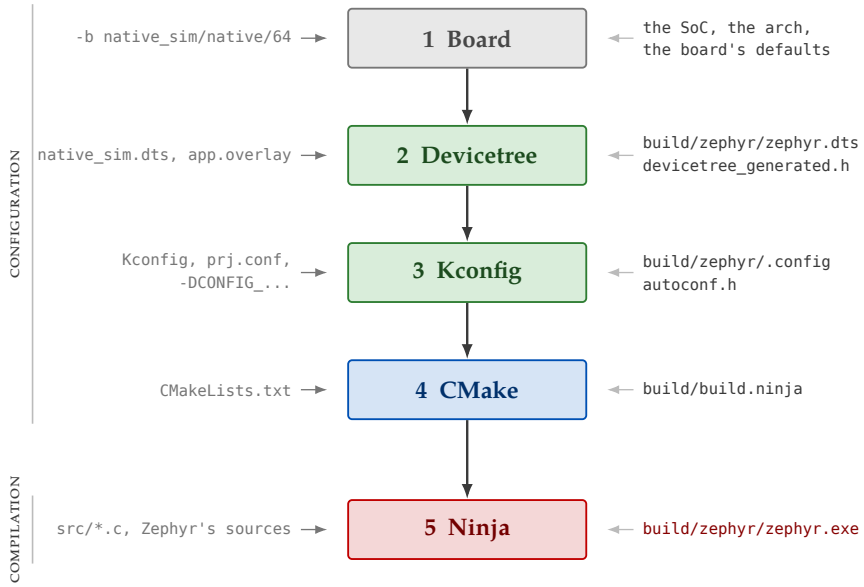


Figure 1.1 The build, in the order it happens. Configuration runs once per build directory, and again when an input changes; compilation follows it. Kconfig comes after devicetree, so that an option can depend on a node.

The order explains most of Zephyr's error messages. A Kconfig error aborts configuration before a line of C is compiled. A devicetree error aborts it even earlier. And a C file cannot change which drivers are built, because by the time it is compiled, that has long been decided.

The build directory

Among the thousands of files in `build/`, a few are worth knowing by name:

File	What it holds
`build/zephyr/zephyr.exe`	The program, on `native_sim`. On a microcontroller the image would be `zephyr.elf`, `zephyr.bin` and `zephyr.hex`; `native_sim` writes a `zephyr.elf` too.
`build/zephyr/.config`	The final Kconfig configuration: every option and its value.
`build/zephyr/include/generated/zephyr/autoconf.h`	The same configuration as C macros, included into every source file.
`build/zephyr/zephyr.dts`	The final devicetree, overlays applied. Chapter 2.
`build/zephyr/include/generated/zephyr/devicetree_generated.h`	The devicetree as C macros. Chapter 2.
`build/zephyr/zephyr.map`	The linker map: where every symbol went.
`build/compile_commands.json`	Every compiler command line, for editors.

Running

On `native_sim`, the program is run either by the build or directly:

```
west build -t run                              # Run build/zephyr/zephyr.exe.
./build/zephyr/zephyr.exe -stop_at=3 -no-rt    # Run it with options.
```

1.2.6 Kconfig

Zephyr has thousands of configuration options. Kconfig, the configuration language from the Linux kernel, is how they are declared, how their dependencies are enforced, and how an application sets them.

Declaring options

An option is a *symbol* with a type, a prompt, a default, and optionally dependencies:

```
config LOG
        bool "Logging"
        select PRINTK if USERSPACE
        help
          Global switch for the logger, when turned off log calls will not be
          compiled in.

if LOG

config LOG_DEFAULT_LEVEL
        int "Default log level"
        default 3
        range 0 4

endif # LOG
```

(Both are Zephyr's own, from `subsys/logging/Kconfig` and `Kconfig.filtering`, shortened.)

Keyword	Meaning
`bool`, `int`, `hex`, `string`	The type, followed by the prompt shown in menuconfig. A symbol with no prompt cannot be set by the user; only defaults and `select` set it.
`default`	The value when nothing sets it. Several `default ... if ...` lines may follow each other; the first whose condition holds applies.
`depends on`	The symbol does not exist, and is `n`, unless the condition holds. `if LOG` ... `endif` adds `depends on LOG` to every symbol between them.
`select`	Forces another bool symbol to `y` whenever this one is `y`, dependencies or not.
`range`	The allowed values of an `int` or `hex`.
`menu`, `choice`	Group symbols in menuconfig; a `choice` allows exactly one of its bool symbols.

Where a value comes from

An option's final value is the last one assigned, from these sources in this order:

1. The **default** in the option's declaration.
2. The **board's defaults**, from the board's `_defconfig` file. For this book's board, that is `deps/zephyr/boards/native/native_sim/native_sim_64_defconfig`.
3. **`prj.conf`**, and any board-specific `.conf` file in the application's `boards/` directory.
4. **Extra configuration files**, given with `-DEXTRA_CONF_FILE=<file>`.
5. **The command line**: `-DCONFIG_LOG_DEFAULT_LEVEL=2` after `--` on the `west build` line.

Then Kconfig checks the result. Every assignment is only a *request*: an option whose dependencies do not hold stays `n`, and an `int` outside its range falls back on its default. **Zephyr treats that as an error**, because a request that silently did not take is how configurations rot. With `CONFIG_SHELL_LOG_BACKEND=y` but no shell:

```
warning: SHELL_LOG_BACKEND (defined at subsys/shell/Kconfig:278) was assigned the ↪
    ↪ value 'y' but got
the value 'n'. Check these unsatisfied dependencies: SHELL (=n). ...

error: Aborting due to Kconfig warnings
```

The message names the dependency that failed. The fix is to enable it too, or to find out why it cannot be.

Reading the result

After a build, `build/zephyr/.config` holds every symbol's value, and grep is the fastest way to answer *what did the build actually use?*:

```
grep -E "CONFIG_LOG_MODE|CONFIG_SYS_CLOCK_TICKS_PER_SEC" build/zephyr/.config
```

```
CONFIG_SYS_CLOCK_TICKS_PER_SEC=100
CONFIG_LOG_MODE_IMMEDIATE=y
```

For browsing, `west build -t menuconfig` opens the same configuration in the terminal, and `west build -t guiconfig` in a window, if Ubuntu's `python3-tk` package is installed. `/` searches, and `?` on a symbol shows its definition, dependencies and what selects it, which is the fastest way to find out *why* an option has the value it has. **Changes made in menuconfig live only in the build directory**: they are lost with the next pristine build. Anything that should last belongs in `prj.conf`.

Using options from C

Every option that is set becomes a macro in `autoconf.h`, which the build includes into every source file: `CONFIG_LOG_DEFAULT_LEVEL` is `3`, `CONFIG_BOARD_TARGET` is `"native_sim/native/64"`, and a bool that is `y` is `1`. **A bool that is `n` is not defined at all.** There are two ways to test one:

```
#ifdef CONFIG_APP_SHOUT
        shout(greeting, loud, sizeof(loud));
#endif

if (IS_ENABLED(CONFIG_APP_SHOUT)) {
        shout(greeting, loud, sizeof(loud));
}
```

Prefer **`IS_ENABLED()`**. It expands to `1` or `0`, so the code inside is compiled and type-checked in every configuration and removed by the compiler in the ones where it cannot run. Code behind `#ifdef` is not even parsed when the option is off, so it breaks unnoticed. Keep `#ifdef` for what cannot compile at all when the option is off, such as a reference to a struct member that exists only with the option.

Both forms share one trap. `IS_ENABLED(CONFIG_APP_SHOUTT)`, misspelled, compiles without a warning and is `0`, because an undefined macro is simply an option that is off. Kconfig catches a misspelled option in `prj.conf`; nothing catches one in C.

Using options from CMake

Kconfig has run by the time `find_package(Zephyr)` returns, so every option is a CMake variable too, and CMake can decide what to compile:

```
target_sources_ifdef(CONFIG_APP_SHOUT app PRIVATE src/shout.c)
```

That is how all of Zephyr is built: nearly every driver and subsystem is a list of `*_sources_ifdef(CONFIG_...)` lines.

Options of your own

An application with a file named `Kconfig` gets options of its own. That file **replaces** Zephyr's top-level Kconfig file for the build, so it must bring Zephyr's back in at the end:

```
menu "Application"

config APP_REPEAT
        int "How many times to print the greeting"
        range 1 10
        default 3

endmenu

source "Kconfig.zephyr"
```

Prefix the options with `APP_`, or with the application's name, so that they cannot collide with Zephyr's. They then behave exactly like Zephyr's own: they appear in menuconfig, are set from `prj.conf`, and reach C and CMake as `CONFIG_APP_REPEAT`.

1.2.7 `printk` and logging

`printk`

`printk()` prints to the console, with `printf()`-style formatting from Zephyr's own small formatter. It writes synchronously, from any context, and is on by default. It is the right tool for a quick look, and the wrong one for anything that ships.

The logging subsystem

Logging is enabled with `CONFIG_LOG=y`. Each source file belongs to a **log module**, registered once, with a name and a level:

```
#include <zephyr/logging/log.h>

LOG_MODULE_REGISTER(monitor, LOG_LEVEL_INF);

LOG_ERR("Temperature %d C is critical", celsius);
LOG_WRN("Temperature %d C is high", celsius);
LOG_INF("Temperature %d C", celsius);
LOG_DBG("Raw sample 0x%04x", raw);
```

Every message is printed with a timestamp, its level and its module, and in color on a terminal:

```
[00:00:01.530,000] <wrn> monitor: Temperature 33 C is high
```

Other files of the same module use `LOG_MODULE_DECLARE(monitor, LOG_LEVEL_INF)` instead of registering it again. `LOG_HEXDUMP_INF(data, length, "label")` prints a buffer.

Levels

The levels are numbered: `LOG_LEVEL_NONE` is 0, `ERR` 1, `WRN` 2, `INF` 3 and `DBG` 4. **A module's level is a compile-time filter**: a message above it is not merely suppressed, it is not compiled into the image at all, format string included. That is why debug messages can be left in the code for free, and why turning them on needs a rebuild.

`LOG_MODULE_REGISTER(monitor)`, with no level, uses `CONFIG_LOG_DEFAULT_LEVEL`, which is 3 unless something sets it. Zephyr's own modules each have a Kconfig option for their level instead, such as `CONFIG_I2C_LOG_LEVEL`, and an application can give its modules one the same way, with a template from Zephyr's logging Kconfig:

```
module = MONITOR
module-str = monitor
source "subsys/logging/Kconfig.template.log_config"
```

That declares a choice of levels, set in `prj.conf` with a line such as `CONFIG_MONITOR_LOG_LEVEL_DBG=y`, and the resulting number as `CONFIG_MONITOR_LOG_LEVEL`. The module is registered with `LOG_MODULE_REGISTER(monitor, CONFIG_MONITOR_LOG_LEVEL)`.

Deferred and immediate

There are two ways a message can reach the console, and the choice matters on a microcontroller:

- **Deferred** logging, the default on real hardware, only copies the message's arguments into a buffer; a low-priority log thread formats and prints it later. Logging from an interrupt handler then costs microseconds, not the milliseconds a UART takes. The price is that a log message and a `printk()` right after it may appear in the opposite order, and that messages still in the buffer are lost if the system crashes.
- **Immediate** logging formats and prints in the caller, like `printk()`.

`native_sim` logs in immediate mode by default (`CONFIG_LOG_MODE_IMMEDIATE`), because printing on a host costs nothing that matters and ordered output is easier to read. Keep in mind that the same application on a board will be deferred unless told otherwise.

1.2.8 Time, briefly

Chapter 3 covers time properly. Two things are needed before then.

Delays are written in milliseconds and kept in ticks. `k_msleep(500)` and `k_sleep(K_MSEC(500))` are the same call. The kernel converts the duration to *ticks* of its system clock, of which `native_sim` has `CONFIG_SYS_CLOCK_TICKS_PER_SEC=100`, one every 10 ms. `k_uptime_get()` returns the milliseconds since boot.

A sleep lasts at least as long as asked, and usually a little longer. A thread that calls `k_msleep(500)` at some moment within a tick cannot know how much of that tick is left, so the kernel waits the 50 ticks the duration needs, plus the rest of the current one. That is why the `lectures/L01/examples/hello` example wakes at 510 ms and 1020 ms, not at 500 and 1000. On `native_sim` the numbers are the same on every run; on a board they would be the same too, which is the point of a tick-based kernel.

1.2.9 Summary

- Zephyr is a kernel, a driver model, subsystems, and a build that selects and configures all of them at compile time.
- `native_sim` builds Zephyr into a Linux program with simulated time: deterministic and fast, but a thread that never waits stops the world.
- A west workspace is a manifest and the repositories it pins. `west update` makes the workspace match the manifest.
- An application is `CMakeLists.txt`, `prj.conf` and sources. `find_package(Zephyr)` is where the build takes over.
- The build configures in order: board, devicetree, Kconfig, CMake. Then it compiles.
- A Kconfig value is the last one assigned, from the defaults through `prj.conf` to the command line, and an assignment that does not take aborts the build. `build/zephyr/.config` shows the result.
- Use `IS_ENABLED()` in C, and `*_ifdef()` in CMake. A misspelled `CONFIG_` macro in C is off, silently.
- Log through log modules. A module's level decides at compile time which messages exist.

1.3 Review

What you should be able to do now

After this chapter, you should:

- Be able to set up the workspace and build and run an application on `native_sim`.
- Understand what the west manifest pins, and where Zephyr lives in the workspace.
- Understand the three files every Zephyr application has, and what each one contributes.
- Understand the order in which a Zephyr build configures itself: board, devicetree, Kconfig, CMake.
- Be able to find the configuration a build ended up with, in `.config` and in menuconfig.
- Be able to add Kconfig options of your own and use them from C and CMake.
- Be able to log at the right level, and to change which levels a build keeps.

Questions to test yourself

You should be able to explain:

1. What does west do, and what does the manifest `west.yml` pin?
2. What are the roles of `CMakeLists.txt`, `prj.conf` and devicetree in a Zephyr build?
3. Where does a Kconfig option's final value come from, and where can you read it after a build?
4. Why does the build abort when `prj.conf` sets an option whose dependencies are not met?
5. Why is a misspelled `CONFIG_` macro in C more dangerous than a misspelled option in `prj.conf`?
6. What is the difference between `#ifdef CONFIG_X` and `IS_ENABLED(CONFIG_X)`?
7. Why does `k_msleep(1000)` on `native_sim` wake up after 1010 ms?
8. What does a log module's level control, and why is a filtered-out message not in the binary?

Looking ahead

Chapter 2 is about devicetree:

- How a board and its peripherals are described: nodes, properties, labels and aliases.
- Bindings, which say what a node's properties mean.
- Overlays, which change a board's devicetree for one application.
- Getting devices and properties from C at compile time.

You will blink an LED and read a button through Zephyr's emulated GPIO controller.

1.4 Exercises

These exercises reinforce the concepts from §1.2.

> **How to check your work.** Write each exercise set as its own application under `lectures/L01/exercises`, with the file names given there, and run `make test` from the repository root. A set's tests switch on as soon as its files exist, and a set you have not started is reported as `SKIP`. Each application grows through its set, and the tests check where it ends up. Programs run on `native_sim` in simulated time, so their output, timestamps included, is compared with the expected output line by line.
> The solutions are published in the companion repository, under `lectures/L01/appendix/solutions`, a few days after the lecture. Try each exercise yourself before you read them.

Remember `source env.sh` in every new terminal. Build and run an exercise from the repository root:

```
west build -p -b native_sim/native/64 lectures/L01/exercises/exercise1
west build -t run
```

EXERCISE SET 1

A first application

Create a directory named `exercise1` to work in for this exercise set.

Exercise 1.1 The three files

CODE

In this exercise you will create the smallest Zephyr application that builds.

Tasks

Create these files in `exercise1`:

- `CMakeLists.txt`: require CMake 3.20, find the Zephyr package, name the project `exercise1`, and add `src/main.c` to the `app` target.
- `prj.conf`: an empty file, or one holding only a comment.
- `src/main.c`: a `main()` that prints `Hello, Zephyr!` with `printk()` and returns `0`.

Build and run the application.

Reflection

- What happens if you delete `prj.conf` and build again with `-p`?
- The program prints its greeting and then keeps running. Why does it not exit when `main()` returns? How do you stop it?
- Where in `build/` is the program you ran?

Exercise 1.2 The uptime

CODE

Tasks

Extend `main()` so that after the greeting it prints the uptime three times, with `k_msleep(1000)` after each print:

```
Hello, Zephyr!
Uptime: 0 ms
Uptime: 1010 ms
Uptime: 2020 ms
```

Use `k_uptime_get()`, which returns an `int64_t`, printed with `%lld`.

Run the program with `./build/zephyr/zephyr.exe -stop_at=5`, and then again with `-stop_at=5 -no-rt`.

Reflection

- Why is the second uptime 1010 ms and not 1000 ms? Find the option in `build/zephyr/.config` that decides it.
- What is the difference between the two runs? Is the output different?
- Would the uptimes be the same if the host were busy compiling something else at the same time? Why?

EXERCISE SET 2

Logging

Create an application named `exercise2` to work in for this exercise set, with the same three files as `exercise1`.

Exercise 1.3 A temperature monitor

CODE

In this exercise you will replace `printk()` with the logging subsystem.

Tasks

- Enable logging in `prj.conf`.
- Register a log module named `monitor` **without** giving a level: `LOG_MODULE_REGISTER(monitor);`.
- In `main.c`, define the readings as an array:

```
static const int readings[] = {21, 24, 28, 33, 36, 29};
```

- In `main()`, go through the readings, and for each one log:
  - `Temperature <t> C is critical` as an error, if it is 35 or more.
  - `Temperature <t> C is high` as a warning, if it is 30 or more.
  - `Temperature <t> C` as information, otherwise.
- Sleep 500 ms after each reading.

The output must be:

```
[00:00:00.000,000] <inf> monitor: Temperature 21 C
[00:00:00.510,000] <inf> monitor: Temperature 24 C
[00:00:01.020,000] <inf> monitor: Temperature 28 C
[00:00:01.530,000] <wrn> monitor: Temperature 33 C is high
[00:00:02.040,000] <err> monitor: Temperature 36 C is critical
[00:00:02.550,000] <inf> monitor: Temperature 29 C
```

Hint: `ARRAY_SIZE(readings)`, from `<zephyr/sys/util.h>`, which `<zephyr/kernel.h>` includes, gives the number of elements.

Exercise 1.4 Only what matters CODE

Tasks

Without changing any file, build the application so that it logs only warnings and errors, by setting the default log level on the command line:

```
west build -p -b native_sim/native/64 lectures/L01/exercises/exercise2 -- -↩
    ↪ DCONFIG_LOG_DEFAULT_LEVEL=2
```

The output must now be only:

```
[00:00:01.530,000] <wrn> monitor: Temperature 33 C is high
[00:00:02.040,000] <err> monitor: Temperature 36 C is critical
```

Reflection

- Why did this change the monitor's output, when the command line said nothing about `monitor`?
- Would it still work if the module were registered as `LOG_MODULE_REGISTER(monitor, LOG_LEVEL_INF)`?
- Are the information messages still in the program, only not printed? Explain how you know.
- After this build, run `west build -t run` a few times, and then build again **without** `-p` and without the `-D` option. Which log level does that build use? Look for the answer in `build/zephyr/.config`, and explain it.

EXERCISE SET 3

Options of your own

Create an application named `exercise3` to work in for this exercise set.

Exercise 1.5 The threshold CODE

In this exercise you will give the application Kconfig options of its own.

Tasks

- Create a `Kconfig` file in `exercise3` with a menu named `Application`, holding:
  - `APP_HIGH_C`, an `int` with the prompt `Temperature at which a reading is high, in degrees Celsius`, a range from -40 to 125, and a default of 30.
- End the file so that Zephyr's own options are still available.
- In `prj.conf`, set `APP_HIGH_C` to 25.
- In `src/main.c`, go through these readings, printing each with `printk()`, and mark those at or above `CONFIG_APP_HIGH_C` with `(high)`:

```c
static const int readings[] = {-18, 21, 24, 28, 33};
```

The output must be:

```
-18 C
21 C
24 C
28 C (high)
33 C (high)
```

Reflection

- Find `APP_HIGH_C` in `west build -t menuconfig`. What does `?` show about it?
- Set `CONFIG_APP_HIGH_C=200` in `prj.conf` and build. What happens, and why is that better than the alternative?
- Remove the last line of your `Kconfig` file and build with `-p`. What happens?

Exercise 1.6 Fahrenheit, when asked for

CODE

Tasks

- Add a `bool` option `APP_FAHRENHEIT` to the `Application` menu, with the prompt `Print temperatures in degrees Fahrenheit`.
- Create `src/convert.h` and `src/convert.c` with a function:

  ```c
  int celsius_to_fahrenheit(int celsius);
  ```

  which returns the temperature in degrees Fahrenheit, **rounded to the nearest degree**, using only integer arithmetic. For example, 37 °C is 98.6 °F, which rounds to 99, and -18 °C is -0.4 °F, which rounds to 0.
- In `CMakeLists.txt`, compile `src/convert.c` **only** when `CONFIG_APP_FAHRENHEIT` is enabled.
- In `main()`, when the option is enabled, print each reading in Fahrenheit, as `<t> F`, still marked `(high)` by its value in Celsius. Use `IS_ENABLED()`, not `#ifdef`.
- Create a file `fahrenheit.conf` in `exercise3` that enables the option.

Built with `-DEXTRA_CONF_FILE=fahrenheit.conf`, the output must be:

```
0 F
70 F
75 F
82 F (high)
91 F (high)
```

Built without it, the output must be unchanged from Exercise 1.5.

Hint: Work in tenths of a degree: 37 °C is 986 tenths of a degree Fahrenheit. C's integer division truncates towards zero, so `(tenths + 5) / 10` rounds a positive number of tenths to the nearest degree, but not a negative one.

Reflection

- Without `fahrenheit.conf`, `convert.c` is not compiled, yet `main.c` still calls `celsius_to_fahrenheit()`. Why does the program link?
- Would it still link with `#ifdef CONFIG_APP_FAHRENHEIT` around the call instead? Which of the two would you rather maintain, and why?
- Change `IS_ENABLED(CONFIG_APP_FAHRENHEIT)` to `IS_ENABLED(CONFIG_APP_FARENHEIT)` and build with `fahrenheit.conf`. What happens? What would have caught it?

EXERCISE SET 4

Reading the workspace and the build

No code in this set: answer the questions by looking. Use the build of any exercise above.

Exercise 1.7 The workspace

REFLECTION

Tasks

Answer, using west's commands rather than by opening files:

- Which revision of Zephyr is the workspace using, and where is it checked out?
- Which configuration value makes `west build` work without `-b`?

Exercise 1.8 The build

REFLECTION

Tasks

Answer, by looking in the build directory:

- Which log mode (`CONFIG_LOG_MODE_...`) did the `exercise2` build use? Why that one?
- What is the value of `CONFIG_BOARD`, and what is the value of `CONFIG_BOARD_TARGET`? Why are they different?
- How many `#define` lines does `autoconf.h` hold? Does it contain the options that are `n`?
- Find the definition of `k_msleep()` with `grep -n -A3 "k_msleep(int32_t ms)" deps/zephyr/include/zephyr/kernel.h`. What does it call, and what does `static inline` mean for whether `k_msleep` exists as a function in your program?

CHAPTER 2

Devicetree

The board as data, and the macros that read it.

In this chapter

- Why Zephyr describes hardware in data rather than in code.
- Devicetree syntax: nodes, properties, unit addresses, labels, phandles, aliases and `chosen`.
- The board's devicetree: `native_sim` and its emulated peripherals.
- Overlays: changing the board's devicetree for one application, and which overlays a build uses.
- `/zephyr,user`: values of the application's own, without a binding.
- Bindings: what a node's properties mean, which are required, and their types.
- From devicetree to C: node identifiers, `DT_ALIAS()`, `DT_NODELABEL()`, `DT_PROP()`, and `DT_FOREACH_STATUS_OKAY()`.
- Getting a device: `DEVICE_DT_GET()`, `GPIO_DT_SPEC_GET()` and `device_is_ready()`.
- GPIO: outputs, inputs and edge interrupts, through the emulated GPIO controller.
- Reading devicetree errors: `__device_dts_ord_` and friends.

Zephyr keeps the hardware out of the code. What exists, where it is wired and what it is called live in the devicetree, a description the build turns into C macros, and a driver or an application reads the description rather than the datasheet. This chapter takes that apart: the language, the board's own tree, the overlays that change it for one application, the bindings that give a node meaning, and the macros that get a device out of it at compile time. It ends with GPIO, the first device API the book uses.

The examples are `lectures/L02/examples/blinky`, `lectures/L02/examples/button` and `lectures/L02/examples/bindings`, and the exercises in §2.3 build the same three things yourself, against a devicetree the test suite changes underneath you.

2.1 Devicetree

This section covers how Zephyr describes hardware: the devicetree language, the board's tree and the overlays that change it, the bindings that give it meaning, and the macros that turn

it into C. It ends with GPIO, the first device API this book uses, on `native_sim`'s emulated GPIO controller. By the end you should be able to explain every file in the `lectures/L02/examples/blinky`, `lectures/L02/examples/button` and `lectures/L02/examples/bindings` examples, and every devicetree error their READMEs show.

2.1.1 Why hardware is data

An LED blinker for one board is easy to write: the LED is pin 13 of port B, so the code configures pin 13 of port B. The trouble starts with the second board, where the LED is pin 5 of port A and lights when the pin is *low*. The usual answer is a header of `#defines` per board, and a `#ifdef` per difference, and it grows with every board and every peripheral.

Zephyr's answer is to describe the hardware in a separate language, **devicetree**, and to keep that description out of the C code entirely. A board says, in its devicetree, *there is an LED, it is on this pin of this controller, and it is active low*, and gives the LED a role name, `led0`. The application asks for `led0`, and the build fills in the rest. The same C file then builds for every board that has a `led0`, unchanged.

Devicetree comes from Open Firmware, by way of Linux, and the language is the same. What Zephyr does with it is not. Linux compiles the tree into a binary blob that the kernel **parses at boot**. Zephyr processes the tree **at build time**, into a C header of macros, and nothing of the tree itself is left in the image. There is no lookup at run time, no parsing, and no memory spent on it; there is also no way to change the hardware description without rebuilding.

A build uses two kinds of devicetree input:

Input	What it says	Example
Devicetree sources: `.dts`, `.dtsi` and `.overlay` files **Bindings**: `.yaml` files	*What hardware there is*: nodes and their properties. *What the properties mean*: which a node may have, their types, which are required.	`led_0 { gpios = <&gpio0 0 GPIO_ACTIVE_HIGH>; };` `gpios: type: phandle-array, required: true`

2.1.2 The language

Nodes and properties

A devicetree is a tree of **nodes**, each with **properties**. This is a piece of the devicetree of `native_sim`, the board this book builds for, from `deps/zephyr/boards/native/native_sim/native_sim.dts`:

```
/ {
        model = "Native Sim Board";
        compatible = "zephyr,posix";
```

```
        leds: leds {
                compatible = "gpio-leds";

                led0: led_0 {
                        gpios = <&gpio0 0 GPIO_ACTIVE_HIGH>;
                        label = "Green LED";
                };
        };

        i2c0: i2c@100 {
                status = "okay";
                compatible = "zephyr,i2c-emul-controller";
                clock-frequency = <I2C_BITRATE_STANDARD>;
                #address-cells = <1>;
                #size-cells = <0>;
                reg = <0x100 4>;
        };

        gpio0: gpio_emul {
                status = "okay";
                compatible = "zephyr,gpio-emul";
                rising-edge;
                falling-edge;
                high-level;
                low-level;
                gpio-controller;
                #gpio-cells = <2>;
        };
};
```

Reading it from the outside in:

- `/` is the root node. Every other node is a descendant, and is named by its **path**, like a file: `/leds/led_0`.
- **`i2c@100`** is a node name with a **unit address**, the part after `@`. It is the node's first `reg` address, and it makes names unique when there are several of a kind.
- **`gpio0:`** before a node is a **label**. Labels are how other parts of the tree, and the C code, refer to a node without spelling out its path.
- **`&gpio0`** in a property is a **phandle**: a reference to the node labelled `gpio0`.
- **`gpios = <&gpio0 0 GPIO_ACTIVE_HIGH>`** is a *phandle-array*: a phandle followed by numbers the referenced node gives meaning to. `gpio0`'s `#gpio-cells = <2>` says each reference takes two: the pin, then the flags.

- **`status`** says whether a node is in use. `"okay"`, or no `status` at all, means enabled; `"disabled"` means the node is described, but nothing is built for it.
- **`compatible`** says what the node *is*. It selects the node's binding, and with it, the driver. The part before the comma names a vendor.

Property values come in a few forms:

Form	Example	Meaning
Nothing	`rising-edge;`	A boolean: true if present.
String	`label = "Green LED";`	A string.
Cells	`reg = <0x100 4>;`	32-bit numbers, in angle brackets.
Phandle	`zephyr,console = &uart0;`	A reference to another node.
Mixed cells	`gpios = <&gpio0 0 GPIO_ACTIVE_HIGH>;`	Phandles and numbers in one list.
String list	`compatible = "a,b", "c";`	Several strings, comma-separated.
Bytes	`mac = [00 11 22 33 44 55];`	8-bit values, in square brackets.

`GPIO_ACTIVE_HIGH` is a C macro. Devicetree files are run through the C preprocessor before anything else reads them, so `native_sim.dts` can `#include <zephyr/dt-bindings/gpio/gpio.h>`, where `GPIO_ACTIVE_LOW` is `(1 << 0)` and `GPIO_PULL_UP` is `(1 << 4)`. The preprocessed file is `build/zephyr/zephyr.dts.pre`.

Aliases and chosen

Two nodes directly under the root are special, and both hold only phandles:

```
chosen {
        zephyr,console = &uart0;
        zephyr,shell-uart = &uart0;
        zephyr,flash = &flash0;
};

aliases {
        led0 = &led0;
        i2c-0 = &i2c0;
};
```

- **`/aliases`** gives nodes *role names* that applications can rely on across boards. `led0` is the first LED, `sw0` the first button, wherever a board has them. Zephyr's samples, and this book's examples, find their hardware this way.
- **`/chosen`** tells *Zephyr itself* which node to use for a job: `zephyr,console` is where `printk()` goes.

A node therefore has up to three names: its **path** (`/leds/led_0`), which is unique; its **labels** (`led0`), which belong to the source files and are for the tree to refer to itself; and the **aliases** pointing at it (`led0`), which are the stable interface to applications. The label and the

alias here happen to be spelled the same; they are different things, and an overlay can point the alias at another node without touching the label.

2.1.3 The board's devicetree

A build starts from its board's devicetree. For `native_sim/native/64` that is `deps/zephyr/boards/native/native_sim/native_sim_64.dts`, which is one line, `#include "native_sim.dts"`, which in turn includes `posix.dtsi` and `skeleton.dtsi`. `.dtsi` files are fragments shared between boards; a real SoC's `.dtsi` describes every peripheral on the chip, mostly `status = "disabled"`, and each board enables the ones it wires up.

`native_sim`'s tree describes the simulated hardware, and this book uses much of it:

Node	Label	Compatible	Used in
`/gpio_emul`	`gpio0`	`zephyr,gpio-emul`	This chapter
`/leds/led_0`	`led0`	child of `gpio-leds`	This chapter
`/i2c@100`	`i2c0`	`zephyr,i2c-emul-controller`	Chapter 5
`/flash-controller@0`	`flashcontroller0`	`zephyr,sim-flash`	Chapter 6
`/uart`	`uart0`	`zephyr,native-pty-uart`	The console, and Chapter 6's shell

2.1.4 Overlays

An application changes its board's devicetree with **overlays**: devicetree fragments merged on top of the board's tree before anything is generated from it. An overlay can add nodes, change properties, and delete either:

```
/* Add nodes under the root. Nodes with the same path are merged, not replaced. */
/ {
        buttons {
                compatible = "gpio-keys";

                button0: button_0 {
                        gpios = <&gpio0 1 (GPIO_PULL_UP | GPIO_ACTIVE_LOW)>;
                        label = "User button";
                };
        };

        aliases {
                sw0 = &button0;
        };
};

/* Change an existing node, found by its label. */
```

```
&gpio0 {
        status = "disabled";
};

/* Remove a property, or a whole node. */
&led0 {
        /delete-property/ label;
};
/delete-node/ &led0;
```

The first block is `lectures/L02/examples/button/app.overlay`'s, the second is `lectures/L02/examples/blinky/broken.overlay`'s deliberately broken overlay. Deleting `led0` outright breaks the board's own `aliases` node, which still points at it: `devicetree error: /aliases: undefined node label 'led0'`.

Which overlays a build uses

The build looks for overlays in this order, and **uses the first that exists**:

1. The files given with **`-DDTC_OVERLAY_FILE=<files>`**. These *replace* everything below.
2. **`boards/<board>.overlay`** in the application, with the board target's slashes written as underscores: `boards/native_sim_native_64.overlay`.
3. **`<board>.overlay`** in the application directory itself.
4. **`app.overlay`**.

Files given with **`-DEXTRA_DTC_OVERLAY_FILE=<files>`** are then merged on top of whichever was chosen, and that is the option to use for an extra overlay. The order is unlike Kconfig's, where a board's `.conf` file is merged *with* `prj.conf`: an application with both `app.overlay` and `boards/native_sim_native_64.overlay` gets only the second, and the nodes in `app.overlay` are silently missing from the build. The build log says which files it took:

```
-- Found BOARD.dts: .../deps/zephyr/boards/native/native_sim/native_sim_64.dts
-- Found devicetree overlay: .../lectures/L02/examples/button/app.overlay
```

The result

The merged tree is written to **`build/zephyr/zephyr.dts`**, with every node and property annotated with the file and line it came from. That is the file to read when a build does not do what the overlay seemed to say:

```
aliases {
        led0 = &led0;        /* in deps/zephyr/boards/native/native_sim/native_sim↩
    ↪ .dts:35 */
```

```
        rtc = &rtc;          /* in deps/zephyr/boards/native/native_sim/native_sim↩
    ↪ .dts:36 */
        sw0 = &button0;      /* in lectures/L02/examples/button/app.overlay:20 */
};
```

Numbers come out in hex, and macros expanded: the button's flags are `0x11`, `GPIO_PULL_UP | GPIO_ACTIVE_LOW`.

2.1.5 Bindings

A devicetree on its own is only syntax: nothing in `gpios = <&gpio0 0 0>` says it is a GPIO reference, or that an LED must have one. That is what a **binding** says. A binding is a YAML file for one `compatible`, and this is Zephyr's for `gpio-leds`, from `deps/zephyr/dts/bindings/led/gpio-leds.yaml`, without its examples:

```
description: |
  Group of GPIO-controlled LEDs.

  Each LED is defined in a child node of the gpio-leds node.

compatible: "gpio-leds"

child-binding:
  description: GPIO LED child node

  include:
    - name: led-node.yaml
      property-allowlist:
        - label

  properties:
    gpios:
      type: phandle-array
      required: true
```

`child-binding` applies to the children of a `gpio-leds` node, which is why `led_0` needs no `compatible` of its own. Every node in the tree is checked against its binding, and **macros are generated only for the properties a binding declares**.

Writing one

The `lectures/L02/examples/bindings` example has a binding of its own, in `dts/bindings/qacademy,blink-pattern.yaml`:

```
compatible: "qacademy,blink-pattern"

include: base.yaml

properties:
  pattern-name:
    type: string
    required: true

  led-gpios:
    type: phandle-array
    required: true

  delays-ms:
    type: array
    required: true

  repeat:
    type: int
    default: 1
```

Key	Meaning
`compatible`	The nodes this binding is for.
`include`	Other bindings to inherit properties from. `base.yaml` brings the ones every node may have, such as `status`, `compatible` and `reg`.
`type`	`boolean`, `int`, `array` (of ints), `uint8-array`, `string`, `string-array`, `phandle`, `phandles` or `phandle-array`.
`required`	The build fails if a node lacks the property.
`default`	The value when a node does not set it. A property with a default always exists, as far as C can tell.
`enum`	The only values allowed. C gets the value's index in the list, too.
`child-binding`	A binding for the node's children.

A property whose name ends in `-gpios` is by convention a phandle-array of GPIOs, and the GPIO macros below take its name.

Where bindings are found

The build searches `deps/zephyr/dts/bindings`, **the application's own `dts/bindings`**, and the `dts/bindings` of every Zephyr module (Chapter 5). Nothing in `CMakeLists.txt` names the directory; it is searched because it is there. The top of the generated header lists what was searched:

```
 * Directories with bindings:
 *   .../lectures/L02/examples/bindings/dts/bindings, $ZEPHYR_BASE/dts/bindings
```

The part of a `compatible` before the comma is a **vendor prefix**, and the build checks it against `vendor-prefixes.txt` files in the same directories. Without the example's own `dts/bindings/vendor-prefixes.txt`, a plain build still succeeds, with a warning:

```
node '/pattern-sos' compatible 'qacademy,blink-pattern' has unknown vendor prefix ↩
    ↪ 'qacademy'
```

Twister, which Chapter 5 uses to build tests, treats devicetree warnings as errors, and fails.

Nodes without a binding

A node whose `compatible` has no binding, or that has no `compatible` at all and no parent with a `child-binding`, is still in the tree, and C can find out that it exists. But none of its properties get macros. With `nobinding { count = <5>; };` in an overlay, `DT_PROP(DT_PATH(nobinding), count)` does not compile:

```
error: 'DT_N_S_nobinding_P_count' undeclared here (not in a function); did you ↩
    ↪ mean 'DT_N_S_nobinding_PARENT'?
```

There is one exception, made for exactly the case of an application that wants a few values of its own without writing a binding: the node **`/zephyr,user`**. Its property types are inferred from their values, so this works with no YAML at all:

```
/ {
        zephyr,user {
                blink-ms = <200 300>;
                led-gpios = <&gpio0 4 GPIO_ACTIVE_LOW>;
        };
};
```

Errors a binding catches

A binding turns mistakes into errors **before a line of C is compiled**, and names the file and line. These are all from Zephyr 4.4.2 builds.

A required property is missing:

```
devicetree error: 'pattern-name' is marked as required in 'properties:' in '.../↩
    ↪ bindings/dts/bindings/qacademy,blink-pattern.yaml', but does not appear in ↩
    ↪ <Node /pattern-heartbeat in .../bindings/app.overlay:18>
```

A value is not in the property's `enum`:

```
devicetree error: value of property 'level' on /probe in .../zephyr/zephyr.dts.pre
    ↪  ('fatal') is not in 'enum' list in .../qacademy,probe.yaml (['info', '
    ↪ warning', 'critical'])
```

A value has the wrong type. This one arrives at the end of a Python traceback, which is easy to mistake for a crash of the build system:

```
devicetree.dtlib.DTError: expected property 'limit' on /probe in .../type.overlay
    ↪ :1 to be assigned with 'limit = < (number) >;', not 'limit = "ten";'
```

And two that are not about bindings, but look the same:

```
devicetree error: .../label.overlay:1 (column 1): parse error: undefined node
    ↪ label 'gpio9'
devicetree error: .../syntax.overlay:1 (column 59): parse error: expected ';' or
    ↪ ','
```

A compatible also chooses a driver

For every compatible with an enabled node, the build defines a Kconfig symbol, `DT_HAS_<COMPATIBLE>_ENABLED`, and drivers depend on it. `native_sim`'s `gpio0` is why `build/zephyr/.config` has:

```
CONFIG_DT_HAS_ZEPHYR_GPIO_EMUL_ENABLED=y
CONFIG_GPIO_EMUL=y
```

`CONFIG_GPIO_EMUL` has `default y` and `depends on DT_HAS_ZEPHYR_GPIO_EMUL_ENABLED`: once GPIO is enabled, the driver is built because the devicetree has the part. It works in both directions. The `lectures/L02/examples/button` example's `buttons` node is `compatible = "gpio-keys"`, which makes `CONFIG_DT_HAS_GPIO_KEYS_ENABLED=y`, but the input subsystem is off, so no driver claims the button, and the example handles the pin itself. Build it with `-DCONFIG_INPUT=y`, and the `gpio-keys` input driver is enabled by the same rule, and refuses the node, which lacks the key code that driver needs:

```
error: static assertion failed: "zephyr-code must be specified to use the input-
    ↪ gpio-keys driver"
```

A `compatible` is not a comment. Choose it for the driver that should serve the node.

2.1.6 From devicetree to C

The generated header

The merged tree, checked against its bindings, is turned into **`build/zephyr/include/generated/zephyr/devicetree_generated.h`**: some five thousand lines of `#defines` for `native_sim`, included into every file through `<zephyr/devicetree.h>`. It is never read by hand in full, but it is worth reading in part, because every devicetree macro in C expands into one of its names.

Every node gets a **node identifier**, a C token made from its path: `DT_N`, then `_S_` for each slash, with the node names lowercased and every character that is not a letter or digit turned into `_`:

Path	Node identifier
`/`	`DT_N`
`/leds/led_0`	`DT_N_S_leds_S_led_0`
`/i2c@100`	`DT_N_S_i2c_100`
`/buttons/button_0`	`DT_N_S_buttons_S_button_0`

Everything about a node is then a macro named after its identifier. For the button, among many:

```
#define DT_N_S_buttons_S_button_0_P_gpios_IDX_0_PH DT_N_S_gpio_emul
#define DT_N_S_buttons_S_button_0_P_gpios_IDX_0_VAL_pin 1
#define DT_N_S_buttons_S_button_0_P_gpios_IDX_0_VAL_flags 17
#define DT_N_S_buttons_S_button_0_P_label "User button"
#define DT_N_S_buttons_S_button_0_STATUS_okay 1
#define DT_N_ALIAS_sw0         DT_N_S_buttons_S_button_0
#define DT_N_NODELABEL_button0 DT_N_S_buttons_S_button_0
```

Getting a node identifier

Nobody writes `DT_N_S_buttons_S_button_0`. These macros build it:

Macro	Finds the node by	Example
`DT_ALIAS(alias)`	Alias. The portable way.	`DT_ALIAS(sw0)`
`DT_NODELABEL(label)`	Label.	`DT_NODELABEL(button0)`
`DT_PATH(...)`	Path, one argument per node.	`DT_PATH(buttons, button_0)`
`DT_CHOSEN(prop)`	A `/chosen` property.	`DT_CHOSEN(zephyr_console)`
`DT_INST(inst, compat)`	Instance number of a compatible.	`DT_INST(0, gpio_keys)`

The arguments are written as C tokens: dashes and commas become underscores, so the alias `i2c-0` is `DT_ALIAS(i2c_0)`, the chosen property `zephyr,console` is `DT_CHOSEN(zephyr_console)`, and the compatible `qacademy,blink-pattern` is `qacademy_blink_pattern`. Most of these macros are one token paste each: `DT_ALIAS(sw0)` is `DT_N_ALIAS_sw0`, which is `DT_N_S_buttons_S_button_0`.

A node identifier is not a value. It cannot be stored in a variable, compared at run time, or chosen from a string: it is a token for the preprocessor to paste into other names. Everything in this section is resolved at compile time, and costs nothing at run time; the price is that a program cannot look up a node it did not name in its source.

Properties

With a node identifier, the property macros read the tree:

Macro	Result
`DT_PROP(node, prop)`	The value: an integer, a string literal, or for an array, a brace-enclosed initializer.
`DT_PROP_LEN(node, prop)`	The number of elements of an array, or of references in a phandle-array.
`DT_PROP_OR(node, prop, default)`	The value, or `default` if the node does not have the property.
`DT_NODE_HAS_PROP(node, prop)`	1 if the node has the property, else 0.
`DT_ENUM_IDX(node, prop)`	The index of the value in the binding's `enum`.
`DT_NODE_EXISTS(node)`	1 if the node exists, enabled or not.
`DT_NODE_HAS_STATUS_OKAY(node)`	1 if the node exists and is enabled.

Property names are written with underscores too: `delays-ms` is `DT_PROP(node, delays_ms)`. And since an array property is an initializer, it can initialize an array:

```
static const int blink_ms[] = DT_PROP(DT_PATH(zephyr_user), blink_ms);
```

A property that does not exist is a compile error naming the macro that is missing, which after this section is readable. With `blinks` missing from `zephyr,user`:

```
error: 'DT_N_S_zephyr_user_P_blinks' undeclared (first use in this function); did ↩
    ↪ you mean 'DT_N_S_zephyr_user_P_blink_ms'?
```

A missing alias fails the same way, one level earlier: `DT_ALIAS(led1)` expands to `DT_N_ALIAS_led1`, which nothing defines, and every macro built on it fails with it in its name, such as `'DT_N_ALIAS_led1_P_gpios_IDX_0_VAL_pin' undeclared`.

Every node of a kind

`DT_FOREACH_STATUS_OKAY(compat, fn)` expands `fn(node_id)` once for every enabled node with a compatible. The `lectures/L02/examples/bindings/src/main.c` example uses it to build a table at compile time, so that adding a node to the overlay adds a table entry, with no change to the C:

```
#define PATTERN_ENTRY(node_id)                                  \
    {                                                           \
```

```
        .name = DT_PROP(node_id, pattern_name),             \
        .led = GPIO_DT_SPEC_GET(node_id, led_gpios),        \
        .steps = DT_PROP_LEN(node_id, delays_ms),           \
        .repeat = DT_PROP(node_id, repeat),                 \
    },
static const struct pattern patterns[] = {
    DT_FOREACH_STATUS_OKAY(qacademy_blink_pattern, PATTERN_ENTRY)};
```

The macro is itself only a name in the generated header, holding the list:

```
#define DT_FOREACH_OKAY_qacademy_blink_pattern(fn) fn(DT_N_S_pattern_sos) fn(↩
    ↪ DT_N_S_pattern_heartbeat)
```

`fn` adds its own separators, here the trailing comma. Disabled nodes are left out, but they still exist: `DT_NODE_EXISTS(DT_NODELABEL(strobe))` is 1.

Every node with a compatible also has an **instance number**, the `inst` of `DT_INST()`. Zephyr's own documentation of `DT_INST()` says what they are and, more importantly, what they are not: enabled nodes are numbered from 0, disabled nodes after them, and *"there is no general guarantee that the same node will have the same instance number between builds"*. The order in which `DT_FOREACH_STATUS_OKAY()` visits the nodes is not guaranteed either. A table built with it is in the same order as the loop that reads it, which is all a program should rely on.

Checking the tree at compile time

Because all of this is compile-time constants, the build can check the devicetree against what the code assumes, with `BUILD_ASSERT()`:

```
BUILD_ASSERT(DT_PROP(DT_NODELABEL(heartbeat), repeat) == 2, "heartbeat should play↩
    ↪  twice");
```

2.1.7 Devices

A device per node

A **device** is Zephyr's run-time object for a piece of hardware with a driver: a `struct device`, with a name, an API, configuration, state, and an init function the kernel runs at boot. Drivers define one for every enabled node of their compatible (Chapter 5 writes one), and C gets a pointer to it from the node:

```
const struct device *gpio = DEVICE_DT_GET(DT_NODELABEL(gpio0));
```

`DEVICE_DT_GET()` is a compile-time address, not a search. The device's name, `gpio->name`, is the node's `label` property if it has one, and otherwise its name: `gpio_emul`.

A device can exist and still not be usable, because its init function ran and failed: a chip did not answer, a clock did not start. `device_is_ready()` returns true only for a device that initialized successfully, and code that uses a device checks it first. The GPIO API has a shorthand for a pin, `gpio_is_ready_dt(&spec)`.

Ordinals, and the two errors

Nodes are numbered in **dependency order**: a node comes after every node it refers to, so that a device is always initialized after the devices it depends on. The list is at the top of the generated header:

```
 * Node dependency ordering (ordinal and path):
 *   0   /
...
 *   22  /gpio_emul
 *   23  /buttons
 *   24  /buttons/button_0
```

and each node lists what it depends on:

```
#define DT_N_S_buttons_S_button_0_REQUIRES_ORDS \
        22, /* /gpio_emul */ \
        23, /* /buttons */
```

The device object for a node is named after its ordinal: `DEVICE_DT_GET()` of `/gpio_emul` in this build is `&__device_dts_ord_22`. **Ordinals belong to one build.** `/gpio_emul` is 22 in the `button` example's build, 32 in `blinky`'s, and something else again once an overlay adds a node. An ordinal in an error message means nothing until it is looked up in *that* build's header.

That name is what appears in the two errors a missing device causes, and the difference between them says where to look. The `lectures/L02/examples/blinky` example's live demo produces both:

```
error: '__device_dts_ord_32' undeclared here (not in a function); did you mean '↪
    ↪ __device_dts_ord_38'?
```

Undeclared is a compile error. A device object is declared only for an enabled node, so the node exists but is disabled, or is not what the code thinks it is. Look at the devicetree. (The *did you mean* is noise: 38 is only the nearest number that does exist.) CMake has usually warned already, in a line easy to scroll past: `No SOURCES given to Zephyr library: drivers__gpio`.

```
undefined reference to `__device_dts_ord_32'
```

Undefined reference is a link error. The node is enabled, so the object was declared, but no driver defined it: the driver was not built. Look at Kconfig, here `CONFIG_GPIO=n`.

2.1.8 GPIO

A pin, from the devicetree

A GPIO pin is a controller device, a pin number and flags, and `struct gpio_dt_spec` holds exactly that:

```
struct gpio_dt_spec {
        const struct device *port;
        gpio_pin_t pin;
        gpio_dt_flags_t dt_flags;
};
```

`GPIO_DT_SPEC_GET(node, prop)` fills one from a phandle-array property, entirely at compile time:

```
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);
```

For the board's `led0`, that is `gpio_emul`'s device, pin 0, and flags 0. For a property with another name, the name goes in: `GPIO_DT_SPEC_GET(node_id, led_gpios)`.

Logical and physical

The GPIO functions ending in `_dt` take a `gpio_dt_spec`, and work in **logical** levels: 1 means *active*, whatever that is on the pin. The flags from the devicetree decide what *active* is. For an LED declared `GPIO_ACTIVE_LOW`, on `native_sim`:

Call	Logical	Pin
`gpio_pin_configure_dt(&led, GPIO_OUTPUT_INACTIVE)`	0	high
`gpio_pin_set_dt(&led, 1)`	1	low
`gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE)`	1	low

That is how the C code stays the same when a board wires its LED the other way round: *on* is always 1, and only the devicetree knows whether 1 is high. Functions with `_raw` in their name, such as `gpio_pin_set_raw()`, bypass the flags and set the pin itself; use them only when the level on the wire is what matters.

The flags given to `gpio_pin_configure_dt()` are combined with the flags from the devicetree, so `GPIO_INPUT` on a button declared `GPIO_PULL_UP | GPIO_ACTIVE_LOW` configures an input with its pull-up enabled.

Interrupts and callbacks

A button is read by interrupt rather than by polling. Three calls set it up, as in the `lectures/L02/examples/button/src/main.c` example:

```
gpio_pin_interrupt_configure_dt(&button, GPIO_INT_EDGE_TO_ACTIVE);
gpio_init_callback(&button_cb, button_pressed, BIT(button.pin));
gpio_add_callback_dt(&button, &button_cb);
```

`GPIO_INT_EDGE_TO_ACTIVE` is logical too: an interrupt when the button becomes pressed, which is a falling edge for an active-low button and a rising one for an active-high button. `gpio_init_callback()` sets up a `struct gpio_callback` with the function to call and the pins it is for, and `gpio_add_callback_dt()` adds it to the controller's list. The function receives the controller, the callback, and a mask of the pins that fired:

```
static void button_pressed(const struct device *port, struct gpio_callback *cb,
                           gpio_port_pins_t pins)
```

On a microcontroller, the callback runs in the GPIO controller's interrupt service routine. It must not sleep, block or take long; Chapter 3 covers what it may do instead.

The emulated controller

`native_sim` has no pins, so `gpio_emul` keeps them in memory, and has an API of its own, in `<zephyr/drivers/gpio/gpio_emul.h>`, for a program or test to play the world outside the chip:

```
int gpio_emul_input_set(const struct device *port, gpio_pin_t pin, int value);
int gpio_emul_output_get(const struct device *port, gpio_pin_t pin);
```

Both take and return **physical** levels, not logical ones. To press an active-low button, a program drives its pin *low*; to find out whether an active-low LED is on, it checks for a *low* output. Both have `_dt` variants taking a `gpio_dt_spec`, which are still physical. These functions exist only on the emulator, and are the lines of the examples that would not build for a real board.

Two of the emulator's behaviours match hardware, and one does not:

- **An interrupt needs an edge.** Setting an input to the level it already has fires nothing, and an input configured for `GPIO_INT_EDGE_TO_ACTIVE` fires nothing when it becomes inactive.
- **Pull resistors set the level.** Configuring an input with `GPIO_PULL_UP` sets its level high, and `GPIO_PULL_DOWN` low, as a real resistor would on an open pin.
- **The callback runs in the context that changed the pin.** `gpio_emul_input_set()` calls the callbacks itself, before it returns. Called from a thread, the callback runs in that thread, and `k_is_in_isr()` in it is false; called from a timer's expiry function, which runs in interrupt context, it runs in interrupt context. On a microcontroller it would always be an interrupt. Write callbacks as if they were: the emulator will not catch one that sleeps.

2.1.9 Summary

- Devicetree describes the hardware, and C code refers to it by role. Zephyr turns the tree into C macros at build time, so there is nothing to parse at run time and nothing to change without a rebuild.
- A node has a path, labels, and possibly aliases pointing at it. `/aliases` gives applications portable names; `/chosen` tells Zephyr which nodes to use.
- Overlays add, change and delete nodes. A build uses one set of overlays, `DTC_OVERLAY_FILE`, a board overlay, or `app.overlay`, and merges `EXTRA_DTC_OVERLAY_FILE` on top. `build/zephyr/zephyr.dts` shows the result, and where each line came from.
- Bindings give nodes their meaning: property types, required properties, defaults and enums. Only declared properties reach C, except in `/zephyr,user`. A binding's errors come early and name the line.
- A `compatible` chooses a node's binding, and through `DT_HAS_<COMPAT>_ENABLED`, its driver.
- `DT_ALIAS()`, `DT_NODELABEL()` and friends give node identifiers, which are compile-time tokens. `DT_PROP()` and friends read properties, and `DT_FOREACH_STATUS_OKAY()` builds tables. Do not rely on instance numbers or on the order of the loop.
- `DEVICE_DT_GET()` gives a device, and `device_is_ready()` says whether it can be used. `__device_dts_ord_N` *undeclared* means a disabled node; *undefined reference* means a driver that was not built. Look N up in that build's `devicetree_generated.h`.
- GPIO through `gpio_dt_spec` is logical: 1 is active, and the devicetree says what active is. `gpio_emul`'s own functions are physical, and its callbacks run in the context that changed the pin.

2.2 Review

What you should be able to do now

After this chapter, you should:

- Understand how devicetree separates *what hardware there is* from *the code that uses it*.
- Be able to read a board's devicetree, and the merged result of a build in `zephyr.dts`.
- Be able to add and change nodes and properties with an overlay.
- Be able to write a binding for a node of your own.
- Be able to get devices and properties from the devicetree in C, entirely at compile time.
- Be able to drive and read GPIO pins, and react to edges with a callback.
- Be able to trace a devicetree-related build error back to the node that caused it.

Questions to test yourself

You should be able to explain:

1. What is the difference between a node's name, its label, and an alias?
2. What does a binding add to a devicetree node, and what happens to a node whose binding is missing?
3. Why must a `compatible` string's vendor prefix be known to the build?
4. How does an overlay change a board's devicetree, and where can you see the merged result?
5. Why is `DT_PROP(node, prop)` resolved at compile time, and what does that make impossible?
6. What does `device_is_ready()` check, and why must a driver's user call it?
7. What does `GPIO_ACTIVE_LOW` in the devicetree change in the C code that uses the pin?
8. What is the difference between `'__device_dts_ord_32' undeclared` and `undefined reference to '__device_dts_ord_32'`, and what does the number tell you?

Looking ahead

Kernel I, threads and time:

- Threads, priorities, and cooperative and preemptive scheduling.
- Timers, and which context their expiry functions run in.
- Work queues, and deferring work out of interrupt context.
- What `native_sim`'s simulated time means for code that spins, and for stack sizes.

You will trace, line by line, which thread runs when, and why.

2.3 Exercises

These exercises reinforce the concepts from §2.1.

> **How to check your work.** Write each exercise set as its own application under `lectures/L02/exercises`, with the file names given there, and run `make test` from the repository root. A set's tests switch on as soon as its files exist, and a set you have not started is reported as `SKIP`. Each application grows through its set, and the tests check where it ends up. Every set is built twice: once as you wrote it, and once with an extra overlay from the test suite merged on top, which moves pins, turns polarities around, and adds or removes nodes. A program that takes its hardware from the devicetree prints the second expected output without a change; a program with a pin number or a polarity written into its C code does not.
> The solutions are published in the companion repository, under `lectures/L02/appendix/solutions`, a few days after the lecture. Try each exercise yourself before you read them.

Remember `source env.sh` in every new terminal. Build and run an exercise from the repository root:

```
west build -p -b native_sim/native/64 lectures/L02/exercises/exercise1
west build -t run
```

To build with the same extra overlay as the tests, from the repository root:

```
west build -p -b native_sim/native/64 lectures/L02/exercises/exercise1 -- \
    -DEXTRA_DTC_OVERLAY_FILE=$PWD/lectures/L02/exercises/test/exercise1-extra.overlay
```

Print with `printk()` in every set, so that the output has no timestamps other than the ones the exercise asks for.

EXERCISE SET 1

The board's LED

Create an application named `exercise1` to work in for this exercise set, with a `CMakeLists.txt` and `src/main.c` as in Chapter 1, and a `prj.conf` that enables GPIO (`CONFIG_GPIO=y`).

Exercise 2.1 Finding the LED

CODE

In this exercise you will get the board's LED from the devicetree.

Tasks

- In `main.c`, get a `struct gpio_dt_spec` for the `gpios` property of the node with the alias `led0`, at file scope.
- In `main()`, check that the LED's controller is ready. If it is not, print `GPIO controller <name> is not ready` and return.
- Print the LED's pin, its controller's name, and whether it is active high or active low, from the spec's flags:

  ```
  LED: pin 0 on gpio_emul, active high
  ```

- Configure the pin as an output, initially inactive.

Reflection

- Find `led0` in `deps/zephyr/boards/native/native_sim/native_sim.dts`. Which line is the alias, and which is the label? Which of the two did your code use?
- Where does the name `gpio_emul` come from?
- What would the same program print on a board whose `led0` is declared `<&gpio1 5 GPIO_ACTIVE_LOW>`, and what in your code would change?

Exercise 2.2 Blinking as the devicetree says

CODE

Tasks

- Create `app.overlay` in `exercise1`, adding a `zephyr,user` node under the root with two properties:
  - `blink-ms = <200 300>;`: how long the LED is on, and then off, in milliseconds.
  - `blinks = <3>;`: how many times to blink.
- In `main.c`, read `blink-ms` into a `static const int` array at file scope, and use `blinks` directly where you need it.
- After configuring the pin, blink the LED `blinks` times. For each blink:
  - Turn the LED on with `gpio_pin_set_dt()`, print `LED on, pin <level> at <uptime> ms`, and sleep for the on time.
  - Turn the LED off, print `LED off, pin <level> at <uptime> ms`, and sleep for the off time.

  `<level>` is `high` or `low`: what the emulated controller is driving on the pin, from `gpio_emul_output_get()` in `<zephyr/drivers/gpio/gpio_emul.h>`.
- Finally print `Done after <blinks> blinks`.

The output must be:

```
LED: pin 0 on gpio_emul, active high
LED on, pin high at 0 ms
LED off, pin low at 210 ms
LED on, pin high at 520 ms
LED off, pin low at 730 ms
LED on, pin high at 1040 ms
LED off, pin low at 1250 ms
Done after 3 blinks
```

The tests build the application a second time with `lectures/L02/exercises/test/exercise1-extra.overlay`, which points `led0` at a new, active-low LED on pin 5, and blinks it twice, for 50 and 150 ms. The output must then be:

```
LED: pin 5 on gpio_emul, active low
LED on, pin low at 0 ms
LED off, pin high at 60 ms
LED on, pin low at 220 ms
LED off, pin high at 280 ms
Done after 2 blinks
```

Hint: `DT_PATH(zephyr_user)` is the node identifier of `/zephyr,user`. An array property is an initializer: `static const int blink_ms[] = DT_PROP(node, blink_ms);`.

Reflection

- You wrote no binding for `zephyr,user`. How did the build know the types of its properties?
- Remove `blinks` from the overlay and build. Explain the error. Then make the program blink once when `blinks` is missing, without adding it back.
- In the second build, the LED is *on* while the pin is *low*. Which line of your code decided that, and why did it not need to change?
- In the second build, find `blink-ms` in `build/zephyr/zephyr.dts`. How does the file say which overlay the value came from, and why is it not `app.overlay`, which also sets it?

EXERCISE SET 2

A button of your own

Create an application named `exercise2` to work in for this exercise set, with the same `CMakeLists.txt` and `prj.conf` as `exercise1`.

Exercise 2.3 Describing the hardware

CODE

In this exercise you will add hardware the board does not have, with an overlay.

Tasks

Create `app.overlay` in `exercise2`, which adds:

- A second LED, as a child of the board's existing `leds` node: pin 2 of `gpio0`, active high, with the node label `led1`.
- A node named `buttons` under the root, with `compatible = "gpio-keys"`, holding one button: pin 1 of `gpio0`, active low with a pull-up, with the node label `button0`.
- The aliases `led1` for the new LED and `sw0` for the button.

Use exactly these node labels: the test suite's overlay refers to them.

Reflection

- Why does the new LED need no `compatible` property of its own?
- Build the application and open `build/zephyr/zephyr.dts`. What are the button's flags, in hex, and how are they made up?

Exercise 2.4 The callback

CODE

Tasks

In `main.c`:

- Get `gpio_dt_specs` for the aliases `led1` and `sw0`, and check that both are ready.
- Print `Button on pin <pin>, LED 1 on pin <pin>`.
- Configure LED 1 as an output, initially inactive, and the button as an input.
- Configure an interrupt on the button for when it becomes **active**, and add a callback for it.
- In the callback, count the press, toggle a `bool` that says whether LED 1 is on, set LED 1 with `gpio_pin_set_dt()`, and print:

  ```
  Press <count> at <uptime> ms, LED 1 <on|off>, pin <high|low>
  ```

  where the pin's level is read back with `gpio_emul_output_get()`.

Exercise 2.5 The finger

CODE

Tasks

Nobody can press an emulated button, so `main()` does it:

- Write a function `static void press(bool pressed)` that sets the button's input with `gpio_emul_input_set()`. It takes whether the button is pressed, and must work out the **level on the pin** from the button's flags.
- After setting up the callback, `main()` presses the button three times. Before each press it sleeps for the next value of `static const int gap_ms[] = {100, 200, 200};`, then presses, sleeps 50 ms, and releases.
- Finally print `<count> presses counted`.

The output must be:

```
Button on pin 1, LED 1 on pin 2
Press 1 at 110 ms, LED 1 on, pin high
Press 2 at 380 ms, LED 1 off, pin low
Press 3 at 650 ms, LED 1 on, pin high
3 presses counted
```

The second build merges `lectures/L02/exercises/test/exercise2-extra.overlay`, which moves the button to pin 7 as an active-high button with a pull-down, and LED 1 to pin 3 as an active-low LED. The output must then be:

```
Button on pin 7, LED 1 on pin 3
Press 1 at 110 ms, LED 1 on, pin low
Press 2 at 380 ms, LED 1 off, pin high
Press 3 at 650 ms, LED 1 on, pin low
3 presses counted
```

Hint: An active-low button is pressed when its pin is low.

Reflection

- Make `press()` always drive the pin low for a press, and run the second build. At what times are the presses counted now, and why?
- Print `k_is_in_isr()` in the callback. What does it print, and what would the same callback run in on a microcontroller? What does that forbid in the callback?
- Nothing happens when the finger releases the button. Why not?
- The button's input level is high before the first press, although nothing in your code set it. What did?
- Build with `-DDTC_OVERLAY_FILE=` and the path to `exercise2-extra.overlay`, instead of `-DEXTRA_DTC_OVERLAY_FILE=`. What happens, and why?

EXERCISE SET 3

A binding of your own

Create an application named `exercise3` to work in for this exercise set, with the same `CMakeLists.txt` and `prj.conf` as `exercise1`.

Exercise 2.6 The binding

CODE

In this exercise you will define a kind of node of your own.

Tasks

Create `dts/bindings/qacademy,alarm.yaml` in `exercise3`: a binding for the compatible `qacademy,alarm`, a temperature alarm with an LED, which includes `base.yaml` and has these properties:

Property	Type	Required	Meaning
`alarm-name`	string	yes	Human-readable name.
`threshold`	int	yes	The temperature, in degrees Celsius, at which the alarm goes on.
`hysteresis`	int	no, default 2	How many degrees below the threshold the temperature must fall before the alarm goes off.
`severity`	string	yes	One of `info`, `warning` or `critical`, in that order.
`led-gpios`	phandle-array	yes	The LED that shows the alarm.

Create `dts/bindings/vendor-prefixes.txt`, registering the vendor prefix `qacademy` (the prefix, a tab, and a vendor name).

Exercise 2.7 The alarms

CODE

Tasks

Create `app.overlay` in `exercise3`, adding three nodes under the root, in this order. Every alarm's LED is on `gpio0` and active high, and `alarm-fan` is the only node with `status = "disabled"`:

Node name	Label	`alarm-name`	`threshold`	`hysteresis`	`severity`	LED pin
`alarm-overheat`	`overheat`	`Overheat`	70	not set	`critical`	10
`alarm-warm`	`warm`	`Warm`	45	5	`warning`	11
`alarm-fan`	`fan`	`Fan`	25	not set	`info`	12

Reflection

- Remove `severity` from one node and build. Then set it to `"fatal"` and build. Which step of the build reports each error, and has any C been compiled yet?
- Delete `vendor-prefixes.txt` and build. What changes?

Exercise 2.8 The table

CODE

Tasks

In `main.c`:

- Define a `struct alarm` holding the name, threshold, hysteresis, severity (as an `int`), a `struct gpio_dt_spec` for the LED, and a `bool` saying whether the alarm is on.
- Build an array of them, `alarms`, with **one entry for each enabled `qacademy,alarm` node**, using `DT_FOREACH_STATUS_OKAY()`. Store the severity as its index in the binding's `enum`, and keep a table of the names `"info"`, `"warning"` and `"critical"` to print it.
- Print `<count> alarms`, and then, for each alarm:

  ```
  <name>: <severity> at <threshold> C, hysteresis <hysteresis>, LED pin <pin>
  ```

  and configure its LED as an output, initially inactive.
- Go through the readings `static const int readings[] = {20, 50, 42, 75, 66, 30};`. For each, print `Reading <t> C`, and then for each alarm, in table order:
  - An alarm that is off goes on if the reading is at or above its threshold.
  - An alarm that is on goes off if the reading is below its threshold minus its hysteresis.
  - When an alarm changes, set its LED, and print, indented by two spaces:

    ```
      <name> (<severity>) <on|off>, pin <pin> <high|low>
    ```

  The output must be:

```
2 alarms
Overheat: critical at 70 C, hysteresis 2, LED pin 10
Warm: warning at 45 C, hysteresis 5, LED pin 11
Reading 20 C
Reading 50 C
  Warm (warning) on, pin 11 high
Reading 42 C
Reading 75 C
  Overheat (critical) on, pin 10 high
Reading 66 C
  Overheat (critical) off, pin 10 low
Reading 30 C
  Warm (warning) off, pin 11 low
```

The second build merges `lectures/L02/exercises/test/exercise3-extra.overlay`, which deletes the `warm` node, enables `fan`, and changes `overheat`'s threshold to 60 and hysteresis to 10. The output must then be:

```
2 alarms
Overheat: critical at 60 C, hysteresis 10, LED pin 10
Fan: info at 25 C, hysteresis 2, LED pin 12
Reading 20 C
Reading 50 C
  Fan (info) on, pin 12 high
Reading 42 C
Reading 75 C
  Overheat (critical) on, pin 10 high
Reading 66 C
Reading 30 C
  Overheat (critical) off, pin 10 low
```

Hint: `DT_ENUM_IDX(node_id, severity)` is the index of the node's `severity` in the binding's `enum`. The compatible, written as a C token, is `qacademy_alarm`.

Reflection

- Overheat's hysteresis is 2, although its node does not set it. Where does the 2 come from, and what would happen without it in the binding?
- In what order are the alarms in the table? Find the list in `build/zephyr/include/generated/zephyr/devicetree_generated.h`. Can a program rely on that order?
- `alarm-fan` is disabled in the first build. Which instance number does it have, and does `DT_NODE_EXISTS(DT_NODELABEL(fan))` give 1 or 0?
- Add `BUILD_ASSERT(DT_PROP(DT_NODELABEL(warm), hysteresis) == 5);` to `main.c`, and run the second build. What happens, and why is the error not an assertion failure?
- The table is not `const`, because each alarm's `active` changes. Which fields could be `const`, and how would you split the struct so that they are?

EXERCISE SET 4

Reading the devicetree

No code in this set: answer the questions by looking. Build `exercise1` and `exercise2` into separate build directories first, e.g. with `-d build/exercise1` and `-d build/exercise2`.

Exercise 2.9 The merged tree

REFLECTION

Tasks

Answer, using `zephyr.dts` in the `exercise2` build:

- Which file and line defined the alias `sw0`? And the alias `led0`?
- What does `/chosen` say the console is, and what is the node's compatible?

Exercise 2.10 The generated header

REFLECTION

Tasks

Answer, using `devicetree_generated.h`:

- What is the node identifier of the button in `exercise2`, and what does `DT_ALIAS(sw0)` expand to, step by step?
- What is the ordinal of `/gpio_emul` in the `exercise1` build, and in the `exercise2` build? Why can they differ?
- Which ordinals does the button's node depend on directly, and why those?

Exercise 2.11 Two errors, one number

REFLECTION

Tasks

- Build `exercise1` with `-DEXTRA_DTC_OVERLAY_FILE=` and the path to `lectures/L02/examples/blinky/broken.overlay`. Which error do you get, and which node does its number name?
- Build `exercise1` with `-DCONFIG_GPIO=n` instead. Which error do you get now, and what is the difference in what went wrong?

CHAPTER 3

Kernel I: Threads, Time and Work Queues

Threads, time, and what a simulator will not pretend.

In this chapter

- Threads in Zephyr: `K_THREAD_DEFINE()`, `k_thread_create()`, stacks, names and states.
- Priorities: cooperative (negative) and preemptible (non-negative) threads, and the `main` thread.
- When the scheduler runs: blocking, `k_yield()`, and a higher-priority thread becoming ready.
- Time: ticks, timeouts, `k_sleep()`, `k_uptime_get()`, and why a sleep ends on a tick.
- Kernel timers, and expiry functions that run in interrupt context.
- Work queues: the system work queue, delayable work, and work queues of your own.
- Choosing between an ISR, a work item and a thread.
- `native_sim`'s time model: why a busy loop stops the world, and what that says about real hardware.
- Stack sizing without a stack analyzer: `CONFIG_STACK_USAGE` and `-fstack-usage`.

Here the kernel starts. If you have used an RTOS before, most of this chapter is a translation exercise: threads, priorities, sleeping and timers, in Zephyr's spelling. What is worth slowing down for is the part that is not a translation, namely when exactly the scheduler runs, which context a timer's expiry function is in, and why so much Zephyr code answers an interrupt by submitting a work item instead of doing the work.

The chapter ends where `native_sim` stops behaving like a microcontroller. Its time is simulated, so a thread that never waits stops the clock for everything, and the stack checks a board would give you are not there. Both are worth knowing exactly, and both are on purpose in `lectures/L03/examples/busy_loop` and `lectures/L03/examples/stack_usage`.

3.1 Threads, Time and Work Queues

This section covers the part of the kernel that decides what runs when: threads and their priorities, the scheduler's rules, time and timeouts, kernel timers, interrupt context, and work queues. It ends with the two places where `native_sim` behaves differently from a microcontroller: code that spins, and stacks. By the end you should be able to predict, line by line, the traces of the `threads` and `timer_work` examples, and explain why `busy_loop` never finishes. All four of this chapter's examples are in `lectures/L03/examples`.

If you have taken **Real-Time Kernel Design**, you have built everything below yourself. Zephyr's version has more features and different names, but the same shape.

3.1.1 Threads

What a thread is

A Zephyr thread is:

- an **entry function**, `void entry(void *p1, void *p2, void *p3)`, with three arguments of the caller's choosing;
- a **stack** of its own, whose size is fixed when the thread is defined;
- a **`struct k_thread`**, the kernel's bookkeeping: saved registers, state, priority, name;
- a **priority**, and a few **options**.

Returning from the entry function ends the thread. There is no process around it and no memory protection between threads on most targets: every thread sees the same global variables.

Creating a thread at build time

Most threads in an embedded application exist for the whole of its life, and are best defined statically:

```
K_THREAD_DEFINE(sensor, 1024, sensor_entry, NULL, NULL, NULL, 3, 0, 0);
```

The arguments are the thread's name, its stack size in bytes, the entry function and its three arguments, the priority, the options, and a **start delay** in milliseconds. `K_THREAD_DEFINE()` allocates the stack and the `struct k_thread` at compile time; nothing is allocated at run time. The name becomes both a C identifier, `sensor`, of type `k_tid_t`, and the thread's name for `k_thread_name_get()` when `CONFIG_THREAD_NAME=y`.

Statically defined threads are started by the kernel **after every `APPLICATION`-level initialization and immediately before `main()` is called**, in the `main` thread (`z_init_static_threads()` in `deps/zephyr/kernel/init.c`). It starts them with the scheduler locked, so that none of them runs until all of them have been started, and then unlocks it. From that moment, the scheduler's ordinary rules decide who runs first, `main` included.

A delay of `K_FOREVER` instead of a number creates the thread without starting it; `k_thread_start()` starts it later.

Creating a thread at run time

A thread whose existence depends on run-time conditions is created with `k_thread_create()`. Its stack must still be allocated, usually statically:

```
K_THREAD_STACK_DEFINE(worker_stack, 1024);
static struct k_thread worker_thread;

k_tid_t tid = k_thread_create(&worker_thread, worker_stack, K_THREAD_STACK_SIZEOF(↩
    ↪ worker_stack),
                              worker_entry, NULL, NULL, NULL, 5, 0, K_NO_WAIT);
k_thread_name_set(tid, "worker");
```

Use `K_THREAD_STACK_DEFINE()` and `K_THREAD_STACK_SIZEOF()` rather than a `char` array and `sizeof`: on some targets a thread stack needs guard areas and alignment that the macros add. `k_thread_join()` waits for a thread to end, and `k_thread_abort()` ends one from outside.

Thread states

At any moment a thread is in one of these states (the flags are in `deps/zephyr/include/zephyr/kernel_structs.h`):

State	Meaning
Ready	Could run, and is waiting for the scheduler to pick it.
Running	Has the CPU. One thread per CPU.
Pending	Waiting for a kernel object: a semaphore, a queue, a mutex, a timer's status.
Sleeping	Waiting for time to pass, in `k_sleep()` and its relatives.
Suspended	Stopped by `k_thread_suspend()` until `k_thread_resume()`.
Dead	Returned from its entry function, or aborted.

Pending and sleeping threads are together called **blocked**. The scheduler only ever chooses among ready threads.

3.1.2 Priorities

Every thread has an integer priority, and **a lower number is a higher priority**. The range is split in two:

Priorities	Kind	Defaults
-16 to -1	**Cooperative**	`CONFIG_NUM_COOP_PRIORITIES=16`. `K_PRIO_COOP(0)` is -16, `K_PRIO_COOP(15)` is -1.
0 to 14	**Preemptible**	`CONFIG_NUM_PREEMPT_PRIORITIES=15`. `K_PRIO_PREEMPT(x)` is `x`.
15	The idle thread	`K_IDLE_PRIO`, lower than any application thread.

The difference between the two kinds is the most important thing in this chapter:

- A **cooperative** thread, once running, keeps the CPU until it gives it up itself: by blocking, sleeping, yielding or returning. Not even a cooperative thread of higher priority can take it.
- A **preemptible** thread loses the CPU the moment a thread of **higher** priority becomes ready.

Interrupts are not threads, and interrupt every thread, cooperative or not. What a cooperative thread is protected from is other *threads* running before it gives up the CPU, even if an interrupt made them ready.

A few threads have priorities worth knowing:

Thread	Priority	Set by
`main`	0	`CONFIG_MAIN_THREAD_PRIORITY`
The system work queue, `sysworkq`	-1	`CONFIG_SYSTEM_WORKQUEUE_PRIORITY`
Idle	15	`K_IDLE_PRIO`

`k_thread_priority_set()` changes a thread's priority at run time.

Meta-IRQ priorities are the highest cooperative priorities, reserved with `CONFIG_NUM_METAIRQ_PRIORITIES` (0 by default, so there are none). A ready meta-IRQ thread runs even before a running cooperative thread, and even when the scheduler is locked. They exist for drivers that need a thread which behaves almost like an interrupt handler, and applications should not need them.

3.1.3 When the scheduler runs

The rule, with the exception that makes it interesting: **the running thread is the highest-priority ready thread, unless the running thread is cooperative**. The scheduler applies it at a small number of moments:

Moment	What can happen
The running thread **blocks, sleeps or returns**	The highest-priority ready thread runs, whatever the kind of the thread that stopped.
The running thread **makes another thread ready**, e.g. with `k_sem_give()`	If the running thread is preemptible and the other thread has a higher priority, the switch happens *inside* `k_sem_give()`, before it returns.
An **interrupt makes a thread ready**, e.g. a timer expires	Same rule, when the interrupt handler returns.
The running thread calls **`k_yield()`**	Another ready thread of the **same or higher** priority runs, if there is one. If there is none, `k_yield()` returns at once.
A **time slice** ends	With `CONFIG_TIMESLICING=y` (the default), a preemptible thread that has run for `CONFIG_TIMESLICE_SIZE` ms (20 by default) gives way to a ready thread of the same priority. Priorities above `CONFIG_TIMESLICE_PRIORITY` (0) are not sliced.

Two rules about equal priorities follow from the table and surprise people. A thread made ready does **not** preempt a running thread of its own priority: it waits until the running thread blocks, yields or runs out of its time slice. And `k_yield()` never gives the CPU to a lower priority, which is why it cannot be used to let a lower-priority thread run.

`k_sched_lock()` makes the running thread behave as if it were cooperative until `k_sched_unlock()`. It is how the kernel starts the static threads, and a cheap way to keep a short sequence from being interleaved with other threads. Interrupts still run.

The `threads` example shows each of these moments in turn, as a trace that is the same on every run.

3.1.4 Time

Ticks

The kernel counts time in **ticks** of its system clock, and every timeout is kept as a number of ticks. `CONFIG_SYS_CLOCK_TICKS_PER_SEC` sets the rate, and its defaults, in `deps/zephyr/kernel/Kconfig`, are 100 for `native_sim` and QEMU, which makes a tick 10 ms, and 10000 for a tickless kernel (`CONFIG_TICKLESS_KERNEL`), as on many boards. `native_sim` is tickless too, but of the two defaults the first one that applies wins, so its tick stays 10 ms. **Nothing about time is finer than a tick**: a 25 ms timeout on `native_sim` becomes three ticks.

`k_uptime_get()` returns the milliseconds since boot, and `k_uptime_ticks()` the ticks.

Timeouts

Every call that can wait takes a `k_timeout_t`:

Timeout	Meaning
`K_NO_WAIT`	Do not wait: succeed at once or fail at once.
`K_FOREVER`	Wait for as long as it takes.
`K_MSEC(ms)`, `K_SECONDS(s)`, `K_USEC(us)`, `K_TICKS(t)`	Wait at most this long, **relative** to now.
`K_TIMEOUT_ABS_MS(ms)`, `K_TIMEOUT_ABS_TICKS(t)`	Wait until the uptime reaches this value. Needs `CONFIG_TIMEOUT_64BIT`, which is on by default.

`k_msleep(ms)` is `k_sleep(K_MSEC(ms))`.

Why a relative timeout lasts one tick longer

A thread that asks to sleep for three ticks does so at some moment *during* the current tick, and the kernel does not know how much of that tick has already passed. To guarantee **at least** the time asked for, it adds one tick. The line in `z_add_timeout()`, in `deps/zephyr/kernel/timeout.c`, is:

```
to->dticks = timeout.ticks + 1 + ticks_elapsed;
```

So on `native_sim`, `k_msleep(30)` returns after 40 ms, and `k_msleep(500)` after 510. The duration is first rounded **up** to whole ticks, so `k_msleep(25)` also returns after 40 ms.

A loop that sleeps for its period after doing its work therefore **drifts**: each round starts a tick later than the one before, plus however long the work took. **An absolute timeout does not drift.** It needs no extra tick, since the deadline is a fixed point, so a loop that sleeps until the next multiple of its period wakes exactly on it:

```
for (int64_t next = period_ms; ; next += period_ms) {
        k_sleep(K_TIMEOUT_ABS_MS(next));
        do_work();
}
```

If the deadline has already passed when the thread goes to sleep, the thread wakes at the next tick: `z_add_timeout()` waits `max(1, dticks)` ticks for an absolute timeout. A periodic thread that falls behind therefore catches up one tick at a time, rather than all at once.

Busy waiting

`k_busy_wait(usec)` keeps the CPU busy for a number of microseconds, without giving it up. It is for delays too short for a tick, such as the few microseconds a peripheral needs between two register writes, and is almost never the right tool for anything longer. Two properties matter later. It measures elapsed time, not time spent running, so a thread preempted during a busy wait does not get the lost time back. And, unlike an ordinary loop, it lets simulated time pass on `native_sim`.

3.1.5 Kernel timers

A **kernel timer** calls a function when a timeout expires, once or periodically:

```
static void sample_expired(struct k_timer *timer);

K_TIMER_DEFINE(sample_timer, sample_expired, NULL);

k_timer_start(&sample_timer, K_MSEC(100), K_MSEC(100));
```

`k_timer_start()` takes the delay to the first expiry and the period after it; a period of `K_NO_WAIT` makes the timer one-shot. `k_timer_stop()` stops it. The two functions given to `K_TIMER_DEFINE()` are both optional:

- The **expiry function** runs *in the system clock's interrupt handler*, every time the timer expires. It is interrupt context, with every restriction the next section lists.
- The **stop function** runs when the timer is stopped with `k_timer_stop()` while running, in the context of whoever called `k_timer_stop()`.

A thread can also wait for a timer instead of giving it a function: `k_timer_status_sync()` blocks until the timer expires, and returns at once if it has already expired since its status was last read, or is stopped. `k_timer_status_get()` returns how many times the timer has expired since then, without blocking. Both reset the count.

Timers do not add the extra tick. `k_timer_start()` subtracts one tick from a relative delay before handing it to `z_add_timeout()`, which adds it back, and every period is counted from the previous expiry rather than from when the expiry function happened to run. The comment in `deps/zephyr/kernel/timer.c` explains that the period "is always guaranteed to be reset from within the timer ISR, so no round up is desired and 1 is subtracted in there". That is why a 500 ms one-shot timer started at 0 ms expires at exactly 500 ms, and a periodic 100 ms timer at 100, 200 and 300 ms, with no drift, while `k_msleep(500)` returns at 510 ms.

3.1.6 Interrupt context

Code in an interrupt handler runs *on top of* whatever thread was running, on a separate stack (`CONFIG_ISR_STACK_SIZE`, 2048 bytes by default). It is not a thread, so **it must never block or sleep**: there is no thread to put to sleep, and the thread underneath has nothing to do with it. `k_is_in_isr()` tells code which context it is in.

What that means in practice:

- A kernel function may only be called from an ISR if its documentation says so. In `deps/zephyr/include/zephyr/kernel.h`, those carry the `@isr_ok` tag: `k_sem_give()`, `k_work_submit()`, `k_msgq_put()` and many more.
- A function that can wait may be called from an ISR only with `K_NO_WAIT`, as `k_sem_take()`'s documentation says: "*`timeout` must be set to `K_NO_WAIT` if called from ISR*".

- **Mutexes may not be used from ISRs at all**: a mutex has an owner, and an ISR is not a thread that could own it.
- An ISR should be short, because while it runs, no thread runs.

Breaking the rule is not always caught. A `k_msleep()` in a timer's expiry function builds and, on `native_sim`, even appears to work. With **`CONFIG_ASSERT=y`**, the kernel checks, and the same program stops at once:

```
ASSERTION FAIL [!arch_is_in_isr()] @ WEST_TOPDIR/deps/zephyr/kernel/sched.c:1222
```

Enable assertions while developing: they cost flash and time, and catch exactly this kind of mistake.

On a board, a peripheral's interrupt handler is connected with `IRQ_CONNECT()`, which the driver usually does for you. In this book the interrupt handler you will meet most is the system clock's, through timer expiry functions, and on `native_sim` the emulated GPIO controller's callbacks run in the context of whoever changed the pin, as Chapter 2 showed.

3.1.7 Work queues

Interrupt handlers must be short and may not block, so real work is moved out of them: into a thread. A **work queue** is a thread with a queue of **work items**, each a function to call. An ISR submits an item, and the work queue's thread calls its function, soon after, in thread context, where it may sleep, block, lock a mutex and log.

Work items

```
static void sample_handler(struct k_work *work);

K_WORK_DEFINE(sample_work, sample_handler);

k_work_submit(&sample_work);
```

`k_work_submit()` puts the item on the **system work queue** and is safe to call from an ISR. It returns:

Return value	Meaning
1	The item was idle, and is now queued.
0	The item was already queued. Nothing changes: it stays in its place and runs **once**.
2	The item was running, and is now queued again, to the queue it is running on.
negative	An error, such as a queue that has not been started.

The second row is a property to design around: **a work item is not a counter**. Submitting it three times before it runs gives one call. If every event matters, the events must be counted or queued somewhere else, and the handler must process all of them.

A handler receives only the `struct k_work` pointer. To give it context, embed the `k_work` in a larger structure and recover the structure with `CONTAINER_OF()`.

The system work queue

Zephyr starts one work queue for everyone, in a thread named `sysworkq`:

Option	Default
`CONFIG_SYSTEM_WORKQUEUE_PRIORITY`	-1: the lowest cooperative priority
`CONFIG_SYSTEM_WORKQUEUE_STACK_SIZE`	1024
`CONFIG_SYSTEM_WORKQUEUE_NO_YIELD`	Not set: the queue yields between items

Being cooperative, a handler on the system work queue is never preempted by another thread, and every subsystem that uses the queue waits for it. **Keep handlers there short, and never block in them for long.** The Kconfig help says why the queue still yields between items: to keep a long queue from starving other threads.

Work queues of your own

Work that is slow, or that must not wait behind other subsystems' work, gets a queue of its own:

```
K_THREAD_STACK_DEFINE(store_stack, 1024);
static struct k_work_q store_queue;

const struct k_work_queue_config config = {.name = "store_q"};

k_work_queue_start(&store_queue, store_stack, K_THREAD_STACK_SIZEOF(store_stack), ↩
    ↪ 5, &config);
k_work_submit_to_queue(&store_queue, &store_work);
```

The queue's priority is its thread's priority, and may be preemptible. `struct k_work_queue_config` also has `no_yield` and `essential`.

Delayable work

A **delayable** work item is submitted after a delay, through a timeout inside it:

```
K_WORK_DELAYABLE_DEFINE(idle_work, idle_handler);

k_work_schedule(&idle_work, K_MSEC(250));     /* If not already scheduled: in 250 ↩
    ↪ ms. */
k_work_reschedule(&idle_work, K_MSEC(250));   /* In 250 ms from now, whatever was ↩
    ↪ scheduled. */
```

The two differ exactly where it matters:

- **`k_work_schedule()`** does nothing if the item is already scheduled or queued: the first deadline wins. It returns 0 in that case.
- **`k_work_reschedule()`** replaces any deadline with the new one: the last call wins.

`k_work_reschedule()` is therefore the tool for a timeout that should expire only once events *stop* coming, such as an inactivity timeout or a debounce. `k_work_cancel_delayable()` cancels a scheduled item, and the `_sync` variants of the cancel functions also wait for a running handler to finish. Inside a handler, `k_work_delayable_from_work()` turns the `struct k_work *` back into the delayable item. The `timer_work` example uses all three kinds of queue.

3.1.8 ISR, work item or thread?

Use	For	Because
An ISR, or a timer's expiry function	Recording what happened, giving a semaphore, submitting work. Microseconds.	Nothing else can react before the next thread runs.
A work item on the system work queue	Short follow-up work that may block briefly: reading a register over I2C, logging.	No thread and stack of its own to pay for.
A work queue of your own	Follow-up work that is slow, or must not wait for other subsystems.	Its own priority, its own stack, and nobody else's work in the way.
A thread	Work with state that lives across many events, such as a protocol or a control loop, and waits in more than one place.	A thread keeps its place in its own code between events; a handler starts from the top every time.

3.1.9 `native_sim`'s time model

`native_sim` advances simulated time only when the CPU is idle, that is, when every thread is blocked or sleeping. Code that runs takes **no** simulated time. Everything above behaves as on a microcontroller as long as threads wait with kernel calls, and the traces of this chapter's examples are exact for that reason. Three things do not:

A busy loop stops the world. A thread that waits with `while (!flag) {}` for an interrupt to set `flag` never lets the CPU go idle, so simulated time never moves, the interrupt never arrives, and nothing else ever runs. On a microcontroller the timer interrupt would preempt the loop after the right time, and the program would merely waste power. The `busy_loop` example shows both. The fix is the same on both: **block on a kernel object**, such as a semaphore the ISR gives, or the timer itself with `k_timer_status_sync()`. Zephyr's documentation of the architecture (`deps/zephyr/boards/native/doc/arch_soc.rst`) suggests `k_cpu_idle()` inside such a loop as a last resort.

A stuck program ignores the first `Ctrl+C`. `native_sim` handles `SIGINT` by asking the simulation to stop at its next step, and a stuck simulation takes no next step. A second `Ctrl+C`

ends it. For the same reason `timeout 5 ./zephyr.exe` does not stop a stuck program, and `timeout -s KILL 5 ./zephyr.exe` does. The repository's test runner kills a program that runs for 30 seconds of real time and reports it as a probable busy loop.

Preemption and time slices happen only at kernel calls and during `k_busy_wait()`. An interrupt can only arrive when simulated time moves, so a preemptible thread running ordinary code is never interrupted, and a loop of a million iterations is never sliced. `k_busy_wait()` does move time, and during it everything works: a thread that wakes preempts, and two preemptible threads of equal priority busy-waiting 50 ms each take turns every 20 ms, one time slice. This chapter's exercises use `k_busy_wait()` wherever a thread has to stand for work that takes CPU time.

3.1.10 Stacks

Every thread has a stack of the size its definition gave, and nothing stops a thread from using more: it writes past the end, into whatever lies there, usually another thread's stack or kernel data. Stack overflow is one of the most common causes of crashes that make no sense in an RTOS application, and stack sizes are one of the most common things to get wrong.

The stacks the kernel creates have Kconfig options. On `native_sim/native/64`:

Option	Value
`CONFIG_MAIN_STACK_SIZE`	1024
`CONFIG_SYSTEM_WORKQUEUE_STACK_SIZE`	1024
`CONFIG_ISR_STACK_SIZE`	2048
`CONFIG_IDLE_STACK_SIZE`	256

On a board

On a microcontroller, three tools find stack problems at run time:

- The **thread analyzer** (`CONFIG_THREAD_ANALYZER`) fills each stack with a known pattern at creation and reports how much of it has been overwritten: each thread's high-water mark.
- The **stack sentinel** (`CONFIG_STACK_SENTINEL`) stores a magic value at the far end of each stack, and checks it at every context switch, among other places.
- **Hardware stack protection** (`CONFIG_HW_STACK_PROTECTION`) uses the MPU to fault on the first write past the stack.

On `native_sim`

None of them works on `native_sim`, for one reason: a `native_sim` thread does not run on the stack Zephyr gave it. It runs on the stack of the Linux thread that backs it, which is megabytes deep, and Zephyr's stack only holds a little bookkeeping. The help text of `CONFIG_ARCH_POSIX_RECOMMENDED_STACK_SIZE`, in `deps/zephyr/arch/posix/Kconfig`, says

so, and the thread analyzer's Kconfig says `depends on !ARCH_POSIX`. Asking for it aborts the build:

```
warning: THREAD_ANALYZER (defined at subsys/debug/thread_analyzer/Kconfig:6) was ↩
    ↪ assigned the value
'y' but got the value 'n'. Check these unsatisfied dependencies: (!ARCH_POSIX) (=n↩
    ↪ ).
```

The documentation (`deps/zephyr/boards/native/doc/arch_soc.rst`) lists stack canaries and hardware stack protection as unsupported too. The stack sentinel does build, but it guards a stack the thread never uses: the `stack_usage` example, whose thread needs twice its stack, runs without complaint with `CONFIG_STACK_SENTINEL=y` as well as without.

Static stack analysis

What works everywhere is asking the compiler. **`CONFIG_STACK_USAGE=y`** compiles every file with GCC's `-fstack-usage`, which writes, beside each object file, a `.su` file with the stack frame size of every function in it:

```
.../src/main.c:18:13:print_report       2096    dynamic,bounded
.../src/main.c:27:13:reporter_entry     16      static
```

A frame is one function's own need, not including the functions it calls. **A thread needs the sum of the frames along its deepest call chain**, plus, on targets where interrupts run on the interrupted thread's stack, what an interrupt pushes onto it. The `.su` files give the frames; working out the chains is up to you, or to tools built on the same data. The numbers depend on the architecture and the compiler options, so `native_sim`'s x86-64 frames are a method to practise, not numbers to copy into a board's configuration.

3.1.11 Summary

- A thread is an entry function, a stack, a priority and a `struct k_thread`. Define long-lived threads with `K_THREAD_DEFINE()`; the kernel starts them just before `main()`, with the scheduler locked until all of them are started.
- Lower numbers are higher priorities. Negative priorities are cooperative, and a cooperative thread keeps the CPU until it gives it up. A preemptible thread loses it as soon as a higher-priority thread is ready.
- The scheduler acts when a thread blocks, sleeps or returns, when a thread or an interrupt makes a higher-priority thread ready, at `k_yield()`, and at the end of a time slice.
- Time is kept in ticks. A relative timeout is rounded up and gets one extra tick; an absolute timeout does not, which makes it the tool for periodic loops. Timers compensate for the extra tick, and do not drift.
- A timer's expiry function runs in interrupt context: no sleeping, no blocking, no mutexes, and `K_NO_WAIT` for anything that can wait. `CONFIG_ASSERT=y` catches the mistake.

- Defer work from ISRs to work queues. A work item submitted while queued runs once. Use `k_work_reschedule()` for a timeout that restarts with every event.
- On `native_sim`, code takes no simulated time: a busy loop stalls everything, and preemption only happens at kernel calls and during `k_busy_wait()`.
- Stack sizes are unchecked on `native_sim`. Use `CONFIG_STACK_USAGE` there, and the thread analyzer on a board.

3.2 Review

What you should be able to do now

After this chapter, you should:

- Be able to create threads statically and dynamically, and choose their priorities.
- Understand exactly when a cooperative and a preemptible thread can lose the CPU.
- Be able to predict the order in which a set of threads runs.
- Understand how timeouts are converted to ticks, and what that does to the length of a sleep.
- Understand which context a timer's expiry function runs in, and what may not be done there.
- Be able to defer work from interrupt context to a work queue, and reschedule delayable work.
- Understand what `native_sim` simulates faithfully about scheduling, and what it does not.
- Be able to estimate a thread's stack needs from static analysis.

Questions to test yourself

You should be able to explain:

1. When can a cooperative thread lose the CPU, and when can a preemptible one?
2. Why does `k_yield()` sometimes return at once, even though other threads are ready?
3. Why does `k_msleep(30)` return after 40 ms on a 100 Hz tick, while a 500 ms one-shot timer fires at exactly 500 ms?
4. What must not be done in a timer's expiry function, and why?
5. Why is the system work queue a cooperative thread, and what does that mean for the work items on it?
6. When would you use a work queue of your own rather than the system work queue?
7. Why does `while (!flag) {}` stall `native_sim` forever, when on a microcontroller it would only waste power?
8. Why are Zephyr's thread analyzer and stack overflow checks not available on `native_sim`?

Looking ahead

Chapter 4 is Kernel II, communication and application architecture:

- Semaphores, mutexes, message queues and events.
- Priority inversion, produced and then fixed.
- zbus: publish and subscribe between parts of an application.
- The State Machine Framework.

You will produce priority inversion on purpose, and measure what a mutex does about it.

3.3 Exercises

These exercises reinforce the concepts from §3.1.

> **How to check your work.** Write each exercise set as its own application under `lectures/L03/exercises`, with the file names given there, and run `make test` from the repository root. A set's tests switch on as soon as its files exist, and a set you have not started is reported as `SKIP`. Each application grows through its set, and the tests check where it ends up. Programs run on `native_sim` in simulated time, so their output, timestamps included, is compared with the expected output line by line.
> The solutions are published in the companion repository, under `lectures/L03/appendix/solutions`, a few days after the lecture. Try each exercise yourself before you read them.

Remember `source env.sh` in every new terminal. Build and run an exercise from the repository root:

```
west build -p -b native_sim/native/64 lectures/L03/exercises/exercise1
./build/zephyr/zephyr.exe -stop_at=2 -no-rt
```

A program that never stops by itself, even with `-stop_at`, is stuck in a busy loop: press `Ctrl+C` **twice**.

EXERCISE SET 1

Who runs when

Create an application named `exercise1` to work in for this exercise set, with a `CMakeLists.txt` as in Chapter 1, and `CONFIG_THREAD_NAME=y` in `prj.conf`.

Every line the program prints comes from this function, which prints the uptime, and the name and priority of the thread that calls it. Copy it into `src/main.c`:

```
static void trace(const char *what)
{
        k_tid_t self = k_current_get();

        printk("%4lld ms  %-7s (%2d)  %s\n", k_uptime_get(), k_thread_name_get(
    ↪ self),
                k_thread_priority_get(self), what);
}
```

Exercise 3.1 Three threads and main

CODE

Tasks

- Write an entry function `periodic()` for a thread, which for now only calls `trace("started")`.
- Define three threads with `K_THREAD_DEFINE()`, all running `periodic()`, each with a stack of 1024 bytes, no options and no start delay:

Thread	Priority
`control`	-1
`sensor`	3
`logger`	7

- In `main()`, call `trace("started")` and return.

Before you run it, write down the order in which you expect the four lines. Then run it.

Reflection

- In which order are the four lines printed? Why does `control` print before `main`, while `sensor` and `logger` print after it?
- Change `control`'s priority to 1 and run it again. What changes, and why?

Exercise 3.2 Periodic threads

CODE

Tasks

Pass each thread two numbers through its first two arguments, with `INT_TO_POINTER()`, and read them in `periodic()` with `POINTER_TO_INT()`:

Thread	p1: period, in ms	p2: work, in ms
`control`	300	150
`sensor`	200	0
`logger`	100	0

Extend `periodic()` so that, after `trace("started")`, it runs one round per period up to 600 ms:

```
for (int round = 1; round * period_ms <= 600; round++) {
        k_msleep(period_ms);
        trace("awake");
        if (work_ms > 0) {
                k_busy_wait(work_ms * USEC_PER_MSEC);
                trace("work done");
        }
}
```

`k_busy_wait()` stands for work that keeps the CPU busy: unlike a loop, it lets simulated time pass.

Reflection

- At which uptimes does `logger` wake up? Why are they not multiples of 100?
- `logger`'s third round should start at 330 ms. When is it printed, and why?

Exercise 3.3 On the dot

CODE

Tasks

Replace the relative sleep with an **absolute** one, so that each thread wakes at exactly every multiple of its period, up to and including 600 ms. The loop becomes a loop over the next wake-up time, `next`, from `period_ms` to 600 in steps of `period_ms`, with `k_sleep(K_TIMEOUT_ABS_MS(next))` at the top of each round.

The output must be:

```
   0 ms  control (-1)  started
   0 ms  main    ( 0)  started
   0 ms  sensor  ( 3)  started
   0 ms  logger  ( 7)  started
 100 ms  logger  ( 7)  awake
 200 ms  sensor  ( 3)  awake
 200 ms  logger  ( 7)  awake
 300 ms  control (-1)  awake
 450 ms  control (-1)  work done
 450 ms  sensor  ( 3)  awake
 450 ms  logger  ( 7)  awake
 460 ms  logger  ( 7)  awake
 500 ms  logger  ( 7)  awake
 600 ms  control (-1)  awake
 750 ms  control (-1)  work done
```

```
750 ms  sensor  ( 3)  awake
750 ms  logger  ( 7)  awake
```

Hint: `next` must be an `int64_t`, like the uptime.

Reflection

- Which of `logger`'s rounds is printed at 450 ms, which at 460 ms, and why is the second one 10 ms later rather than at the same moment?
- Which round of `sensor` is printed at 450 ms?
- `control` is cooperative. Make it preemptible, with priority 1. Does anything change between 300 and 450 ms? Why, or why not?
- Keep `control` at priority 1 and give `sensor` priority 0. What happens at 400 ms now? Why is `control`'s work still done at 450 ms, although it was interrupted?
- Replace `k_busy_wait()` with a `for` loop that counts to a few hundred million. What happens to the trace on `native_sim`, and why would it be different on a microcontroller?

EXERCISE SET 2

A timer and the system work queue

Create an application named `exercise2` to work in for this exercise set, with `CONFIG_THREAD_NAME=y` in `prj.conf`.

Use this function to name the context that code runs in:

```c
static const char *context(void)
{
        return k_is_in_isr() ? "ISR" : k_thread_name_get(k_current_get());
}
```

Exercise 3.4 From ISR to thread

CODE

Tasks

- Define a work item `sample_work` with `K_WORK_DEFINE()`, whose handler `sample_handler()` counts the samples it has taken.
- Define a timer `sample_timer` with `K_TIMER_DEFINE()`, whose expiry function `timer_expired()` stores `context()` in a variable `expiry_context` and submits `sample_work` to the system work queue.
- In `main()`, start the timer with a delay of 250 ms and a period of 250 ms, then print `timer started in <context>`, prefixed with the uptime as in the output below, and return.
- In `sample_handler()`, print a line for each sample with its number, `expiry_context` and `context()`, and stop the timer with `k_timer_stop()` after the fourth sample.

Every line starts with the uptime, formatted with `"%4lld ms "`. Exercise 3.5 gives the lines their final form.

Exercise 3.5 What a submission does, and a stop function CODE

Tasks

- In `timer_expired()`, submit `sample_work` **twice**, and store the two return values of `k_work_submit()`.
- Give the timer a stop function, `timer_stopped()`, which prints `timer stopped, stop function in <context>`.
- Print the return values and both contexts in `sample_handler()`.

The output must be:

```
   0 ms  timer started in main
 250 ms  sample 1: expiry in ISR, submitted 1 then 0, handler in sysworkq
 500 ms  sample 2: expiry in ISR, submitted 1 then 0, handler in sysworkq
 750 ms  sample 3: expiry in ISR, submitted 1 then 0, handler in sysworkq
1000 ms  sample 4: expiry in ISR, submitted 1 then 0, handler in sysworkq
1000 ms  timer stopped, stop function in sysworkq
```

Reflection

- The work was submitted twice for every expiry. How many times did the handler run, and what do the return values 1 and 0 mean?
- Why does the stop function run in `sysworkq`, and not in an ISR like the expiry function?
- The samples come at 250, 500, 750 and 1000 ms, while `k_msleep(250)` in a loop would wake at 260, 520, and so on. Why do the timer's expiries not drift?
- Add `k_msleep(10)` to `timer_expired()` and run it. Then build again with `-- -DCONFIG_ASSERT=y` and run it. What happens each time, and what does that tell you about assertions?
- Remove the `k_msleep(10)`, and instead add `k_msleep(300)` to `sample_handler()`, after the print and before the check that stops the timer, so that handling a sample takes longer than the period. When are the samples taken now? What does `k_work_submit()` return, and why is there a fifth sample, after the timer was stopped?

EXERCISE SET 3

Debouncing

Create an application named `exercise3` to work in for this exercise set.

A mechanical button does not change level once when pressed: its contacts bounce, and the pin toggles several times over a few milliseconds. **Debouncing** means reporting a new level only when the pin has stayed at it for a while. This set uses Chapter 2's emulated GPIO controller, and a thread that plays a finger with bouncing contacts.

Use these files as they are. `prj.conf`:

```
# The GPIO API, for the button.
CONFIG_GPIO=y

# A tick every millisecond, so that a bounce a few milliseconds long can be timed.
CONFIG_SYS_CLOCK_TICKS_PER_SEC=1000
```

`app.overlay`, a button on pin 1 of the emulated GPIO controller:

```
#include <zephyr/dt-bindings/gpio/gpio.h>

/ {
        aliases {
                sw0 = &button0;
        };

        buttons {
                compatible = "gpio-keys";

                button0: button_0 {
                        gpios = <&gpio0 1 (GPIO_PULL_UP | GPIO_ACTIVE_LOW)>;
                        label = "Button";
                };
        };
};
```

And in `src/main.c`, the finger: a press that bounces, a release that bounces, and a glitch too short to be a press. `edges` is the count of edges your callback has seen, below.

```
/* Set the button's physical level: 0 is pressed, since the button is active low. ↩
    ↪ */
static void level(int value, int hold_ms)
{
        gpio_emul_input_set(button.port, button.pin, value);
        k_msleep(hold_ms);
```

```
}

static void finger(void *p1, void *p2, void *p3)
{
        k_sleep(K_TIMEOUT_ABS_MS(100));
        level(0, 2);
        level(1, 1);
        level(0, 3);
        level(1, 1);
        level(0, 300);

        level(1, 4);
        level(0, 2);
        level(1, 300);

        level(0, 5);
        level(1, 300);

        printk("%4lld ms  finger done, %ld edges\n", k_uptime_get(), (long)
    ↪ atomic_get(&edges));
}

K_THREAD_DEFINE(finger_thread, 1024, finger, NULL, NULL, NULL, 7, 0, 0);
```

Exercise 3.6 Every edge

CODE

Tasks

- Get the button as a `struct gpio_dt_spec` named `button`, from the `sw0` alias.
- In `main()`, check that the button is ready, configure it as an input, configure an interrupt on **both** edges, and add a callback.
- In the callback, count every edge in an `atomic_t` named `edges`.

Run it: the finger reports 10 edges at 1028 ms.

Exercise 3.7 Only real presses

CODE

Tasks

- Define a delayable work item, `debounce_work`, with a handler that reads the button's logical level with `gpio_pin_get_dt()`, and prints a line only when that level differs from the last one it reported (the button starts released):

```
<uptime> ms  pressed, after <edges> edges
<uptime> ms  released, after <edges> edges
```

- In the callback, after counting the edge, (re)schedule `debounce_work` to run 20 ms after the **last** edge.

The output must be:

```
 132 ms  pressed, after 5 edges
 441 ms  released, after 8 edges
1028 ms  finger done, 10 edges
```

Hint: Of `k_work_schedule()` and `k_work_reschedule()`, only one moves an already scheduled deadline.

Reflection

- The last edge of the press is at 111 ms. Why is the press reported at 132 ms, and not at 131?
- Why is the glitch at 721 ms, two edges, never reported?
- Try the other one of `k_work_schedule()` and `k_work_reschedule()`. When are the press and the release reported now? The output still has the same events: describe a finger for which it would not.
- On `native_sim`, the callback runs in the finger's thread. In which context would it run on a microcontroller, and why is it still correct to call your scheduling function from it?
- Only the work handler reads and writes the variable holding the last reported level, so it needs no lock. Why can the handler not run twice at the same time?
- Set `CONFIG_SYS_CLOCK_TICKS_PER_SEC=100` and run it. When are the press and the release reported, and why?

EXERCISE SET 4

Stop spinning

Create an application named `exercise4` to work in for this exercise set, with a `prj.conf` holding only a comment.

This program reads three channels of a converter, whose conversion takes 40 ms, simulated with a timer. Copy it into `src/main.c`:

```
#include <stdbool.h>

#include <zephyr/kernel.h>

/* How long the simulated conversion takes. */
```

```
#define CONVERSION_MS 40

/* Set by the timer's expiry function when the conversion is done. */
static volatile bool conversion_done;

static void conversion_expired(struct k_timer *timer)
{
        conversion_done = true;
}

K_TIMER_DEFINE(conversion_timer, conversion_expired, NULL);

/* Start a conversion on a channel, wait for it to finish, and return the result. */
static int convert(int channel)
{
        conversion_done = false;
        k_timer_start(&conversion_timer, K_MSEC(CONVERSION_MS), K_NO_WAIT);

        while (!conversion_done) {
        }

        return 1000 + channel * 111;
}

int main(void)
{
        for (int channel = 0; channel < 3; channel++) {
                const int value = convert(channel);

                printk("%3lld ms  channel %d: %d\n", k_uptime_get(), channel, value);
        }
        printk("%3lld ms  all channels converted\n", k_uptime_get());

        return 0;
}
```

Exercise 3.8 The stall CODE

Tasks

Build it, and run it with `./build/zephyr/zephyr.exe -stop_at=1 -no-rt`. Then run `make test`.

Reflection

- Why does the program print nothing, not even channel 0, and why does `-stop_at=1` not stop it?
- What happens when you press `Ctrl+C` once? And twice?
- What does `make test` report for the set, and after how long?
- Would this program work on a microcontroller? Would it be a good program there?
- Would `k_yield()` inside the loop fix it on `native_sim`? Try it, and explain.

Exercise 3.9 A semaphore CODE

Tasks

Replace the flag with a semaphore: the expiry function gives it, and `convert()` takes it with `K_FOREVER` instead of spinning.

The output must be:

```
 40 ms  channel 0: 1000
 80 ms  channel 1: 1111
120 ms  channel 2: 1222
120 ms  all channels converted
```

Exercise 3.10 No semaphore at all CODE

Tasks

Remove the semaphore and the expiry function as well, define the timer with `K_TIMER_DEFINE(conversion_timer, NULL, NULL)`, and wait for it in `convert()` with `k_timer_status_sync()`. The output must be unchanged.

Reflection

- Why do the conversions finish at exactly 40, 80 and 120 ms?
- Suppose the timer expired before `convert()` started to wait. Would the semaphore version still work? Would the `k_timer_status_sync()` version?
- Zephyr's documentation of `native_sim` suggests calling `k_cpu_idle()` inside a busy loop that cannot be removed. Try it in the original program. Does it work, and why is 4.3 still the better fix?

EXERCISE SET 5

Stacks without a stack analyzer

No code in this set: answer the questions by building and looking.

Exercise 3.11 Frames

REFLECTION

Tasks

Build your `exercise2` with static stack usage analysis:

```
west build -p -b native_sim/native/64 lectures/L03/exercises/exercise2 -- -
    ↪ DCONFIG_STACK_USAGE=y
```

Answer, using `build/CMakeFiles/app.dir/src/main.c.su` and the other `.su` files in `build/`:

- How large is `sample_handler()`'s stack frame?
- On which thread's stack does `sample_handler()` run, and how large is that stack? Find the option that sets it in `build/zephyr/.config`.
- Is `sample_handler()`'s frame all it needs? Which functions does it call, and how large are their frames?
- On a microcontroller, on which stack would `timer_expired()` run, and which option sets its size?

Exercise 3.12 What does not work here

REFLECTION

Tasks

Build `exercise2` again with `-- -DCONFIG_THREAD_ANALYZER=y`.

Reflection

- What happens, and which dependency is not met?
- Why can a thread analyzer that measures how much of each thread's stack was used not work on `native_sim`?

CHAPTER 4

Kernel II: Communication and Application Architecture

How the parts of an application talk to each other.

IN THIS CHAPTER

- Kernel objects for synchronization: semaphores, mutexes and events.
- Priority inversion: how it happens, and how a mutex's priority inheritance bounds it.
- Kernel objects for passing data: message queues, FIFOs, and `k_poll()` for waiting on several.
- Blocking, timeouts and `K_NO_WAIT`: what a thread should do when a queue is full.
- Memory without `malloc()`: static allocation, `k_heap` and memory slabs.
- zbus: channels, validators, listeners, subscribers and message subscribers.
- The State Machine Framework: states, transitions, entry, run and exit actions, and parent states.
- Structuring an application so that its parts know about channels and events, not about each other.

A system of threads is only as good as what passes between them. This chapter is the catalogue: semaphores, mutexes, events, message queues and polling, what each one guarantees, and the trouble each one causes when it is used for the wrong job. Priority inversion gets a section of its own, because it is the one bug in this chapter that a correct-looking program can have for years.

The second half is architecture rather than primitives: zbus, which lets the parts of an application publish to a channel instead of knowing about each other, and the State Machine Framework, which is the difference between a device's behaviour written down and the same behaviour spread across four threads. The examples are in `lectures/L04/examples`.

4.1 Kernel Objects: Synchronization, Data and Memory

Chapter 3 was about threads on their own: who runs, when, and for how long. This section is about threads together: how they wait for each other, protect what they share, hand each

other data, and get memory without `malloc()`. By the end you should be able to explain every line the `lectures/L04/examples/msgq` and `lectures/L04/examples/priority_inversion` examples print. §4.2 builds on it with two ways of structuring an application.

4.1.1 Waiting, and how long for

Every kernel call that can block takes a **timeout** as its last argument, of type `k_timeout_t`:

Timeout	Meaning
`K_NO_WAIT`	Do not wait. If the call cannot succeed at once, it fails at once.
`K_MSEC(n)`, `K_SECONDS(n)`, …	Wait at most that long, then fail.
`K_FOREVER`	Wait for as long as it takes.

A call that fails tells you why through its return value, and the two failures mean different things. `-EBUSY` (or `-ENOMSG`, or `-ENOMEM`, depending on the object) says *it was not possible right now, and you asked not to wait*. `-EAGAIN` says *you waited as long as you allowed, and it still did not become possible*. Both are ordinary results, not errors to ignore: a thread that takes a lock with a timeout and does not check whether it got it is a data race waiting to happen.

An interrupt handler must always pass `K_NO_WAIT`, because an ISR has no thread to put to sleep. The API documentation marks every function an ISR may call with *isr-ok*, and the ones that take a timeout add *timeout must be set to `K_NO_WAIT` if called from ISR*.

A thread waiting on an object waits in a queue ordered by priority. When the object becomes available, the highest-priority waiter gets it, and among waiters of equal priority, the one that has waited longest. A thread that becomes ready this way preempts the current thread if it has a higher priority, exactly as a thread waking from `k_sleep()` would.

4.1.2 Semaphores

A `k_sem` is a counter with a limit:

```
K_SEM_DEFINE(data_ready, 0, 1);   /* Initial count 0, limit 1. */

/* In an interrupt handler, or another thread: */
k_sem_give(&data_ready);

/* In the thread that processes the data: */
if (k_sem_take(&data_ready, K_MSEC(100)) == 0) {
        process();
}
```

`k_sem_take()` decrements the count if it is above zero, and otherwise waits. It returns `0` when it got the semaphore, `-EBUSY` when it did not and was told not to wait, and `-EAGAIN` when its timeout expired. `k_sem_give()` wakes the first waiter if there is one, and otherwise

increments the count, **but never above the limit**: three gives to a semaphore with limit 1 and no waiters leave its count at 1. It never blocks, and may be called from an ISR.

Semaphores are the tool for **signalling and counting**: *data is ready*, *an event happened*, *there are N free buffers*. A semaphore with limit 1 signals that something happened at least once since the last take, not how many times.

A semaphore has no owner. Any thread may give it, whether or not that thread took it, and the kernel does not know or care which thread "holds" it. That is exactly what makes a semaphore right for signalling, and wrong as a lock, for a reason the next two sections are about.

4.1.3 Mutexes

A `k_mutex` is a lock with an owner:

```
K_MUTEX_DEFINE(bus_lock);

k_mutex_lock(&bus_lock, K_FOREVER);
/* ... use the bus: the critical section ... */
k_mutex_unlock(&bus_lock);
```

Three things follow from having an owner:

- **Only the owner may unlock it.** `k_mutex_unlock()` from another thread returns `-EPERM`, and on a mutex that is not locked at all, `-EINVAL`.
- **It is recursive.** The owner may lock it again without blocking: the kernel counts the locks, and the mutex is released when the owner has unlocked it as many times as it locked it. A helper that locks the bus can therefore be called from code that already holds the bus lock.
- **It cannot be used in an ISR.** An interrupt handler is not a thread, so it cannot own anything. An ISR that needs to share data with a thread uses a spinlock, `irq_lock()`, or better, hands the data over through a message queue.

And one thing that matters more than all three: **priority inheritance**.

Priority inversion

Three threads share one CPU: `high`, `medium` and `low`. `low` locks a resource that `high` also needs. Then:

1. `low` takes the lock and starts its critical section.
2. `high` becomes ready, preempts `low`, tries to take the lock, and blocks.
3. `medium` becomes ready. It needs no lock at all. It has a higher priority than `low`, so it preempts `low`, and runs for as long as it has work.
4. Only when `medium` is done does `low` get the CPU back, finish its critical section, and release the lock. Only then can `high` run.

`high`, the most important thread in the system, has waited for `medium`, a less important thread it has nothing to do with. That is **priority inversion**, and what makes it dangerous is step 3: `high`'s wait is no longer bounded by `low`'s critical section, which the designer knows and keeps short, but by the run time of every thread with a priority between the two, which nobody bounded. The Mars Pathfinder lander reset itself repeatedly in 1997 for exactly this reason.

The `lectures/L04/examples/priority_inversion` example produces it on purpose. With a semaphore as the lock, `high waited 579 ms`: 69 ms of `low`'s remaining work plus all 500 ms of `medium`'s.

Priority inheritance

When a thread blocks on a locked `k_mutex`, the kernel raises the owner's priority to the waiter's, if the waiter's is higher. The owner now runs at `high`'s priority, so `medium` can no longer preempt it; it finishes its critical section and unlocks, and at the unlock the kernel restores the priority it had when it locked the mutex. With a mutex in the example, `high waited 79 ms`, just the rest of `low`'s critical section, and the trace shows `low (priority 5) gives the mutex back`: `low` was running at `high`'s priority.

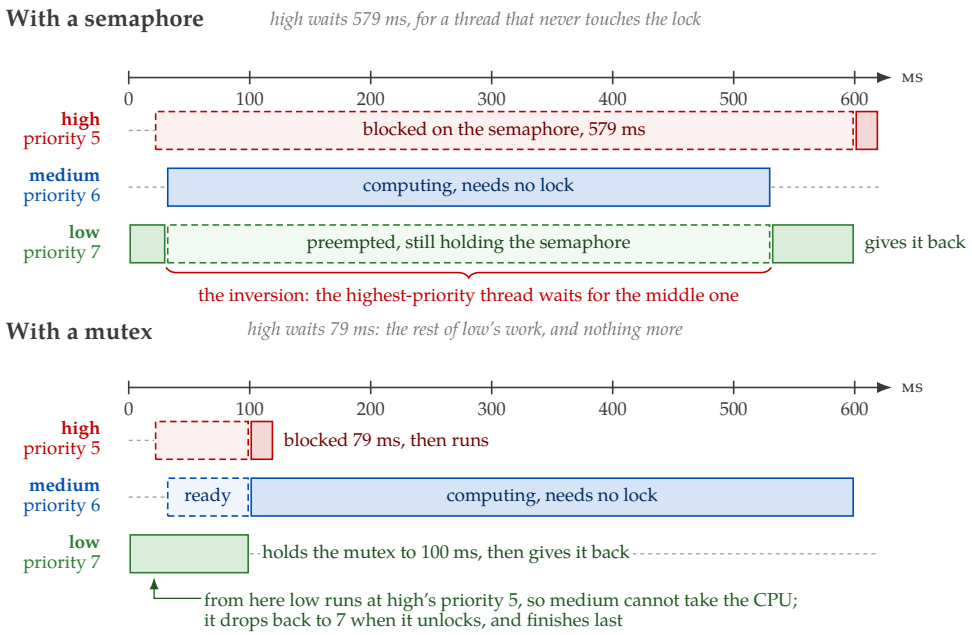


Figure 4.1 The two runs of the `priority_inversion` example on one scale. A solid bar is a thread with the CPU; a dashed one is a thread that is ready or blocked. With the semaphore, `high` waits for work that has nothing to do with the lock; with the mutex, it waits only for `low`'s critical section.

If the waiting thread gives up because its timeout expires, the kernel lowers the owner's priority again, to that of the highest-priority thread still waiting, or back to its own.

What priority inheritance does **not** do:

- **It does not remove the wait.** `high` still waits for `low`'s critical section. Inheritance bounds the wait by the critical section's length; keeping that short is still the designer's job.
- **It does not cope well with several mutexes.** The owner's original priority is saved in each mutex when it is locked. If a thread holds several mutexes and releases them in a different order than its priority was raised, its priority can be restored wrongly. Zephyr's documentation recommends that a thread lock only one mutex at a time when several are shared between threads of different priorities.
- **It does not prevent deadlock.** Two threads that lock two mutexes in opposite orders still wait for each other forever. Always take nested locks in the same order.

`CONFIG_PRIORITY_CEILING` caps how high inheritance may raise a priority. Its default, -128, allows any.

On `native_sim`: making a thread use CPU time

Priority inversion needs a thread that *computes*: holds the CPU without blocking. On `native_sim`, code runs in zero simulated time, so a loop of instructions takes no time at all, and `medium` would finish before anything could preempt it. The examples and exercises use **`k_busy_wait()`** instead, which advances simulated time while keeping the CPU, and during which interrupts are still handled: a thread that becomes ready meanwhile preempts the busy thread, as it would on a microcontroller. The examples busy-wait one millisecond at a time, because a busy wait ends at a fixed deadline even if the thread was preempted in between: one call of `k_busy_wait(100000)` would count time spent preempted as work done.

4.1.4 Events

A `k_event` is a set of 32 bits that threads can wait on:

```
K_EVENT_DEFINE(heater_events);

#define EVENT_START BIT(0)
#define EVENT_FAULT BIT(4)

/* From anywhere, including an ISR: */
k_event_post(&heater_events, EVENT_FAULT);

/* In the thread: wait for any of the events, and clear the ones returned. */
uint32_t events = k_event_wait_safe(&heater_events, EVENT_START | EVENT_FAULT,
                                    false, K_FOREVER);
```

`k_event_post()` sets bits, `k_event_set()` replaces them all, and `k_event_clear()` clears some. `k_event_wait()` returns as soon as any of the requested bits is set, and `k_event_wait_all()` when all of them are. Both return the matching bits, or 0 on a timeout. The `_safe` versions also clear the bits they return in the same step, so that an event posted between the wait returning and a separate `k_event_clear()` cannot be lost. Events need `CONFIG_EVENTS=y`.

An event object is a set of bits, not a queue. Posting a bit that is already set does nothing, so three posts of `EVENT_START` before the thread runs are one `EVENT_START`, and the order in which different bits were posted is lost. The `lectures/L04/examples/smf` example shows it happening. Use events for *which conditions are true now*, and a message queue when the count or the order matters.

4.1.5 Message queues

A `k_msgq` passes fixed-size messages between threads, by copying them:

```
struct sample {
        uint32_t seq;
        int64_t timestamp_ms;
};

/* Room for four samples, allocated statically.
 * The last argument is the alignment. */
K_MSGQ_DEFINE(samples, sizeof(struct sample), 4, alignof(struct sample));

/* The producer: */
struct sample s = {.seq = 1, .timestamp_ms = k_uptime_get()};
int err = k_msgq_put(&samples, &s, K_NO_WAIT);

/* The consumer: */
struct sample s;
k_msgq_get(&samples, &s, K_FOREVER);
```

`k_msgq_put()` copies the message into the queue, and `k_msgq_get()` copies it out, first in, first out. Both return `0` on success, `-ENOMSG` when they could not and were told not to wait, and `-EAGAIN` when their timeout expired. Put may be called from an ISR with `K_NO_WAIT`, which makes a message queue the standard way to hand data from an interrupt handler to a thread.

Because messages are copied, **the sender's variable can go out of scope as soon as the put returns**, and the queue's memory is the only memory involved: `max_msgs × msg_size` bytes, allocated by `K_MSGQ_DEFINE()` at link time. Keep messages small, a few tens of bytes. Larger data travels better as a pointer in a small message, or through a FIFO.

When a thread is already waiting in `k_msgq_get()`, a put copies the message straight into that thread and makes it ready, and if it has the higher priority, it runs before the put even returns. That is why the `lectures/L04/examples/msgq` example prints `consumed #1` before `produced #1`, and a queue that is `0/4` right after the first put.

Other calls: `k_msgq_peek()` reads the next message without removing it, `k_msgq_put_front()` puts a message at the head, `k_msgq_purge()` discards everything and wakes blocked senders with `-ENOMSG`, and `k_msgq_num_used_get()` and `k_msgq_num_free_get()` report how full the queue is.

When the queue is full

A full queue is a design decision, not an accident, and the producer's timeout is where it is made:

- **`K_FOREVER`: slow the producer down.** Nothing is lost, and the producer runs at the consumer's pace. Right when every message matters and the producer can wait, such as commands.
- **`K_NO_WAIT`: drop the new message.** The producer keeps its rate, and must decide what to do with the failure: count it, report it. The only option in an ISR.
- **Make room: drop the oldest.** With one producer, `k_msgq_get()` with `K_NO_WAIT` and a second put keep the newest data, which is usually what sensor data wants.
- **A timeout: wait a little.** Absorbs short bursts without letting a stuck consumer stop the producer.

The example's two phases show the first two: with `K_NO_WAIT` samples #8, #10 and #12 are dropped, and with `K_FOREVER` nothing is, but from #21 on every put blocks for 150 ms.

4.1.6 FIFOs and LIFOs

A `k_fifo` passes data items **by reference**:

```
struct packet {
        void *reserved;      /* The first word belongs to the kernel. */
        size_t length;
        uint8_t data[128];
};

K_FIFO_DEFINE(rx_packets);

/* Put never blocks and may be called from an ISR; get returns NULL on timeout. */
k_fifo_put(&rx_packets, packet);
struct packet *p = k_fifo_get(&rx_packets, K_FOREVER);
```

Nothing is copied: the FIFO links the items together through their **first word**, which the kernel overwrites, so every item type starts with a pointer-sized member the code never

touches. The item must stay valid until it is taken out, so it cannot be a local variable of the sender; it comes from a static pool, a memory slab or a heap. A FIFO has no size limit, because the items bring their own memory, and it never blocks on put. A `k_lifo` is the same, last in, first out.

Use a message queue for small values and a FIFO for large buffers that already live somewhere.

4.1.7 Waiting for several things: `k_poll()`

A thread that must react to a command *or* new data cannot block in `k_msgq_get()` on one of them. `k_poll()` waits for any of several conditions:

```
struct k_poll_event events[] = {
        K_POLL_EVENT_INITIALIZER(K_POLL_TYPE_SEM_AVAILABLE,
                                 K_POLL_MODE_NOTIFY_ONLY, &data_ready),
        K_POLL_EVENT_INITIALIZER(K_POLL_TYPE_MSGQ_DATA_AVAILABLE,
                                 K_POLL_MODE_NOTIFY_ONLY, &commands),
};

while (true) {
        k_poll(events, ARRAY_SIZE(events), K_FOREVER);

        if (events[0].state == K_POLL_STATE_SEM_AVAILABLE) {
                k_sem_take(&data_ready, K_NO_WAIT);
                process_data();
        }
        if (events[1].state == K_POLL_STATE_MSGQ_DATA_AVAILABLE) {
                k_msgq_get(&commands, &command, K_NO_WAIT);
                handle(&command);
        }
        events[0].state = K_POLL_STATE_NOT_READY;
        events[1].state = K_POLL_STATE_NOT_READY;
}
```

Three rules, all easy to miss:

- **`k_poll()` does not take anything.** It only reports that the semaphore was available: the thread still takes it, with `K_NO_WAIT`, and must handle failing to, since a thread blocked in `k_sem_take()` has precedence over a polling one.
- **Check every event's `state`**, since several may be ready at once.
- **Reset every `state` to `K_POLL_STATE_NOT_READY`** before polling again.

Polling needs `CONFIG_POLL=y`. Besides semaphores and message queues, it can wait on FIFOs and on a `k_poll_signal`, a lightweight flag raised with `k_poll_signal_raise()`.

4.1.8 Choosing

You need to	Use
Protect data shared between threads	`k_mutex`
Protect data shared with an ISR	A spinlock or `irq_lock()`, held briefly; or pass the data, below
Signal that something happened, or count available resources	`k_sem`
Wait for any or all of several conditions	`k_event`, or `k_poll()`
Pass small values, copied, from a thread or an ISR	`k_msgq`
Pass large buffers by reference	`k_fifo`
Decouple the parts of an application	zbus, in §4.2

Never use a semaphore as a lock between threads of different priorities. It works, until a thread of medium priority arrives.

4.1.9 Memory without malloc

Every object in this section was defined with a `K_..._DEFINE()` macro, at file scope, and so were the threads and their stacks. That is not a coincidence: **Zephyr applications allocate statically**, so that the linker knows all the memory the system will ever use, and the question *can we run out of memory at run time?* is answered before the firmware ships. For the same reason, **Zephyr has no system heap by default**: `CONFIG_HEAP_MEM_POOL_SIZE` is 0, and a call to `k_malloc()` does not even link:

```
undefined reference to `k_malloc'
```

When an application does need memory at run time, Zephyr offers two allocators, each backed by a static region of its own, so that one part of the system running out cannot starve another.

Memory slabs

A `k_mem_slab` hands out fixed-size blocks:

```
/* Four blocks of 64 bytes, aligned to 8. */
K_MEM_SLAB_DEFINE(packet_slab, 64, 4, 8);

void *block;

if (k_mem_slab_alloc(&packet_slab, &block, K_NO_WAIT) == 0) {
        /* ... */
        k_mem_slab_free(&packet_slab, block);
}
```

Allocation and freeing take constant time, and since every block has the same size, a slab cannot fragment: after any sequence of allocations and frees, a free block is always usable. When the slab is empty, `k_mem_slab_alloc()` returns `-ENOMEM` without waiting, or waits for a block to be freed. Allocation may be done in an ISR with `K_NO_WAIT`. Slabs are the right tool for many objects of one type: packets, messages, buffers of a fixed size.

Heaps

A `k_heap` allocates blocks of any size from its own region:

```
K_HEAP_DEFINE(frame_heap, 1024);

/* NULL if it could not. */
void *frame = k_heap_alloc(&frame_heap, length, K_MSEC(10));
k_heap_free(&frame_heap, frame);
```

A heap can fragment, so it can fail even when enough bytes are free in total. Size it with margin, and use it where sizes genuinely vary. `k_malloc()` and `k_free()` allocate from the system heap, which exists only when `CONFIG_HEAP_MEM_POOL_SIZE` gives it a size.

The C library's `malloc()` is a separate allocator again. On `native_sim` it is the host's own `malloc()`, which never runs out in practice, so an application that uses it will behave very differently on a board. That is one more reason to stay with the kernel's allocators.

4.1.10 Summary

- Blocking calls take a timeout. `K_NO_WAIT` fails at once, a timeout fails with `-EAGAIN`, and an ISR may only use `K_NO_WAIT`.
- Waiters are served highest priority first, and a woken thread of higher priority runs at once.
- A semaphore is a counter without an owner: use it for signalling and counting.
- A mutex has an owner, is recursive, cannot be used in an ISR, and inherits priority. Use it for locking, keep critical sections short, and hold one mutex at a time.
- Priority inversion lets medium-priority threads delay a high-priority one without bound. Priority inheritance bounds the delay by the critical section.
- On `native_sim`, CPU work is simulated with `k_busy_wait()`, which lets higher-priority threads preempt.
- A `k_event` holds conditions as bits: posts merge, and order is lost.
- A message queue copies small messages, and a FIFO links large items by reference. What a producer does with a full queue is a design decision.
- `k_poll()` waits for several objects, but takes none of them.
- Allocate statically. There is no system heap unless one is configured; memory slabs and heaps give run-time allocation from bounded regions.

4.2 Application Architecture: zbus and the State Machine Framework

§4.1 gave threads the means to cooperate. This section is about keeping that cooperation manageable as an application grows: **zbus**, which lets the parts of an application exchange messages without knowing about each other, and the **State Machine Framework**, which turns *what should happen now?* into a table rather than a tangle of flags. By the end you should be able to explain every line the `lectures/L04/examples/zbus` and `lectures/L04/examples/smf` examples print.

4.2.1 Why decouple

A temperature sensor thread that calls `display_update()`, `alarm_check()` and `logger_write()` knows about three other modules, links against them, and has to change whenever a fourth one wants the temperature. Give it a message queue per consumer instead and it still knows about all of them, only through their queues. Either way, testing the sensor means building everything it talks to.

The alternative is to make the *data* the interface. The sensor publishes temperatures to a **channel**; whoever needs temperatures observes that channel. The sensor does not know who is listening, and the consumers do not know who publishes. Adding a consumer changes no existing code, and a test can publish fake temperatures to exercise the consumers, or observe the channel to check the sensor. zbus is Zephyr's implementation of that idea.

4.2.2 zbus

zbus needs `CONFIG_ZBUS=y`. It has two kinds of thing: **channels**, which hold messages, and **observers**, which are told when a channel changes.

Channels

A channel is a statically allocated message of one type, a lock, and a list of observers:

```
struct temperature_msg {
        uint32_t seq;
        int32_t millidegrees;
};

ZBUS_CHAN_DEFINE(temperature_chan,              /* The channel's name. */
                 struct temperature_msg,        /* The message type. */
                 temperature_is_valid,          /* A validator, or NULL. */
                 NULL,                          /* User data, or NULL. */
                 ZBUS_OBSERVERS(alarm, display), /* Who is notified, in this order
    ↪ . */
```

```
                ZBUS_MSG_INIT(0));              /* The message's initial value. */
```

The channel **owns** its message: there is exactly one copy of it, inside the channel, and it always holds the last value published. Everything else gets copies, or a look while the channel is locked.

A **validator** is a function that checks a message before it is accepted:

```
static bool temperature_is_valid(const void *msg, size_t msg_size)
{
        const struct temperature_msg *t = msg;

        return (t->millidegrees >= -40000) && (t->millidegrees <= 125000);
}
```

A message the validator rejects is never stored, no observer hears of it, and the publisher gets `-ENOMSG`. That keeps the check in one place, next to the data, rather than in every consumer.

Publishing and reading

```
struct temperature_msg t = {.seq = 1, .millidegrees = 21500};
int err = zbus_chan_pub(&temperature_chan, &t, K_MSEC(100));
```

`zbus_chan_pub()` does five things, in order:

1. Runs the validator, and returns `-ENOMSG` if it rejects the message.
2. Locks the channel, waiting at most the timeout, and returns `-EBUSY` or `-EAGAIN` if it cannot.
3. Copies the message into the channel.
4. Notifies every observer, in the order `ZBUS_OBSERVERS()` lists them.
5. Unlocks the channel.

If notifying an observer fails, for example because a subscriber's queue is full, `zbus_chan_pub()` returns that error, **although the message was stored**, and the remaining observers are still notified. The one exception is running out of memory for a message subscriber's copy, which stops the notifications there.

Reading copies the message out, under the lock:

```
struct temperature_msg t;
zbus_chan_read(&temperature_chan, &t, K_MSEC(100));
```

`zbus_chan_claim()` and `zbus_chan_finish()` lock the channel and give access to the message in place, for large messages that should not be copied; hold the claim briefly, since it blocks every publisher and reader.

A note on error numbers. The `lectures/L04/examples/zbus` example compares the return value with `-ENOMSG` rather than printing it, because the number depends on the C library. `native_sim` uses the host's, where `ENOMSG` is 42; Zephyr's own minimal C library defines it as 35. Always compare with the name.

Observers

There are four kinds of observer. What distinguishes them is **which thread runs their code**, and **what they receive**:

Observer	Runs in	Receives
Listener `ZBUS_LISTENER_DEFINE(name, callback)`	The publisher's thread, inside `zbus_chan_pub()`, with the channel locked	A call; it reads the message in place with `zbus_chan_const_msg()`
Subscriber `ZBUS_SUBSCRIBER_DEFINE(name, queue_size)`	A thread of its own, waiting in `zbus_sub_wait()`	A notification that the channel changed; it reads the channel itself
Message subscriber `ZBUS_MSG_SUBSCRIBER_DEFINE(name)`	A thread of its own, waiting in `zbus_sub_wait_msg()`	A copy of every message
Asynchronous listener `ZBUS_ASYNC_LISTENER_DEFINE(name, callback)`	A work queue	A copy of the message, passed to the callback

A listener is synchronous. Its callback runs before `zbus_chan_pub()` returns, in the publisher's thread, with the channel locked, so it is effectively part of the publisher. In the example, `alarm saw #1 ... in thread 'sensor_thread'` is printed before `zbus_chan_pub() returned 0`. A listener must therefore be short and must never block, just like an interrupt handler, and it cannot publish to its own channel, whose lock its thread already holds. A listener is the right tool for something quick and urgent: setting an alarm output, waking a thread.

A subscriber is told, not given. Its notification is a pointer to the channel, put into a message queue of `queue_size` entries. The subscriber's thread reads the channel when it gets to run, and sees whatever the channel holds *then*:

```
ZBUS_SUBSCRIBER_DEFINE(display, 4);

static void display_main(void)
{
        const struct zbus_channel *chan;
        struct temperature_msg t;

        while (zbus_sub_wait(&display, &chan, K_FOREVER) == 0) {
```

```
                zbus_chan_read(chan, &t, K_FOREVER);
                show(&t);
        }
}
```

So a subscriber can miss messages. If two messages are published before the subscriber's thread runs, both notifications are waiting, and both reads return the second message. In the example, the display sees #3 twice and never sees #2. That is not a bug: for a display, only the latest value matters, and skipping stale ones is exactly right. When a subscriber's queue is full, the publisher waits for room for as long as its timeout allows, **still holding the channel's lock**, so the subscriber cannot read the channel to catch up; when the timeout expires, the notification is lost and `zbus_chan_pub()` returns the error. Size the queue for the bursts the subscriber must survive.

A message subscriber gets everything. zbus copies each message into a buffer (a `net_buf`, from the heap by default) and queues the copy, and `zbus_sub_wait_msg()` returns it:

```
ZBUS_MSG_SUBSCRIBER_DEFINE(logger);

while (zbus_sub_wait_msg(&logger, &chan, &t, K_FOREVER) == 0) {
        log_reading(&t);
}
```

In the example, the logger sees both #2 and #3. The cost is a buffer per queued message, and it needs `CONFIG_ZBUS_MSG_SUBSCRIBER=y`. Use it when every message matters, as for a logger or a protocol.

Asynchronous listeners need `CONFIG_ZBUS_ASYNC_LISTENER=y`: their callback runs later, from a work queue, with a copy of the message. They suit work too slow for a listener that does not justify a thread of its own.

Priorities decide the order

A listener's output always appears inside the publish. Everything else happens when the scheduler says so. In the example, the sensor thread has the highest priority, so the subscribers only run once it sleeps; give the sensor a lower priority than the subscribers, and each subscriber runs as soon as its notification is queued, before `zbus_chan_pub()` has even returned, and then the display sees #2 too.

`CONFIG_ZBUS_PRIORITY_BOOST`, enabled by default, can raise a publisher's priority while it holds a channel, above the highest-priority observer thread, so that observers cannot preempt a publish halfway. It only knows an observer's priority once the observer's thread has called `zbus_obs_attach_to_thread()`; the examples do not, so no boost happens there.

Designing with channels

- **One channel per kind of data**, named for the data (`temperature_chan`), not for its producer or consumer.
- **Small, flat messages.** Every publish and read copies them.
- **Choose the observer by what it needs:** a listener for urgent and quick, a subscriber for *the latest value*, a message subscriber for *every value*.
- **Validate in the channel**, not in the consumers.

4.2.3 The State Machine Framework

Most devices are state machines, whether or not their code says so. A heater is idle, heating, holding a temperature, or in a fault, and what an event means depends on which: *temperature reached* matters while heating, and means nothing while idle. Written as flags and `if` statements, that knowledge is spread over every function that handles an event. Written as a state machine, it is in one place, and each state's code only handles what can happen in that state.

The State Machine Framework (SMF) needs `CONFIG_SMF=y`, and `CONFIG_SMF_ANCESTOR_SUPPORT=y` for parent states.

States

A state is a `struct smf_state` with up to three **actions**, functions that take the state machine's object:

- **entry**, run when the state is entered;
- **run**, run each time the state machine runs while in the state, typically once per event;
- **exit**, run when the state is left.

The whole state machine is an array of states:

```
enum heater_state { OPERATIONAL, IDLE, HEATING, HOLD, FAULT };

static const struct smf_state states[] = {
    [OPERATIONAL] = SMF_CREATE_STATE(operational_entry, operational_run,
                                     operational_exit, NULL, NULL),
    [IDLE] = SMF_CREATE_STATE(idle_entry, idle_run, NULL,
                              &states[OPERATIONAL], NULL),
    [HEATING] = SMF_CREATE_STATE(heating_entry, heating_run, heating_exit,
                                 &states[OPERATIONAL], NULL),
    [HOLD] = SMF_CREATE_STATE(hold_entry, hold_run, NULL,
                              &states[OPERATIONAL], NULL),
    [FAULT] = SMF_CREATE_STATE(fault_entry, fault_run, fault_exit, NULL, NULL),
};
```

`SMF_CREATE_STATE(entry, run, exit, parent, initial)` takes the three actions, any of which may be `NULL`, the state's parent, and an initial child state, used only with `CONFIG_SMF_INITIAL_TRANSITION`.

The object and its context

The state machine works on an object of the application's own, whose **first member** must be a `struct smf_ctx`, so that a pointer to the object is also a pointer to the context:

```
struct heater {
        struct smf_ctx ctx;   /* Must be first. */
        uint32_t event;       /* What the run actions look at. */
};
```

`SMF_CTX(&heater)` casts the object to its context for SMF's functions, and every action receives the object as a `void *`.

Running it

Three functions do everything:

- **`smf_set_initial(ctx, state)`** starts the state machine in a state, running the entry actions of the state and its ancestors, outermost first.
- **`smf_run_state(ctx)`** runs the current state's run action once.
- **`smf_set_state(ctx, state)`**, called from an action, transitions to another state: it runs the exit actions of the states being left and the entry actions of the states being entered.

A typical application runs the state machine once per event, from a thread:

```
while (true) {
        k_msgq_get(&heater_events, &heater.event, K_FOREVER);
        smf_run_state(SMF_CTX(&heater));
}
```

A run action decides what the event means in its state:

```
static enum smf_state_result heating_run(void *obj)
{
        struct heater *heater = obj;

        switch (heater->event) {
        case EVENT_TEMP_REACHED:
                smf_set_state(SMF_CTX(heater), &states[HOLD]);
                return SMF_EVENT_HANDLED;
        case EVENT_STOP:
                smf_set_state(SMF_CTX(heater), &states[IDLE]);
```

```
                return SMF_EVENT_HANDLED;
        default:
                return SMF_EVENT_PROPAGATE;
        }
}
```

`smf_set_state()` must not be called from an exit action, where the state machine is already in the middle of a transition; SMF logs an error and ignores it. `smf_set_terminate(ctx, value)` stops the state machine, and makes `smf_run_state()` return the value from then on.

Parent states

With `CONFIG_SMF_ANCESTOR_SUPPORT=y`, states form a hierarchy, and two things change.

Events propagate upwards. A run action returns `SMF_EVENT_HANDLED` or `SMF_EVENT_PROPAGATE`. If the state handled the event, or transitioned, that is the end of it. Otherwise the parent's run action gets the event, then the grandparent's, until one handles it. A state without a run action propagates everything. Behaviour that several states share is written once, in their parent: in the example, a fault in IDLE, HEATING or HOLD is handled by OPERATIONAL alone.

Transitions only leave and enter what changes. A transition runs exit actions from the current state up to, but not including, the lowest ancestor that the old and new state have in common, and entry actions from below that ancestor down to the new state. Moving from HEATING to HOLD, both children of OPERATIONAL, runs HEATING's exit and HOLD's entry, and OPERATIONAL is neither left nor entered. Moving to FAULT, which is outside OPERATIONAL, runs OPERATIONAL's exit as well, and the example's trace shows `OPERATIONAL exit`. Whatever a parent sets up in its entry action, such as switching a motor on, therefore stays set up while the state machine moves between its children.

Without `CONFIG_SMF_ANCESTOR_SUPPORT`, the parent argument is ignored, every state is on its own, and a run action's return value does not matter: that is a **flat** state machine.

Driving it with events

The state machine's thread needs events to run on, and how they arrive matters:

- **A message queue** delivers every event, in order, and can carry data with it. It is the default choice.
- **A `k_event`** delivers *which* events happened, as bits. It is simple and ISR-friendly, but events of the same kind merge and their order is lost: the `lectures/L04/examples/smf` example's README shows `START`, `STOP`, `START`, `START` being handled as `START`, `START`, `STOP`. Use it only when that cannot matter.
- **zbus**: the state machine's thread subscribes to the channels whose messages are its events.

Keep the state machine itself free of threads. If its code only reacts to the event it is given and records what it does, it can be tested by calling it directly with a sequence of events, with no threads and no timing at all. Exercise Set 4 is built that way.

4.2.4 Summary

- zbus decouples an application's parts: publishers and observers only know channels.
- A channel owns one statically allocated message of one type, and a validator can reject messages before they are stored.
- `zbus_chan_pub()` validates, locks, copies, notifies observers in order, and unlocks.
- A listener runs inside the publish, in the publisher's thread, with the channel locked: short, and never blocking.
- A subscriber is notified and reads the latest value, so it can miss intermediate messages. A message subscriber gets a copy of every message.
- Thread priorities decide when subscribers run, relative to the publisher.
- Compare error codes with their names: `native_sim` uses the host's C library.
- SMF describes a state machine as a table of states with entry, run and exit actions.
- `smf_set_initial()` starts it, `smf_run_state()` runs it once per event, and `smf_set_state()` transitions.
- With parent states, unhandled events propagate upwards, and transitions do not leave or re-enter common ancestors.
- Feed a state machine from a message queue when order and count matter, and keep its logic free of threads, so that it can be tested directly.

4.3 Review

What you should be able to do now

After this chapter, you should:

- Be able to choose the right kernel object for a synchronization or communication problem.
- Understand why a semaphore used as a lock allows unbounded priority inversion, and a mutex does not.
- Be able to pass data between threads with a message queue, and handle a full queue deliberately.
- Understand how Zephyr applications avoid dynamic memory, and what to use when they cannot.
- Be able to decouple an application's parts with zbus channels, and choose between its kinds of observer.
- Be able to express a device's behaviour as a hierarchical state machine with SMF.

Questions to test yourself

You should be able to explain:

1. What is the difference between a semaphore and a mutex, beyond the count?
2. What is priority inversion, and why is the medium-priority thread the one that makes it unbounded?
3. How does priority inheritance bound it, and what does it not solve?
4. When should a producer use `K_NO_WAIT` on a full queue, and when a timeout?
5. Why do Zephyr applications usually allocate kernel objects and buffers statically?
6. What is the difference between a zbus listener, a subscriber and a message subscriber, and in which thread does each one run?
7. Why can a zbus subscriber miss a message, and which observer cannot?
8. What does a parent state in SMF save you from writing?
9. On `native_sim`, why must a thread that simulates CPU work use `k_busy_wait()` rather than a loop?

Looking ahead

Chapter 5 is about writing a driver:

- The device driver model: API structs, config and data, instances and init levels.
- A binding and a Kconfig option for a new part.
- Emulators: hardware that exists only in software, on `native_sim`'s I2C bus.
- Testing the driver with ztest, and running the tests with Twister.

You will write a driver for an I2C temperature sensor that does not exist.

4.4 Exercises

These exercises reinforce the concepts from §4.1 and §4.2.

> **How to check your work.** Write each exercise set as its own application under `lectures/L04/exercises`, with the file names given there, and run `make test` from the repository root. A set's tests switch on as soon as its files exist, and a set you have not started is reported as `SKIP`. Each application grows through its set, and the tests check where it ends up. Programs run on `native_sim` in simulated time, so their output, timestamps included, is compared with the expected output line by line. The threads' priorities, start delays and sleeps given below are part of the exercise: change one, and the output changes.
> The solutions are published in the companion repository, under `lectures/L04/appendix/solutions`, a few days after the lecture. Try each exercise yourself before you read them.

Remember `source env.sh` in every new terminal. Build and run an exercise from the repository root:

```
west build -p -b native_sim/native/64 lectures/L04/exercises/exercise1
./build/zephyr/zephyr.exe -stop_at=3 -no-rt
```

EXERCISE SET 1

Readings in a queue

Create an application named `exercise1` to work in for this exercise set, with a `CMakeLists.txt`, a `prj.conf` and a `src/main.c`.

Exercise 4.1 A sampler and a slower storage thread

CODE

In this exercise you will connect a fast producer and a slow consumer with a message queue.

Tasks

- In `prj.conf`, set one tick per millisecond, as this chapter's examples do: `CONFIG_SYS_CLOCK_TICKS_PER_SEC=1000`.
- Define a message type:

```c
struct reading {
  uint32_t seq;
  int64_t timestamp_ms;
};
```

- Define a message queue named `readings` with room for **3** readings.
- Write a thread `sampler`, priority **5**, which takes 8 readings, numbered 1 to 8. For each:
  - Fill in a `struct reading` with the number and `k_uptime_get()`.
  - Put it into the queue **without waiting**. If that fails, print `[<t> ms] queue full, dropped #<n>`, with the reading's number. Otherwise print `[<t> ms] sampled #<n>, queue <used>/3`.
  - Sleep 100 ms.

  After the eighth reading, print `[<t> ms] sampler done`.
- Write a thread `storage`, priority **6**, which forever takes a reading from the queue, waiting as long as it takes, prints `[<t> ms] stored #<n>, <age> ms old` (three spaces after the timestamp), where the age is the uptime minus the reading's timestamp, and sleeps 350 ms.
- Start both threads with `K_THREAD_DEFINE()`, a stack of 1024 bytes, and no start delay.

Print every timestamp with `%4lld`. With `-stop_at=3`, the output must be:

```
[   0 ms] sampled #1, queue 1/3
[   0 ms]   stored #1, 0 ms old
[ 101 ms] sampled #2, queue 1/3
[ 202 ms] sampled #3, queue 2/3
[ 303 ms] sampled #4, queue 3/3
[ 351 ms]   stored #2, 250 ms old
[ 404 ms] sampled #5, queue 3/3
[ 505 ms] queue full, dropped #6
[ 606 ms] queue full, dropped #7
[ 702 ms]   stored #3, 500 ms old
[ 707 ms] sampled #8, queue 3/3
[ 808 ms] sampler done
[1053 ms]   stored #4, 750 ms old
[1404 ms]   stored #5, 1000 ms old
[1755 ms]   stored #8, 1048 ms old
```

Hint: `K_MSGQ_DEFINE()`'s last argument is the message's alignment: `alignof(struct reading)`, from `<stdalign.h>`. `k_msgq_num_used_get()` returns how many messages are queued.

Reflection

- `sampled #1` is printed before `stored #1`. Why? Give `storage` priority 4 instead, and run it again: what changes, and why does the queue read `0/3`?
- Why does the sampler put without waiting, rather than with `K_FOREVER`? Which kind of code could not wait at all?

Exercise 4.2 Keep the newest

CODE

Tasks

Change the sampler so that a full queue drops the **oldest** reading instead of the new one:

- When the put fails, take the oldest reading out of the queue without waiting, print `[<t> ms] queue full, dropped #<n>` with **its** number, and put the new reading again.
- Then print `sampled #<n>` for the new reading, as for any other.

The output must now be:

```
[   0 ms] sampled #1, queue 1/3
[   0 ms]   stored #1, 0 ms old
[ 101 ms] sampled #2, queue 1/3
[ 202 ms] sampled #3, queue 2/3
[ 303 ms] sampled #4, queue 3/3
[ 351 ms]   stored #2, 250 ms old
```

```
[ 404 ms] sampled #5, queue 3/3
[ 505 ms] queue full, dropped #3
[ 505 ms] sampled #6, queue 3/3
[ 606 ms] queue full, dropped #4
[ 606 ms] sampled #7, queue 3/3
[ 702 ms]   stored #5, 298 ms old
[ 707 ms] sampled #8, queue 3/3
[ 808 ms] sampler done
[1053 ms]   stored #6, 548 ms old
[1404 ms]   stored #7, 798 ms old
[1755 ms]   stored #8, 1048 ms old
```

Reflection

- Compare the readings stored, and their ages, with Exercise 4.1. Which policy does a temperature logger want? Which would a queue of commands want?
- Make the queue 8 readings long. Is anything dropped now? How old is the last reading when it is stored? Did the bigger queue help?
- How much RAM does the queue's buffer use on `native_sim/native/64`, and when is it allocated?
- Dropping the oldest takes two calls, a get and a put. Why is that safe in this program, and when would it not be?

EXERCISE SET 2

Priority inversion

Create an application named `exercise2` to work in for this exercise set.

Exercise 4.3 A log behind a semaphore

CODE

In this exercise you will produce priority inversion, measure it, and fix it.

Tasks

- In `prj.conf`, set `CONFIG_SYS_CLOCK_TICKS_PER_SEC=1000`.
- Create a `Kconfig` file with a menu named `Application` holding a `bool` option `APP_USE_MUTEX`, with the prompt `Protect the log with a k_mutex instead of a k_sem`, and end it so that Zephyr's own options are still available. The option stays unused until Exercise 4.4.
- In `src/main.c`, protect a shared log with a binary semaphore named `log_lock`, created available (initial count 1, limit 1).
- Write a function `trace(who, what)` that prints `[<t> ms] <who> (priority <p>) <what>`, with the uptime as `%4lld`, `who` left-aligned in 7 characters (`%-7s`), and the calling thread's current priority from `k_thread_priority_get()`.

- Write a function `compute(ms)` that uses CPU time by calling `k_busy_wait(USEC_PER_MSEC)` `ms` times.
- Write three threads, all with a stack of 1024 bytes:
  - `logger`, priority **9**, no start delay: traces `takes the lock`, takes it, computes for **60** ms, traces `gives the lock back`, and gives it back.
  - `control`, priority **5**, start delay **10** ms: records the uptime, traces `needs the lock`, takes it, traces `has the lock`, gives it back, and prints `control waited <ms> ms`, the time since the uptime it recorded.
  - `network`, priority **7**, start delay **20** ms: traces `starts sending`, computes for **300** ms, and traces `is done`.

The output must be:

```
[   0 ms] logger  (priority 9) takes the lock
[  11 ms] control (priority 5) needs the lock
[  21 ms] network (priority 7) starts sending
[ 321 ms] network (priority 7) is done
[ 360 ms] logger  (priority 9) gives the lock back
[ 360 ms] control (priority 5) has the lock
control waited 349 ms
```

Hint: why `compute()` busy-waits one millisecond at a time rather than calling `k_busy_wait(60000)` once is in §4.1, *On `native_sim`: making a thread use CPU time*.

Reflection

- Before you run it, predict how long `control` waits. Then account for the 349 ms, millisecond by millisecond: who ran when?
- `network` never touches the lock. How did it delay `control` by 300 ms?

Exercise 4.4 The mutex

CODE

Tasks

- When `CONFIG_APP_USE_MUTEX` is enabled, protect the log with a `k_mutex` named `log_lock` instead of the semaphore, and change nothing else. Hide the difference behind two macros, such as `lock()` and `unlock()`, so that the threads' code is the same in both builds.
- Create a file `mutex.conf` that enables the option.

Built with `-DEXTRA_CONF_FILE=mutex.conf`, the output must be:

```
[   0 ms] logger  (priority 9) takes the lock
[  11 ms] control (priority 5) needs the lock
[  60 ms] logger  (priority 5) gives the lock back
```

```
[  60 ms] control (priority 5) has the lock
control waited 49 ms
[  60 ms] network (priority 7) starts sending
[ 360 ms] network (priority 7) is done
```

Built without it, the output must be unchanged from Exercise 4.3.

Reflection

- Why is `logger`'s priority 5 when it gives the lock back? What is it after the unlock?
- `network` was ready at 21 ms. Why does it only start sending at 60 ms?
- `control` still waited 49 ms. Could any lock bring that down to zero? What would?
- With the mutex, make `control` wait at most 20 ms, with `k_mutex_lock(&log_lock, K_MSEC(20))`, and trace `gave up` when that fails. What happens to `logger`'s priority when `control` gives up, and what does `network` do then?

EXERCISE SET 3

A battery on zbus

Create an application named `exercise3` to work in for this exercise set.

Exercise 4.5 The channel and a listener

CODE

In this exercise you will publish battery readings on a zbus channel, and react to them.

Tasks

- Enable zbus in `prj.conf`.
- Define the message type:

```
struct battery_msg {
  uint8_t percent;
  bool charging;
};
```

- Define a channel named `battery_chan`, initially `{.percent = 100, .charging = false}`, with a validator that rejects any message with `percent` above 100.
- Define a listener named `led`, observing the channel, whose callback prints `  led: red` (two spaces first) when the battery is below 20 % and not charging, and `  led: green` otherwise.
- Write a thread `gauge`, priority **6**, stack 1024, no start delay, that publishes these readings, in this order:

```
static const struct battery_msg readings[] = {
  {.percent = 80, .charging = false}, {.percent = 15, .charging = false},
  {.percent = 150, .charging = false}, {.percent = 15, .charging = true},
  {.percent = 60, .charging = true},
};
```

For each, it prints `gauge: <percent> %`, followed by `, charging` if the battery is charging, publishes the reading with a timeout of 50 ms, prints `gauge: rejected` if the channel rejected it, and sleeps 100 ms.

The output must be:

```
gauge: 80 %
  led: green
gauge: 15 %
  led: red
gauge: 150 %
gauge: rejected
gauge: 15 %, charging
  led: green
gauge: 60 %, charging
  led: green
```

Hint: A listener callback reads the message in place with `zbus_chan_const_msg(chan)`. In a format string, `%%` prints a percent sign.

Reflection

- In which thread does the `led` callback run? Check it: enable `CONFIG_THREAD_NAME=y`, and print `k_thread_name_get(k_current_get())` in the callback.
- What number does `zbus_chan_pub()` return for the 150 % reading on `native_sim`? Why should the code compare it with `-ENOMSG` rather than with that number?

Exercise 4.6 A display that subscribes

CODE

Tasks

- Define a subscriber named `display`, with a notification queue of **4**, and add it to the channel's observers after `led`.
- Write a thread `display_thread`, priority **7**, stack 1024, no start delay, which forever waits for a notification, reads the channel, and prints `display: <percent> %`, followed by `, charging` if the battery is charging.

The output must now be:

```
gauge: 80 %
  led: green
  display: 80 %
gauge: 15 %
  led: red
  display: 15 %
gauge: 150 %
gauge: rejected
gauge: 15 %, charging
  led: green
  display: 15 %, charging
gauge: 60 %, charging
  led: green
  display: 60 %, charging
```

Reflection

- Remove the gauge's 100 ms sleep. What does the display print now, and why? Did it receive four notifications?
- Keep the sleep removed, and give `display_thread` priority 5, above the gauge. What does the display print now?
- Why does the `led` never miss a reading, whatever the priorities?

EXERCISE SET 4

A garage door as a state machine

Create an application named `exercise4` to work in for this exercise set, with `target_sources(app PRIVATE src/main.c src/door.c)` in its `CMakeLists.txt`.

Exercise 4.7 The state machine

CODE

In this exercise you will write a hierarchical state machine with SMF, behind an API that tests can drive directly.

Tasks

Copy this header into `src/door.h` unchanged. It is the interface the tests use:

```
#ifndef DOOR_H_
#define DOOR_H_

#include <stdbool.h>

/** What can happen to the door. */
```

```
enum door_event {
        DOOR_EVENT_BUTTON,   /**< The button was pressed. */
        DOOR_EVENT_OPENED,   /**< The door reached its top end switch. */
        DOOR_EVENT_CLOSED,   /**< The door reached its bottom end switch. */
        DOOR_EVENT_OBSTACLE, /**< The light barrier saw something in the way. */
};

/** The states the door can be in. */
enum door_state {
        DOOR_STATE_CLOSED,
        DOOR_STATE_OPENING,
        DOOR_STATE_OPEN,
        DOOR_STATE_CLOSING,
        DOOR_STATE_STOPPED,
};

/** @brief Start the state machine, in the closed state, with the motor off
 *         and never started. */
void door_init(void);

/** @brief Run the state machine once, for one event. */
void door_handle(enum door_event event);

/** @brief Return the state the door is in. */
enum door_state door_get_state(void);

/** @brief Return whether the motor is running. */
bool door_motor_on(void);

/** @brief Return how many times the motor has been started since door_init(). */
unsigned int door_motor_starts(void);

#endif /* DOOR_H_ */
```

In `prj.conf`, enable SMF with parent states. In `src/door.c`, implement the header with SMF, with these six states:

State	Parent	Entry	Handles
`MOVING`	none	Motor on, and count the start	`BUTTON`: go to `STOPPED`. Every other event: ignore it. Exit: motor off.
`CLOSED`	none		`BUTTON`: go to `OPENING`.
`OPENING`	`MOVING`		`OPENED`: go to `OPEN`. Anything else: pass it to the parent.
`OPEN`	none		`BUTTON`: go to `CLOSING`.
`CLOSING`	`MOVING`		`CLOSED`: go to `CLOSED`. `OBSTACLE`: go to `OPENING`. Anything else: pass it to the parent.
`STOPPED`	none		`BUTTON`: go to `CLOSING`.

Events a state does not list are ignored. `door_get_state()` returns the state the door is in, and `MOVING` is never one: the door is always in one of the five states of the header.

Hint: `MOVING`'s entry and exit actions are the only code that touches the motor. Keep the state in a variable that each leaf state's entry action sets.

Exercise 4.8 Drive it

CODE

Tasks

In `src/main.c`, call `door_init()`, print the door's status as `start`, and then handle these events one at a time, printing the status after each, labelled with the event's name:

```
static const enum door_event script[] = {
        DOOR_EVENT_BUTTON,   DOOR_EVENT_OBSTACLE, DOOR_EVENT_OPENED,
        DOOR_EVENT_BUTTON,   DOOR_EVENT_OBSTACLE, DOOR_EVENT_OPENED,
        DOOR_EVENT_BUTTON,   DOOR_EVENT_BUTTON,   DOOR_EVENT_BUTTON,
        DOOR_EVENT_CLOSED,   DOOR_EVENT_OBSTACLE,
};
```

Print each status with `"%-8s -> %-7s motor %-3s (started %u times)\n"`: the label, the state's name, `on` or `off`, and the number of motor starts. The output must be:

```
start    -> CLOSED  motor off (started 0 times)
BUTTON   -> OPENING motor on  (started 1 times)
OBSTACLE -> OPENING motor on  (started 1 times)
OPENED   -> OPEN    motor off (started 1 times)
BUTTON   -> CLOSING motor on  (started 2 times)
OBSTACLE -> OPENING motor on  (started 2 times)
OPENED   -> OPEN    motor off (started 2 times)
BUTTON   -> CLOSING motor on  (started 3 times)
BUTTON   -> STOPPED motor off (started 3 times)
BUTTON   -> CLOSING motor on  (started 4 times)
CLOSED   -> CLOSED  motor off (started 4 times)
OBSTACLE -> CLOSED  motor off (started 4 times)
```

The tests also drive `door.c` directly, without `main.c`, through sequences of events the script does not contain.

Reflection

- `OBSTACLE` while closing sends the door back to `OPENING`, and the motor start count stays at 2. Which rule of SMF makes that happen?
- Which state handles `OBSTACLE` while the door is opening, and what does it do with it?
- Change `OPENING`'s run action to return `SMF_EVENT_HANDLED` for every event. What happens when the button is pressed while the door is opening? Which tests fail?
- Remove `CONFIG_SMF_ANCESTOR_SUPPORT=y` from `prj.conf`. Does it still build? What does the program print now, and why?
- Why can `door.c` be tested without threads, timers, or `native_sim`'s clock?

CHAPTER 5

Writing a Driver

A driver for a part that does not exist.

IN THIS CHAPTER

- The device driver model: `struct device`, the API struct, and per-instance config and data.
- Defining devices from the devicetree: `DT_DRV_COMPAT`, `DEVICE_DT_INST_DEFINE()` and `DT_INST_FOREACH_STATUS_OKAY()`.
- Initialization: init levels, priorities, what `device_is_ready()` reports after a failure, and deferring a device's initialization with `zephyr,deferred-init` and `device_init()`.
- A binding and a Kconfig option for a new part, and a Kconfig option that turns itself on when the devicetree has the part.
- Implementing a subsystem API: the sensor API's `sample_fetch` and `channel_get`.
- Zephyr modules: keeping drivers out of tree, with `zephyr/module.yml`.
- Emulators: a chip in software, on `native_sim`'s emulated I2C bus, and the sensor emulator backend.
- ztest: suites, tests, assertions, and hooks that run around every test.
- Twister: `testcase.yaml`, scenarios, and running a test suite in one command.

Zephyr's device model is what turns a node in the devicetree into a `struct device` the application can use without knowing what is behind it. This chapter writes one: a binding, a Kconfig option that turns itself on when the devicetree has the part, a driver that serves every instance of it, and an implementation of the sensor API, so the application never names the chip at all.

Then it does what no board lets you do. The part is emulated, in software, on `native_sim`'s I2C bus, and the emulator can be told to answer wrongly: a bad identity register, a conversion that never finishes. That is what makes the driver testable, and §5.2 is about the testing: ztest for the tests themselves, Twister for running them. The worked example is `lectures/L05/examples/thermo`.

5.1 Device Drivers

This section covers how Zephyr turns a devicetree node into a device, what a driver provides for it, and how a driver for a new part is packaged: binding, Kconfig option, and module. By the end you should be able to explain every file in `lectures/L05/examples/thermo/module`, and why each one is needed.

5.1.1 What a driver is, in Zephyr

Chapter 2 used devices without asking where they came from: `DEVICE_DT_GET(DT_ALIAS(led0))` returned a `const struct device *`, and the GPIO API did the rest. This chapter is the other side of that call.

A Zephyr driver is three things:

1. **A set of functions** that implement one of Zephyr's device APIs, such as GPIO, I2C or sensor, for one kind of part.
2. **An init function**, which the kernel calls once at boot to bring an instance of the part up.
3. **A macro expansion per devicetree node**, which creates a `struct device` for every node that the driver serves, at compile time, with the node's properties already filled in.

There is no registration at run time and no probing. By the time the program links, every device exists as a constant in flash, and the only thing left to happen at boot is the init function.

5.1.2 `struct device`

Every device is a `struct device`, defined in `<zephyr/device.h>`. Its first five members are the ones a driver writer deals with:

```
struct device {
        const char *name;           /* The devicetree node's name, e.g. "thermo@48". */
        const void *config;         /* Per-instance config: const, from devicetree. */
        const void *api;            /* The function table: the driver's API struct. */
        struct device_state *state; /* Owned by the kernel: init's state and result. */
        void *data;                 /* Per-instance run-time data: mutable. */
        /* ... */
};
```

`config` and `data` are the driver's own types, which the kernel stores as `void *`. The difference between them matters on a microcontroller:

- **`config` is `const`.** It holds what the devicetree says about the instance: its bus, its address, its properties. It is built at compile time and lives in flash.
- **`data` is not.** It holds what changes at run time: the last reading, a mutex, a state machine. It lives in RAM.

In `lectures/L05/examples/thermo/module/drivers/sensor/thermo/thermo.c` they are:

```
struct thermo_config {
        struct i2c_dt_spec i2c;
        uint32_t conversion_timeout_ms;
};

struct thermo_data {
        int16_t raw;
};
```

and every driver function starts by getting them back out of the device:

```
const struct thermo_config *config = dev->config;
struct thermo_data *data = dev->data;
```

The name is the node's name, `thermo@48`, not its label. That is what `dev->name` prints and what the logs show.

5.1.3 The API struct

Each kind of device has an API: a struct of function pointers declared by its subsystem. For the sensor subsystem it is `struct sensor_driver_api` in `<zephyr/drivers/sensor.h>`, whose two mandatory members are:

```
sensor_sample_fetch_t sample_fetch;
sensor_channel_get_t channel_get;
```

The functions an application calls, such as `sensor_sample_fetch(dev)`, are thin wrappers that cast `dev->api` and call through it:

```
static inline int z_impl_sensor_sample_fetch(const struct device *dev)
{
        const struct sensor_driver_api *api =
                (const struct sensor_driver_api *)dev->api;

        return api->sample_fetch(dev, SENSOR_CHAN_ALL);
}
```

A driver fills in its own table with `DEVICE_API()`:

```
static DEVICE_API(sensor, thermo_api) = {
        .sample_fetch = thermo_sample_fetch,
        .channel_get = thermo_channel_get,
};
```

`DEVICE_API(sensor, name)` declares a `const struct sensor_driver_api`, and also places it in a linker section of its own, one per API class. That lets `DEVICE_API_IS(sensor, dev)` check at run time whether a device really has a sensor API, rather than trusting whoever passed the pointer.

This is Zephyr's polymorphism: the application's `sensor_sample_fetch()` works the same for Thermo, for a Bosch accelerometer, or for any of the hundreds of sensor drivers in the tree, and none of them knows about the others.

5.1.4 From devicetree node to device

`DT_DRV_COMPAT` and instances

A driver serves every node with a given `compatible`, and says which one at the top of its file:

```
#define DT_DRV_COMPAT qacademy_thermo
```

The value is the compatible `"qacademy,thermo"` in lowercase, with every character that cannot be in a C identifier turned into an underscore. From then on, the `DT_INST_...` macros count the **instances** of that compatible whose `status` is `okay`, numbered 0, 1, 2, ... in an order the build decides. `DT_INST_PROP(inst, conversion_timeout_ms)` is `DT_PROP()` for instance `inst`.

One device per instance

The driver writes a macro that defines everything for one instance, and expands it for all of them:

```
#define THERMO_DEFINE(inst)                                                 \
    static struct thermo_data thermo_data_##inst;                           \
                                                                            \
    static const struct thermo_config thermo_config_##inst = {              \
        .i2c = I2C_DT_SPEC_INST_GET(inst),                                  \
        .conversion_timeout_ms = DT_INST_PROP(inst, conversion_timeout_ms), \
    };                                                                      \
                                                                            \
    SENSOR_DEVICE_DT_INST_DEFINE(inst, thermo_init, NULL,                   \
                                 &thermo_data_##inst,                       \
                                 &thermo_config_##inst, POST_KERNEL,        \
                                 CONFIG_SENSOR_INIT_PRIORITY, &thermo_api);
```

```
DT_INST_FOREACH_STATUS_OKAY(THERMO_DEFINE)
```

With two Thermo nodes in the devicetree, that is two `thermo_data`, two `thermo_config`, and two devices; with none, it is nothing at all. `I2C_DT_SPEC_INST_GET(inst)` fills in the bus device and the address from the node's parent and its `reg` property.

The device macro takes, in order:

Argument	Meaning
`inst`	The instance. The macro turns it into the node, and the node into the device's name and symbol.
`thermo_init`	The init function.
`NULL`	Power management actions, which Thermo has none of.
`&thermo_data_##inst`	`dev->data`. May be `NULL`.
`&thermo_config_##inst`	`dev->config`.
`POST_KERNEL`	The init level.
`CONFIG_SENSOR_INIT_PRIORITY`	The priority within the level.
`&thermo_api`	`dev->api`.

`SENSOR_DEVICE_DT_INST_DEFINE()` is `DEVICE_DT_INST_DEFINE()` plus the sensor subsystem's bookkeeping; other subsystems' drivers use `DEVICE_DT_INST_DEFINE()` directly.

The symbol behind `DEVICE_DT_GET`

A device is a global variable named after the node's position in the devicetree's dependency order: `__device_dts_ord_32` is the device of the node with ordinal 32, and the ordinals are listed at the top of `build/zephyr/include/generated/zephyr/devicetree_generated.h`. `DEVICE_DT_GET(node)` is a reference to that variable.

That explains the error Chapter 2 met, from the other side. `DEVICE_DT_GET()` of an enabled node that no driver defined a device for compiles, since the name is only a symbol, but the link fails:

```
undefined reference to `__device_dts_ord_32'
```

The node exists; nothing defined its device. The causes are a driver that is not enabled, a driver whose `DT_DRV_COMPAT` does not match the node, or a module that is not in the build. In the Thermo example, removing the module from `CMakeLists.txt` does exactly that.

5.1.5 Initialization

Levels and priorities

The kernel initializes devices in levels, in this order:

Level	When
`EARLY`	Right after entering C, before the kernel's architecture-specific initialization.
`PRE_KERNEL_1`, `PRE_KERNEL_2`	Before the kernel's services are available: no sleeping, no kernel objects. Interrupt controllers, clocks, the system timer, consoles.
`POST_KERNEL`	After the kernel is running. Most device drivers, since they may need to sleep or take a mutex.
`APPLICATION`	Just before `main()`.

Within a level, devices run in order of their **priority**, a number from 0 to 999, lowest first. Subsystems give their drivers a Kconfig option for it: I2C controllers use `CONFIG_I2C_INIT_PRIORITY`, which defaults to `CONFIG_KERNEL_INIT_PRIORITY_DEVICE` (50), and sensors use `CONFIG_SENSOR_INIT_PRIORITY` (90). A sensor on an I2C bus therefore initializes after the bus, which it needs.

The build checks that. `CONFIG_CHECK_INIT_PRIORITIES`, on by default, compares the init order in the image with the dependencies in the devicetree, and fails the build if a device initializes before a device its node depends on, such as a sensor before its bus.

To see the order, list the build's init entries:

```
west build -t initlevels
```

For the Chapter 5 exercises' test application with two Lumo parts, the end of the list is:

```
POST_KERNEL
  __init_enable_logger: enable_logger(NULL)
  __init___device_dts_ord_31: i2c_emul_init(__device_dts_ord_31)
  __init___device_dts_ord_32: lumo_init(__device_dts_ord_32)
  __init___device_dts_ord_33: lumo_init(__device_dts_ord_33)
```

The list is read from the image, so it also shows a device whose initialization is deferred (below), which the kernel then skips at boot.

The init function

An init function takes the device and returns 0 or a negative error number:

```
static int thermo_init(const struct device *dev)
{
        const struct thermo_config *config = dev->config;
        uint8_t id;
        int ret;

        if (!i2c_is_ready_dt(&config->i2c)) {
```

```
                return -ENODEV;
        }

        ret = i2c_reg_read_byte_dt(&config->i2c, THERMO_REG_WHO_AM_I, &id);
        if (ret < 0) {
                return ret;
        }

        if (id != THERMO_WHO_AM_I_VALUE) {
                return -ENODEV;
        }

        return 0;
}
```

What belongs in it: checking that the bus is ready, that the part answers, and that it is the part the devicetree says it is; and putting the part into the state the devicetree asks for. What does not: anything slow that a driver could do on first use instead. Every init function runs before `main()`, one after another.

`device_is_ready()`

The kernel records whether a device's init function ran, and what it returned. `device_is_ready(dev)` is true only if both went well:

```
return dev->state->initialized && (dev->state->init_res == 0U);
```

A failed init is final. Nothing retries it, and every call into the device's API would talk to a part that is not there or not right. That is why every user of a device checks `device_is_ready()` once, before using it, and why a driver's init must fail when the part is wrong rather than log a warning and carry on.

Deferred initialization

A node with the property `zephyr,deferred-init` is not initialized at boot. The application initializes it later, when it chooses, with `device_init(dev)`: for example once a power rail has been switched on. `device_init()` returns what the init function returned, or `-EALREADY` if the device was initialized before, **whether or not that went well**. The Thermo tests use exactly this to test a failing init: they break the emulated part, call `device_init()`, and get `-ENODEV`; a second call, after repairing the part, gets `-EALREADY`, and the device stays not ready.

5.1.6 A binding and a Kconfig option for a new part

The binding

Chapter 2 wrote a binding for a node of the application's own. A driver's binding is the same kind of file, and says what a node for the part must look like:

```
description: |
  Thermo, a made-up I2C temperature sensor ...

compatible: "qacademy,thermo"

include: [sensor-device.yaml, i2c-device.yaml]

properties:
  conversion-timeout-ms:
    type: int
    default: 100
```

`i2c-device.yaml` is Zephyr's common binding for a part on an I2C bus: it requires `reg`, the address, and makes the node valid only as a child of an I2C controller. `sensor-device.yaml` adds the properties every sensor may have. A property with an `enum:` list may only take one of the listed values, and `DT_INST_ENUM_IDX(inst, prop)` gives the index of the value the node chose, at compile time.

The vendor prefix

The part before the comma in a compatible is a **vendor prefix**, and Zephyr keeps a list of known ones in `dts/bindings/vendor-prefixes.txt`. A module can add its own, in a file of the same name:

```
qacademy        QRTECH Academy
```

Without it, a normal build only warns, once per node:

```
node '/i2c@100/lumo@29' compatible 'qacademy,lumo' has unknown vendor prefix '↩
    ↪ qacademy'
```

Twister turns that warning into an error. It configures every build with `-DEXTRA_GEN_EDT_ARGS=--edtlib-Werror`, and the same message becomes a `devicetree error:` that stops the configuration. A module that builds fine by hand and fails under Twister has usually missed exactly this file.

The Kconfig option

Every driver has a Kconfig option that decides whether it is compiled:

```
config THERMO
        bool "Thermo I2C temperature sensor"
        default y
        depends on SENSOR
        depends on DT_HAS_QACADEMY_THERMO_ENABLED
        select I2C
```

`DT_HAS_QACADEMY_THERMO_ENABLED` is generated from the devicetree. For every compatible that has a binding, the build writes a symbol into `build/Kconfig/Kconfig.dts` that is `y` when at least one node with that compatible is enabled:

```
DT_COMPAT_QACADEMY_LUMO := qacademy,lumo

config DT_HAS_QACADEMY_LUMO_ENABLED
        def_bool $(dt_compat_enabled,$(DT_COMPAT_QACADEMY_LUMO))
```

With `default y`, the driver is on exactly when the hardware is there, and nobody sets `CONFIG_THERMO` in a `prj.conf`. This is the pattern of nearly every driver in Zephyr, and the reason enabling a driver usually means adding a devicetree node rather than setting an option. The remaining dependencies are the subsystem the driver plugs into (`SENSOR`, which the application enables) and the bus it needs (`I2C`, which the driver selects).

One consequence is easy to miss. A `default` that does not take is not an assignment, so nothing warns about it: an application with a Thermo node but without `CONFIG_SENSOR=y` builds its devicetree, leaves `THERMO` off, and fails at link time with `undefined reference to '__device_dts_ord_32'`.

5.1.7 The sensor API

Fetch and get

Reading a sensor takes two calls:

```
ret = sensor_sample_fetch(dev);
ret = sensor_channel_get(dev, SENSOR_CHAN_AMBIENT_TEMP, &value);
```

`sample_fetch` talks to the part: it starts a measurement if the part needs one, reads the raw result, and stores it in `dev->data`. It can fail, and it can take time. `sensor_sample_fetch(dev)` fetches every channel the part has (`SENSOR_CHAN_ALL`), and `sensor_sample_fetch_chan(dev, chan)` only one.

`channel_get` only converts the stored result into the channel's unit. It does no I/O, and it returns the same value every time until the next fetch. The split lets a part that measures several things at once, such as an accelerometer's three axes, be read in one bus transaction

and then converted channel by channel. Both return `-ENOTSUP` for a channel the part does not have.

`struct sensor_value`

A reading is a fixed-point number in two parts:

```
struct sensor_value {
        int32_t val1;   /* Integer part. */
        int32_t val2;   /* Fractional part, in millionths. */
};
```

Each channel has a unit: degrees Celsius for `SENSOR_CHAN_AMBIENT_TEMP`, lux for `SENSOR_CHAN_LIGHT`. **Both parts carry the sign**: -1.5 degrees is `val1 = -1, val2 = -500000`, and -0.5 degrees is `val1 = 0, val2 = -500000`. A driver should not assemble the two parts by hand; `sensor_value_from_micro(&value, micro)` does it from a value in millionths, and `sensor_value_to_micro(&value)` converts back. Work in `int64_t` millionths: a 16-bit count times 250000 millionths per count does not fit in 32 bits.

The newer, asynchronous API

Zephyr also has an asynchronous sensor API (`CONFIG_SENSOR_ASYNC_API`), built on RTIO, in which a driver provides `submit` and `get_decoder` instead, and readings are decoded into Q31 fixed-point values. It suits high-rate and streaming sensors. The fetch and get API is what most existing drivers implement, and what this book uses; Q31 values reappear in §5.2, in the emulator backend.

5.1.8 Zephyr modules

Out of tree

A driver for a part that Zephyr does not support does not have to live in Zephyr. A **module** is a directory that the build treats as an extension of Zephyr: its CMake is included into Zephyr's build, its Kconfig into Zephyr's configuration, and its bindings into the devicetree search path. Vendor HALs, mbedTLS and LVGL are all modules; so is `examples/thermo/module`.

What makes a directory a module is one file, `zephyr/module.yml`:

```
name: qacademy-thermo
build:
  cmake: .
  kconfig: Kconfig
  settings:
    dts_root: .
```

Key	Meaning
`name`	The module's name. The build also defines a Kconfig symbol for it: `CONFIG_ZEPHYR_QACADEMY_THERMO_MODULE=y`.
`build: cmake`	The directory whose `CMakeLists.txt` the build includes.
`build: kconfig`	The Kconfig file the build sources.
`settings: dts_root`	A directory whose `dts/bindings/` holds bindings, and whose `dts/bindings/vendor-prefixes.txt` holds vendor prefixes.

The module's `CMakeLists.txt` is included after Kconfig has run, so it compiles the driver only when the driver is enabled:

```
add_subdirectory_ifdef(CONFIG_THERMO drivers/sensor/thermo)
```

and the driver's own `CMakeLists.txt` makes it a library of the Zephyr image, as in-tree drivers do:

```
zephyr_library()
zephyr_library_sources(thermo.c)
```

Adding a module to a build

In a west workspace, the modules are the projects of the manifest, as §1.2 described. A module that is not a west project is added by path, in `ZEPHYR_EXTRA_MODULES`, before `find_package(Zephyr)`:

```
list(APPEND ZEPHYR_EXTRA_MODULES ${CMAKE_CURRENT_SOURCE_DIR}/../module)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
```

or on the command line, as `-DZEPHYR_EXTRA_MODULES=<path>`. The build merges the two, so an application can name the modules it always needs in its `CMakeLists.txt`, and a test can add another, such as an emulator, from outside.

5.1.9 Summary

- A driver is an API implementation, an init function, and a macro expanded once for every enabled devicetree node of its compatible.
- A device's `config` is `const`, from devicetree, in flash; its `data` changes at run time, in RAM.
- `DT_DRV_COMPAT` names the compatible; `DT_INST_FOREACH_STATUS_OKAY()` and `DEVICE_DT_INST_DEFINE()` make one device per instance.
- `undefined reference to '__device_dts_ord_N'` means a node that no driver made a device for.
- Devices initialize by level, then by priority. `west build -t initlevels` shows the order, and the build checks it against the devicetree.

- A failed init is final: `device_is_ready()` is false from then on. `zephyr,deferred-init` and `device_init()` move init to a moment the application chooses.
- A new part needs a binding, a vendor prefix, and a Kconfig option that depends on `DT_HAS_<COMPAT>_ENABLED`, so that the devicetree node alone enables the driver.
- The sensor API splits I/O (`sample_fetch`) from conversion (`channel_get`), and returns fixed-point `sensor_values` whose two parts both carry the sign.
- A module packages drivers, bindings and Kconfig outside Zephyr, and is added to a build with `ZEPHYR_EXTRA_MODULES` or a west manifest.

5.2 Emulators and Testing

A driver for a part nobody has is untestable, unless something plays the part. This section covers Zephyr's emulators, which do, and the two tools that run tests against them: the ztest framework, which the tests are written in, and Twister, which builds and runs them. By the end you should be able to explain every file in `lectures/L05/examples/thermo/module/drivers/sensor/thermo/thermo_emul.c` and `lectures/L05/examples/thermo/tests`.

5.2.1 Emulators

What an emulator is

An **emulator** is a piece of software that stands in for a peripheral chip, at the level of the bus the chip is on. The driver under test is the real driver, unchanged: it calls `i2c_reg_read_byte_dt()` as it would on a board. What changes is below it. On `native_sim`, the I2C controller `i2c0` is `zephyr,i2c-emul-controller`, a driver that, instead of clocking bits onto wires, hands every transfer to the emulator registered at the transfer's address.

```
application ── sensor API ──> thermo.c ── I2C API ──> i2c_emul ──> thermo_emul.c
                              (driver)                (i2c0)       (the "chip")
```

That is what makes emulators more useful than mocks: the driver's I2C calls, its register addresses, its byte order and the order of its transfers all run for real, and only the silicon is software.

The emulated I2C controller needs `CONFIG_EMUL=y`. Without it, `native_sim`'s `i2c0` has no driver, and a sensor on that bus fails to link: in the exercises' application, with `undefined reference to '__device_dts_ord_31'`, where node 31 is `i2c@100`.

Defining one

An emulator is defined per devicetree node, like a device, and usually for the same compatible as the driver:

```
#define DT_DRV_COMPAT qacademy_thermo

#define THERMO_EMUL_DEFINE(inst)                                          \
    static struct thermo_emul_data thermo_emul_data_##inst;               \
    EMUL_DT_INST_DEFINE(inst, thermo_emul_init, &thermo_emul_data_##inst, \
                        NULL, &thermo_emul_i2c_api, &thermo_emul_sensor_api)

DT_INST_FOREACH_STATUS_OKAY(THERMO_EMUL_DEFINE)
```

`EMUL_DT_INST_DEFINE()` takes the instance, an init function, the emulator's data and configuration, the **bus API** it answers transfers with, and an optional **backend API** that tests use to control it. The result is a `struct emul`, which tests get with `EMUL_DT_GET(node)`, just as they get the device with `DEVICE_DT_GET(node)`.

When the emulated I2C controller initializes, it looks up an emulator for each of its child nodes and registers it at the node's address. That is logged at boot:

```
<inf> emul: Registering 1 emulator(s) for i2c@100
<inf> i2c_emul_ctlr: Register emulator 'lumo@29' at I2C addr 29
```

A node without an emulator is logged as `Cannot find emulator for 'lumo@29'`, and every transfer to its address fails with `-EIO`: the bus behaves as if nothing answered there. The controller initializes before the drivers on its bus, so every emulator is in its reset state by the time a driver's init function reads from it.

The bus API

For I2C, the bus API is one function:

```
static const struct i2c_emul_api thermo_emul_i2c_api = {
        .transfer = thermo_emul_transfer,
};
```

It receives the messages of one I2C transfer, exactly as the driver's call produced them. The common register helpers arrive in two shapes:

- `i2c_reg_write_byte_dt()` is **one write message**: the register address, then the value.
- `i2c_reg_read_byte_dt()` and `i2c_burst_read_dt()` are **two messages**: a write of the register address, then a read of one or more bytes, which by convention continue at the following registers.

An emulator's transfer function is therefore a small register map: it keeps the registers in RAM, updates them on writes, and fills in read buffers from them, doing whatever the datasheet says happens in between, such as clearing a one-shot bit when a conversion finishes. Everything a real part would get wrong-footed by, such as a write to a read-only register,

the emulator decides how to handle; Thermo's ignores it and logs a warning, as the part would ignore it.

The backend API

Tests need to set what the part measures. They could call functions of the emulator's own, and Thermo's emulator does have a header of its own, `thermo_emul.h`, for faults no real part has on purpose: a wrong `WHO_AM_I`, a conversion that never finishes.

For the measured value itself there is a generic interface, the **sensor emulator backend** in `<zephyr/drivers/emul_sensor.h>`:

```
static const struct emul_sensor_driver_api thermo_emul_sensor_api = {
        .set_channel = thermo_emul_set_channel,
        .get_sample_range = thermo_emul_get_sample_range,
};
```

and tests call it through `emul_sensor_backend_set_channel()`, which works the same for every sensor emulator in Zephyr that implements it. The value is passed as a **Q31** fixed-point fraction with a shift: the physical value is `value / 2^31 * 2^shift`, in the channel's unit. A shift of 8 covers plus or minus 256 degrees; a shift of 15, 32768 lux. Converting a value in millionths:

```
const int8_t shift = 8;
q31_t value = (q31_t)((micro_c * (1LL << (31 - shift))) / 1000000);

emul_sensor_backend_set_channel(
        emul, (struct sensor_chan_spec){SENSOR_CHAN_AMBIENT_TEMP, 0}, &value, ↩
    ↪ shift);
```

The emulator converts the value into what the part would register, rounded as the part would round it. The test then reads it back through the driver and checks the conversion end to end. The emulated I2C bus also has a lower-level escape hatch, `mock_api` on `struct i2c_emul`, which lets a test intercept transfers itself, for faults such as a bus that returns an error.

What an emulator can and cannot show

An emulator shows whether a driver talks to the part the way the datasheet says: the right registers, the right byte order, the right conversion, the right reaction to a wrong ID or a timeout. It shows it deterministically, on every commit, without hardware.

It cannot show what the author of the emulator did not know or did not model: a real part that needs 30 ms before it answers after power-up, a measurement that changes between reading two registers, electrical noise, or a datasheet that is wrong. An emulator is a second implementation of the datasheet, and a driver that passes against it agrees with that implementation. The first run on a board is still a test.

5.2.2 ztest

A test application

A ztest test is an ordinary Zephyr application, built for a board, whose `main()` is provided by the framework. `prj.conf` enables it:

```
CONFIG_ZTEST=y
```

and the tests are functions in suites:

```
#include <zephyr/ztest.h>

ZTEST_SUITE(thermo, NULL, NULL, NULL, after, NULL);

ZTEST(thermo, test_zero)
{
        zassert_ok(set_temperature(0));

        struct sensor_value value = read_temperature();

        zassert_equal(value.val1, 0);
        zassert_equal(value.val2, 0);
}
```

On `native_sim`, the program runs every suite, prints a summary, and exits with status 0 if every test passed and 1 if any failed, which is what Twister, and the repository's `make test`, check. Tests built with ztest also run in simulated time as fast as the host allows: `native_sim` only slows down to real time by default when the build is not a test.

```
SUITE PASS - 100.00% [thermo]: pass = 11, fail = 0, skip = 0, total = 11 duration ↩
    ↪ = 0.000 seconds
 - PASS - [thermo.test_channel_get_returns_the_last_fetch] duration = 0.000 ↩
    ↪ seconds
 ...
PROJECT EXECUTION SUCCESSFUL
```

Suites and hooks

`ZTEST_SUITE(name, predicate, setup, before, after, teardown)` declares a suite and, optionally, functions that run around it:

Argument	Runs
`predicate`	Before the suite, to decide whether it runs at all.
`setup`	Once, before the suite's first test. Its return value is the suite's *fixture*.
`before`	Before every test.
`after`	After every test, whether it passed or failed.
`teardown`	Once, after the suite's last test.

`ZTEST(suite, name)` defines a test; `ZTEST_F(suite, name)` defines one that receives the fixture. Thermo's suite uses `after` to repair whatever fault a test injected, so that a failing test cannot make the next one fail too.

Tests run in alphabetical order, not in the order they appear in the file. A test that only passes because another ran first is a test that will one day fail for no reason; every test must set up everything it depends on, and the hooks exist for exactly that.

Assertions

Macro	On failure
`zassert_true(cond, msg, ...)`, `zassert_false`, `zassert_equal(a, b, ...)`, `zassert_ok(expr)`, …	Fails the test and ends it.
`zexpect_true`, `zexpect_equal`, …	Fails the test, but lets it continue, to report more than the first failure.
`zassume_true`, `zassume_equal`, …	Skips the rest of the test: for a precondition that is not what is being tested.

Every assertion takes an optional `printf`-style message, which is printed with the file, the line and the failed condition. A good message says what should have happened, in the terms of the datasheet or the exercise, not only that two numbers differed:

```
Assertion failed at .../conversion/src/main.c:71: conversion_test_saturated: (↩
    ↪ value.val1 not equal to 16383)
0xFFFF counts is 16383.75 lux, not -796.-119184
```

What to test in a driver

Against an emulator, a driver's tests cover what a datasheet specifies and what a user relies on:

- **Readiness**: the device is ready when the part is genuine, and not when it is not.
- **Conversion at the edges**: zero, the smallest step, negative values if the part has them, and the limits of the range, where overflows and sign errors live.
- **Configuration from devicetree**: what a property should put into a register, read back from the emulator.
- **The API contract**: unsupported channels, fetch without get and get without fetch.

- **Faults**: a timeout, a wrong ID, a bus error. These are the tests only an emulator makes cheap.

5.2.3 Twister

Test applications as scenarios

Twister is Zephyr's test runner, and the same tool the repository's `make examples` uses. It walks a directory tree, finds every application with a `testcase.yaml` (or, for samples, a `sample.yaml`), and builds, runs and checks each of its **scenarios** on each selected platform, in parallel.

The Thermo tests' `testcase.yaml`:

```
common:
  platform_allow: native_sim/native/64
  integration_platforms:
    - native_sim/native/64
  harness: ztest
  tags:
    - sensor
tests:
  qacademy.l05.thermo.driver: {}
```

Key	Meaning
`tests`	The scenarios, by name. Each builds the application once; keys under a scenario override `common`.
`platform_allow`	The platforms the scenario may run on.
`integration_platforms`	The platforms it runs on when Twister is run with `--integration`, as Zephyr's CI does for pull requests.
`harness: ztest`	How to judge the run: parse ztest's output, and report every test case. A sample uses `harness: console` with regular expressions instead.
`extra_configs`, `extra_args`	Kconfig settings and CMake arguments for the scenario, such as `CONFIG_ASSERT=y` or `EXTRA_CONF_FILE=...`.
`tags`, `filter`	Selecting scenarios: by tag on the command line, or by an expression over the build's configuration.

A suite and a scenario are different things. A ztest **suite** is a group of tests inside one program. A Twister **scenario** is one build of the program: the same tests built twice, once with `CONFIG_FOO=y` and once without, are two scenarios running the same suites.

Running it

```
west twister -T lectures/L05/examples/thermo/tests -p native_sim/native/64
```

`-T` gives the directory to search, and `-p` the platform. Twister writes everything to `twister-out/` in the current directory, renaming an older one to `twister-out.1`: one build directory per scenario, under `twister-out/native_sim_native_64/host_gnu/<path>/<scenario>/`, with the program's output in `handler.log` and the build's in `build.log`, and reports in `twister-out/twister.json` and `twister.xml`. `-i` (`--inline-logs`) prints the log of every failure in the terminal instead.

Twister is stricter than a plain build. It builds with compiler warnings as errors (`CONFIG_COMPILER_WARNINGS_AS_ERRORS=y`) and with devicetree warnings as errors (`--edtlib-Werror`), which is how a missing `vendor-prefixes.txt` that only warned by hand stops a Twister run. `--disable-warnings-as-errors` turns both off, and should not be needed.

Extra CMake arguments for every build go after `-x`, for example a module that is not in the application's `CMakeLists.txt`:

```
west twister -T <tests> -p native_sim/native/64 \
    -x=ZEPHYR_EXTRA_MODULES=<path to module>
```

With `--coverage`, Twister builds with gcov instrumentation and collects coverage of the code the tests ran into an HTML report. Chapter 6 shows it.

The same `testcase.yaml` runs on real boards too, with `--device-testing` and a serial port: the scenarios that pass here against an emulator are what **Hardware-in-the-Loop Testing** runs against the real part.

5.2.4 Summary

- An emulator plays a peripheral chip at its bus. The driver under test is unchanged; on `native_sim`, the emulated I2C controller hands each transfer to the emulator at its address.
- `EMUL_DT_INST_DEFINE()` defines an emulator per devicetree node, with a bus API that answers transfers and an optional backend API that tests control it through. `EMUL_DT_GET()` gets it.
- The sensor emulator backend sets a measured value as Q31 with a shift, the same way for every sensor emulator. Faults no real part has on purpose go in the emulator's own header.
- An emulator is a second implementation of the datasheet: it tests the driver's agreement with it, not with the silicon.
- ztest applications are Zephyr applications; `ZTEST_SUITE()` declares a suite and its hooks, `ZTEST()` a test. Tests run in alphabetical order, so none may depend on another.

- `zassert_*` ends a test, `zexpect_*` lets it continue, and `zassume_*` skips it. Say in the message what should have happened.
- Twister finds `testcase.yaml` files, builds every scenario, runs it, and judges it by its harness, with warnings as errors. A suite is tests in a program; a scenario is a build of a program.

5.3 Review

What you should be able to do now

After this chapter, you should:

- Understand how Zephyr turns a devicetree node into a `struct device` at compile time.
- Be able to write a driver that serves every instance of its compatible in the devicetree.
- Understand the order devices are initialized in, and how to find it for a build.
- Be able to implement a subsystem API, so that applications use the part without knowing it.
- Be able to package drivers, bindings and Kconfig as a Zephyr module.
- Be able to write an emulator that lets a driver run, and fail, without hardware.
- Be able to test a driver with ztest, and run the tests with Twister.

Questions to test yourself

You should be able to explain:

1. What is the difference between a device's `config` and its `data`, and where does each one live?
2. How does `DT_DRV_COMPAT` connect a driver to the devicetree nodes it serves?
3. What decides the order in which devices are initialized, and how can you see it for a build?
4. What does `device_is_ready()` return for a device whose init function failed?
5. Why does the driver's Kconfig option depend on `DT_HAS_QACADEMY_THERMO_ENABLED`?
6. Why does a sensor driver split reading into `sample_fetch` and `channel_get`?
7. What does an emulator make testable that a real part does not?
8. What is the difference between a ztest suite and a Twister scenario?
9. What goes wrong without the module's `vendor-prefixes.txt`, and why only under Twister?

Looking ahead

Chapter 6 is about subsystems and debugging:

- The shell, with commands of your own.
- Settings that survive a restart, on the simulated flash.
- Networking through the host's sockets.
- Debugging on the host: GDB, sanitizers, fatal errors and coverage.

You will make the program talk back, remember things, and crash usefully.

5.4 Exercises

These exercises reinforce the concepts from §5.1 and §5.2.

> **How to check your work.** Write each exercise set in its own directory under `lectures/L05/exercises`, with the file names given there, and run `make test` from the repository root. A set's tests switch on as soon as its files exist, and a set you have not started is reported as `SKIP`. Sets 1 to 3 are one driver growing: start each by copying the previous set's directory. The tests of Sets 1 to 3 are ztest applications in `lectures/L05/exercises/test` that build your module against the course's emulator; Set 4 is checked by its output, and Set 5 by running your own tests. The solutions are published in the companion repository, under `lectures/L05/appendix/solutions`, a few days after the lecture. Try each exercise yourself before you read them.

Remember `source env.sh` in every new terminal.

5.4.1 The part: Lumo

Lumo is a made-up I2C ambient light sensor. It measures continuously: its DATA registers always hold the latest measurement, and there is nothing to start. Its datasheet, in full:

Register	Name	Access	Contents
`0x00`	`CONFIG`	R/W	Bits 1:0, `GAIN`: `0` is a gain of 1, `1` of 2, `2` of 4, `3` of 8. Bits 7:2 are reserved, read as 0 and ignore writes. Reset value `0x00`.
`0x01`	`DATA_LSB`	R	The measurement, low byte.
`0x02`	`DATA_MSB`	R	The measurement, high byte.
`0x0A`	`PART_ID`	R	Always `0x4C` on a genuine part.

- The measurement is an **unsigned 16-bit count, little-endian**: `DATA_LSB` then `DATA_MSB`.
- At a gain of 1, **one count is 0.25 lux**. At a gain of G, one count is 0.25 / G lux.
- A count of `0xFFFF` means the part is **saturated**: the light is at least that bright.
- A read of several bytes continues at the following registers, so both DATA bytes can be read in one transfer. Writes to read-only registers are ignored.

The course provides the part itself: an emulator, in `lectures/L05/exercises/test/lumo_emul`, which is a Zephyr module of its own. It plays a Lumo chip for every enabled `"qacademy,lumo"` node on `native_sim`'s I2C bus. The tests add it to their builds; to build with it by hand, add it on the command line:

```
-DZEPHYR_EXTRA_MODULES=$PWD/lectures/L05/exercises/test/lumo_emul
```

Tests control it through that module's own header, `<qacademy/lumo_emul.h>` (`lumo_emul_set_part_id()`, `lumo_emul_set_counts()` and `lumo_emul_get_config()`), and through the generic sensor emulator backend, with `SENSOR_CHAN_LIGHT` in lux.

EXERCISE SET 1

A module and a device

Create a directory named `exercise1` to work in for this exercise set. The directory is your module.

Exercise 5.1 The module and its Kconfig option

CODE

In this exercise you will create a module that the build can include.

Tasks

Create these files in `exercise1`:

- `zephyr/module.yml`: a module named `qacademy-lumo`, whose CMake and Kconfig are in the module's top directory, and whose `dts_root` is the top directory too.
- `Kconfig`: an option `LUMO`, a `bool` with the prompt `Lumo I2C ambient light sensor`, which:
  - is `y` by default,
  - depends on `SENSOR`, and on the devicetree having an enabled `"qacademy,lumo"` node,
  - selects `I2C`.
- `CMakeLists.txt`: when `CONFIG_LUMO` is enabled, a Zephyr library holding `src/lumo.c`.

Exercise 5.2 The binding

CODE

Tasks

- Create `dts/bindings/qacademy,lumo.yaml`: a binding for the compatible `"qacademy,lumo"`, which includes `sensor-device.yaml` and `i2c-device.yaml`, and has no properties of its own yet.
- Create `dts/bindings/vendor-prefixes.txt`, registering the vendor prefix `qacademy`.

Exercise 5.3 The driver's init

CODE

Tasks

- Create `src/lumo_reg.h` with macros for the register addresses, the `GAIN` mask, and the value of `PART_ID`, from the datasheet above.
- Create `src/lumo.c`, the driver for the compatible `"qacademy,lumo"`:
  - A config struct holding the part's `struct i2c_dt_spec`.
  - A sensor API table, defined with `DEVICE_API(sensor, ...)`, whose `sample_fetch` and `channel_get` functions only return `-ENOTSUP` for now.
  - An init function that:
    - returns `-ENODEV` if the I2C bus is not ready,
    - reads `PART_ID`, and returns the error if the read fails,
    - returns `-ENODEV` if `PART_ID` is not `0x4C`,
    - and otherwise returns 0.
  - One device per enabled instance, defined with `SENSOR_DEVICE_DT_INST_DEFINE()` at `POST_KERNEL` and `CONFIG_SENSOR_INIT_PRIORITY`. The data pointer may be `NULL` for now.

The tests in `lectures/L05/exercises/test/device` put one part at address `0x29`, and a second at `0x2a` whose initialization is deferred, so that a test can give it the wrong `PART_ID` first. To build and run them by hand:

```
west build -p -b native_sim/native/64 -d build/device \
    lectures/L05/exercises/test/device -- \
    -DEXERCISE_DIR=$PWD/lectures/L05/exercises/exercise1
./build/device/zephyr/zephyr.exe
```

The four tests must pass: the option turns itself on, the part is ready, its device has a sensor API, and the wrong `PART_ID` makes `device_init()` return `-ENODEV`.

Reflection

- Remove `vendor-prefixes.txt` and build the tests again by hand. What does the build print? Then build once more with `-DEXTRA_GEN_EDT_ARGS=--edtlib-Werror` added, as Twister does. What happens?
- Run `west build -d build/device -t initlevels`. Where do your two `lumo_init` calls come in the list, and what comes just before them? Why must it be in that order?
- The deferred part is in the `initlevels` list too. Why is it still not ready after boot?
- Find `DT_HAS_QACADEMY_LUMO_ENABLED` in the build directory. Which file defines it, and why could it not exist before you wrote the binding?

EXERCISE SET 2

Reading light

Copy `exercise1` to a directory named `exercise2` to work in for this exercise set.

Exercise 5.4 Fetch and get

CODE

In this exercise you will implement the sensor API, at the part's reset gain of 1.

Tasks

In `src/lumo.c`:

- Add a data struct holding the count of the last fetch, and pass it to the device.
- `sample_fetch` shall:
  - return `-ENOTSUP` for any channel other than `SENSOR_CHAN_ALL` and `SENSOR_CHAN_LIGHT`,
  - read both DATA bytes **in one transfer**, and return the error if the read fails,
  - store the count, and return 0.
- `channel_get` shall:
  - return `-ENOTSUP` for any channel other than `SENSOR_CHAN_LIGHT`,
  - convert the stored count to lux, at 0.25 lux per count, into the `struct sensor_value`.
- Do no I/O in `channel_get`.

The tests in `lectures/L05/exercises/test/conversion` check, among others, that:

Count	Lux	val1	val2
0	0	0	0
1	0.25	0	250000
0x0102	64.5	64	500000
0xFFFF	16383.75	16383	750000

The tests of Set 1 must still pass.

Hint: `sys_get_le16()`, from `<zephyr/sys/byteorder.h>`, turns two little-endian bytes into a `uint16_t`. `sensor_value_from_micro()` fills in a `struct sensor_value` from millionths.

Reflection

- Replace the single transfer with two `i2c_reg_read_byte_dt()` calls, one per DATA byte. Do the tests still pass? What could go wrong on a real part that the emulator cannot show?
- Compute the lux in `int` instead of `int64_t`, by removing the cast. Which test fails? Above how many counts does the multiplication overflow, and how bright is that?
- `test_channel_get_returns_the_last_fetch` changes the part's count between a fetch and a get. Why is the old value the right answer?

EXERCISE SET 3

The gain

Copy `exercise2` to a directory named `exercise3` to work in for this exercise set.

Exercise 5.5 A devicetree property

CODE

In this exercise you will make the gain a property of the devicetree node, written to the part at init.

Tasks

- In the binding, add a property `gain`: an `int`, allowed to be only 1, 2, 4 or 8, with a default of 1.
- In `src/lumo.c`:
  - Add the gain and its `GAIN` bits to the config struct. Take the bits from the property's index in the binding's list of allowed values, at compile time.
  - In the init function, after checking `PART_ID`, write the `GAIN` bits to `CONFIG`, without changing the register's other bits, and return the error if that fails.
  - In `channel_get`, divide by the gain: at a gain of G, one count is 0.25 / G lux.

The tests in `lectures/L05/exercises/test/gain` put three parts on the bus, at gains of 4, 8 and the default, and check what each one's `CONFIG` register holds after boot and what its counts convert to: one count at a gain of 4 is 0.0625 lux, and `0xFFFF` at a gain of 8 is 2047.96875 lux. The tests of Sets 1 and 2 must still pass.

Hint: `DT_INST_ENUM_IDX(inst, prop)` is the index of a property's value in its binding's `enum` list. `i2c_reg_update_byte_dt()` changes only the bits of a register that a mask selects.

Reflection

- Add `gain = <3>;` to a part in an overlay and build the gain tests. What happens, and at which step of the build?
- Why take the `GAIN` bits from `DT_INST_ENUM_IDX()`, rather than computing a base-2 logarithm of the gain at run time?
- At a gain of 8, what is the smallest step between two readings, and the brightest light the part can measure? What are they at a gain of 1?

EXERCISE SET 4

An application

Create an application named `exercise4` to work in for this exercise set.

Exercise 5.6 Reading once a second

CODE

Tasks

- `CMakeLists.txt`: add the module in `../exercise3` to the build, before finding Zephyr. Do **not** add the emulator.
- `prj.conf`: enable the sensor subsystem, emulators, and logging.
- `app.overlay`: a Lumo part labelled `light`, at address `0x29` on `&i2c0`, at the default gain.
- `src/main.c`, with a log module named `light_app` at level `INF`:
  - If the part is not ready, log `<name> is not ready` as an error, with the device's name, and return.
  - Log `Reading <name>`.
  - Then, forever: fetch and get a light reading, log it as `<lux> lux` with two decimals, or `Reading failed: <error>` as an error, and sleep for one second.

Build it with the emulator, and with the emulator's sunrise, which changes the light half a second before each whole second:

```
west build -p -b native_sim/native/64 lectures/L05/exercises/exercise4 -- \
    -DZEPHYR_EXTRA_MODULES=$PWD/lectures/L05/exercises/test/lumo_emul \
    -DCONFIG_EMUL_LUMO_DAYLIGHT=y
./build/zephyr/zephyr.exe -stop_at=6.5
```

The output must contain, in this order, with the emulator's own log lines in between:

```
[00:00:00.000,000] <inf> light_app: Reading lumo@29
[00:00:00.000,000] <inf> light_app: 0.00 lux
[00:00:01.010,000] <inf> light_app: 0.25 lux
[00:00:02.020,000] <inf> light_app: 42.00 lux
[00:00:03.030,000] <inf> light_app: 320.00 lux
[00:00:04.040,000] <inf> light_app: 2750.00 lux
[00:00:05.050,000] <inf> light_app: 12000.00 lux
[00:00:06.060,000] <inf> light_app: 16383.75 lux
```

Reflection

- Build the application without `-DZEPHYR_EXTRA_MODULES=...`. Does it build? What does it print, and why?
- Build it with the emulator, but with `-DCONFIG_EMUL=n` added. What happens, and which devicetree node is the error about?

- The emulator's last light level is 30000 lux, but the reading is 16383.75. Why? Set `gain = <8>` in `app.overlay` and run it again: what do the readings become, and what does that say about choosing a gain?

EXERCISE SET 5

Tests of your own

Create a directory named `exercise5` to work in for this exercise set: a ztest application that tests the driver of Exercise Set 3.

Exercise 5.7 A test suite and a Twister scenario

CODE

Tasks

- `CMakeLists.txt`: add the module in `../exercise3` to the build, and `src/main.c` to the application. As in Set 4, do not add the emulator.
- `prj.conf`: enable ztest, the sensor subsystem and emulators.
- `app.overlay`: two Lumo parts, one at the default gain and one at a gain of 2.
- `src/main.c`: a suite named `lumo`, whose tests check at least:
  - that both parts are ready,
  - counts of 0, 1 and `0xFFFF` at the default gain,
  - a count at a gain of 2,
  - that the gain of 2 was written to the part's `CONFIG` register,
  - and that an unsupported channel returns `-ENOTSUP` from both fetch and get.
- `testcase.yaml`: one scenario, named `qacademy.l05.exercise5.lumo`, allowed only on `native_sim/native/64`, judged by the ztest harness.

Build and run it with the emulator:

```
west build -p -b native_sim/native/64 lectures/L05/exercises/exercise5 -- \
    -DZEPHYR_EXTRA_MODULES=$PWD/lectures/L05/exercises/test/lumo_emul
./build/zephyr/zephyr.exe
```

The suite must pass: the output must contain `SUITE PASS - 100.00% [lumo]` and `PROJECT EXECUTION SUCCESSFUL`.

Reflection

- Run the scenario with Twister:

```
west twister -T lectures/L05/exercises/exercise5 -p native_sim/native/64 \
    -x=ZEPHYR_EXTRA_MODULES=$PWD/lectures/L05/exercises/test/lumo_emul
```

Where does Twister put the output of your test program? What does it report besides pass and fail?

- Break your Set 3 driver on purpose: make `channel_get` ignore the gain. Which of your tests fail? If none do, which test is missing?
- Your tests and the course's tests both pass. What would you still check on the first board with a real Lumo part?

CHAPTER 6

Subsystems and Debugging

Making a program talk back, remember things, and crash usefully.

IN THIS CHAPTER

- The shell: built-in commands, commands of your own with subcommands and arguments, and changing log levels at run time.
- The settings subsystem: key-value pairs that survive a restart, stored with ZMS on flash.
- `native_sim`'s simulated flash: a file, and the command-line options that control it.
- Networking on `native_sim` through offloaded sockets: Zephyr's socket API on the host's sockets.
- Fatal errors: `k_oops()`, `k_panic()`, assertions, and `k_sys_fatal_error_handler()`.
- The host's tools on a Zephyr program: AddressSanitizer and UndefinedBehaviorSanitizer.
- Debugging a `native_sim` program with GDB.
- Code coverage with gcov and Twister.
- What moving to real hardware would change, and what it would not.

This last chapter is about the parts of Zephyr that make a program usable by someone other than its author. A shell, so a running system can be asked what it is doing. Settings that survive a restart, on the flash that `native_sim` keeps in a file. Sockets, which on this board are the host's own, so a Zephyr program can be talked to from an ordinary terminal.

Then it is about what to do when the program is wrong. Zephyr's fatal error handling decides what an impossible value costs, and the host's tools, AddressSanitizer, UndefinedBehaviorSanitizer, GDB and gcov, work on a `native_sim` program exactly as on any other Linux program. That is the best argument this book has for keeping code runnable on `native_sim` even when the product is a board.

6.1 Subsystems: the Shell, Settings and Networking

Chapter 1 to Chapter 5 built programs that print. This section covers three subsystems that make a program part of a system: a shell to talk to it, settings to make it remember, and sockets to put it on a network. All three are the same code on a board and on `native_sim`; what

`native_sim` puts under them, a terminal, a file and the host's network, is what this section is careful to separate from the subsystems themselves. By the end you should be able to explain every file in the `lectures/L06/examples/shell`, `lectures/L06/examples/settings` and `lectures/L06/examples/sockets` examples.

6.1.1 The shell

What it is

Zephyr's shell is a command line inside the firmware: line editing, history, `Tab` completion, a `help` for every command, and colors. Subsystems register commands of their own, so enabling the shell alone already gives you commands for the kernel, and enabling logging or settings with it gives you commands for those. An application adds its own the same way.

```
CONFIG_SHELL=y
```

The shell runs in a thread of its own, and reads and writes through a **backend**. The usual one is the serial backend, on the UART that the devicetree's `chosen` node names `zephyr,shell-uart`; `native_sim`'s devicetree names `uart0`. Others carry the same shell over SEGGER RTT, telnet, a WebSocket, MQTT or RPMsg, and a *dummy* backend runs commands from code, which is how the shell is tested. The thread of the serial backend is called `shell_uart`.

The shell on `native_sim`

`native_sim`'s `uart0` is a pseudo-terminal by default. The program prints `uart connected to pseudotty: /dev/pts/11` (the number varies), and you connect to that from a second terminal. For a course, and for tests, it is simpler to put the UART on the terminal the program was started from:

```
CONFIG_UART_NATIVE_PTY_0_ON_STDINOUT=y
```

or, without rebuilding, the command-line option `-uart_stdinout`. Commands can then be typed, or piped in, which is how a session is scripted:

```
printf 'greet Ada\nled on\nled status\n' | ./build/zephyr/zephyr.exe -stop_at=1
```

The shell echoes each command after its prompt, `uart:~$`, and prints the response on the lines that follow.

Commands

A command is a handler function and a static entry that describes it. Every handler has the same signature:

```c
static int cmd_greet(const struct shell *sh, size_t argc, char **argv)
{
        shell_print(sh, "Hello, %s!", argv[1]);
        return 0;
}
```

- **`sh`** is the shell instance that ran the command, and the one its output must go to: there may be several backends, each with a shell of its own.
- **`argv`** is the command line split into words, **`argv[0]`** being the command's own name, and **`argc`** is how many there are.
- **The return value** is `0` on success and a negative `errno` on failure.

A command with no subcommands is registered with its argument counts:

```c
SHELL_CMD_ARG_REGISTER(greet, NULL, "Greet someone: greet <name> [times]", ↩
    ↪ cmd_greet, 2, 1);
```

The last two numbers are the **mandatory** and **optional** word counts, and the mandatory count **includes the command's own name**: `2, 1` means `greet` and a name, and optionally one more word. The shell checks the count before it calls the handler, so the handler never sees `argc` outside the range. A wrong count prints the command's help instead:

```
uart:~$ greet
greet: wrong parameter count
greet - Greet someone: greet <name> [times]
```

`SHELL_CMD_REGISTER()` is the same without counts, for a command that takes no arguments or checks them itself. Commands with **subcommands** have a table of them, built at compile time:

```c
SHELL_STATIC_SUBCMD_SET_CREATE(sub_led,
        SHELL_CMD(on, NULL, "Turn the LED on.", cmd_led_on),
        SHELL_CMD(off, NULL, "Turn the LED off.", cmd_led_off),
        SHELL_CMD_ARG(blink, NULL, "Blink: blink <times>", cmd_led_blink, 2, 0),
        SHELL_SUBCMD_SET_END);

SHELL_CMD_REGISTER(led, &sub_led, "Control the LED.", NULL);
```

A subcommand is written with `SHELL_CMD()` or, with counts, `SHELL_CMD_ARG()`, and its counts again include its own name: in `led blink 3`, the handler for `blink` gets `argv[0] = "blink"` and `argv[1] = "3"`. A command with subcommands and no handler of its own, `NULL` above, prints its help and the list of subcommands when it is run alone. Because the tables are static, `Tab` can complete them. (`SHELL_DYNAMIC_CMD_CREATE()` creates subcom-

mands at run time, such as a list of device names, and `SHELL_COND_CMD_REGISTER()` registers a command only when a Kconfig option is enabled.)

Output

A handler prints through the shell it was called by, never with `printk()`, whose output would go to the console and not necessarily to the backend the user is on:

Function	Output
`shell_print(sh, fmt, ...)`	Normal output.
`shell_info(sh, fmt, ...)`	Informational output, in green.
`shell_warn(sh, fmt, ...)`	A warning, in yellow.
`shell_error(sh, fmt, ...)`	An error, in red.
`shell_hexdump(sh, data, len)`	A buffer, as a hex dump.

The functions add the newline themselves.

Validating arguments

The shell counts words; what the words say is the handler's business. `atoi()` is the wrong tool, because it cannot fail: `atoi("warm")` is 0 and `atoi("25x")` is 25. `strtol()` reports where it stopped and whether the value overflowed, so a word is a number exactly when it stopped at the end of the word, after at least one digit, with `errno` still 0:

```
char *end;

errno = 0;
long value = strtol(argv[1], &end, 10);

if ((end == argv[1]) || (*end != '\0') || (errno != 0)) {
        shell_error(sh, "not a temperature: '%s'", argv[1]);
        return -EINVAL;
}
```

A value that is a number but out of range is a different error, conventionally `-ERANGE`.

Return values and `retval`

The shell does not print a handler's return value, but it keeps it: the built-in command `retval` prints the value the previous command returned. That makes errors visible to a script, and makes the shell's own conventions visible to you:

```
uart:~$ greet Ada 9
times must be between 1 and 5, not '9'
uart:~$ retval
-22
```

```
uart:~$ foo
foo: command not found
uart:~$ retval
-8
```

`-22` is `-EINVAL`, returned by the handler; `-8` is `-ENOEXEC`, returned by the shell for a command that does not exist. A command with subcommands run without one returns `1` (`SHELL_CMD_HELP_PRINTED`), since all it did was print its help.

The built-in commands

With the shell come commands from the subsystems that are enabled. In Zephyr 4.4:

Command	From	What it does
`help`, `history`, `clear`, `retval`	The shell itself, `help` always and the others by default	The shell's own commands.
`kernel uptime`, `kernel version`	The kernel shell (`CONFIG_KERNEL_SHELL`, on with the shell)	The system's uptime and Zephyr's version.
`kernel thread list`, `kernel thread stacks`	The kernel shell	Every thread, its priority, state and stack usage. Zephyr 3.7 called them `kernel threads` and `kernel stacks`.
`log status`, `log enable <level> <module>`, `log disable <module>`	Logging, with `CONFIG_LOG_CMDS=y`	Every log module's level, changed at run time.
`settings list`, `settings read <name>`, `settings write`, `settings delete`	Settings, with `CONFIG_SETTINGS_SHELL=y`	The stored settings.

Two of them deserve a warning on `native_sim`:

- **`kernel thread list`** prints stack usage such as `usage 40 / 3072 (1 %)`, which means nothing here: Chapter 3 showed that threads run on host stacks. On a board it is how stacks are sized.
- **`log enable dbg app`** answers `app: level set to inf.` for a module compiled at `LOG_LEVEL_INF`. A level can be lowered at run time, never raised above what was compiled in, because the messages above it are not in the program at all (Chapter 1).

6.1.2 Settings

What it is

The settings subsystem stores **key-value pairs** in non-volatile memory, so that a device remembers them across a restart: a calibration, a network name, a setpoint. The keys are names made of parts separated by `/`, such as `thermostat/setpoint`, and the first part is a

subtree that one module owns. The values are bytes, of whatever size and type the owner decides.

```
CONFIG_SETTINGS=y
```

The subsystem does not write to memory itself. It hands the pairs to a **backend**, chosen with a Kconfig `choice`. Its defaults include, in order:

Backend	Chosen by default when
ZMS, Zephyr Memory Storage	`CONFIG_ZMS=y`
NVS, Non-Volatile Storage	`CONFIG_NVS=y`
FCB, Flash Circular Buffer	`CONFIG_FCB=y`
A file	`CONFIG_FILE_SYSTEM=y`
None	nothing else is enabled: settings then live in RAM only

ZMS and NVS are both log-structured key-value stores for flash: they append new values and erase whole sectors when the old values are no longer needed, so that no single flash cell wears out early. ZMS is the newer of the two, and is designed for memories that need no erase as well. The course uses ZMS:

```
CONFIG_FLASH=y
CONFIG_FLASH_MAP=y
CONFIG_ZMS=y
CONFIG_SETTINGS=y
```

Where on the flash is a devicetree question. The ZMS backend uses the partition that the `chosen` node names `zephyr,settings-partition`, or, when there is none, the partition labelled `storage_partition`. `native_sim`'s devicetree has one: 16 KiB at offset `0xfc000` of its 2 MiB flash.

Handlers

A module that owns a subtree registers a **handler** for it: a set of callbacks, any of which may be `NULL`.

Callback	Called by	To
`h_set(name, len, read_cb, cb_arg)`	`settings_load()`, once for every stored setting in the subtree	Take the stored value into the module's variables.
`h_commit()`	`settings_load()`, after every `h_set`	Act on the whole set of values at once.
`h_export(export_func)`	`settings_save()`	Hand every value of the subtree over for saving.
`h_get(name, val, len)`	`settings_runtime_get()`	Return the value the module holds in RAM.

The handler is defined at compile time, like a shell command:

```
SETTINGS_STATIC_HANDLER_DEFINE(thermostat, "thermostat", NULL, thermostat_set, ↩
    ↪ NULL, NULL);
```

`h_set` gets the rest of the name, after the subtree, and **not the value itself**. The subsystem does not know how big the value should be, so the handler checks the stored length and reads the value into storage of its own:

```
static int thermostat_set(const char *name, size_t len, settings_read_cb read_cb, ↩
    ↪ void *cb_arg)
{
        const char *next;

        if (!settings_name_steq(name, "setpoint", &next) || (next != NULL)) {
                return -ENOENT;
        }
        if (len != sizeof(setpoint_c)) {
                return -EINVAL;
        }

        ssize_t read = read_cb(cb_arg, &setpoint_c, sizeof(setpoint_c));

        return (read < 0) ? (int)read : 0;
}
```

`settings_name_steq(name, "setpoint", &next)` is true when `name` begins with `setpoint`, and sets `next` to what follows the next `/`, or to `NULL` if nothing does. The length check is not pedantry: flash written by an older firmware, with a smaller type, is still there after an update.

Loading, saving and deleting

```
settings_subsys_init();                         /* Mount the backend. */
settings_load();                                /* Call h_set for everything stored,↩
    ↪  then h_commit. */
settings_save_one("thermostat/setpoint", &value, sizeof(value));
settings_delete("thermostat/setpoint");
```

Loading happens once, at boot, and on the application's initiative. Until `settings_load()` runs, every variable holds its initial value, which is also what it keeps if nothing was ever stored: on the very first boot, no handler is called at all. Saving writes one value at once; there is no transaction around several.

The flash on `native_sim`

`native_sim`'s flash is simulated in a file, **`flash.bin`, in the directory the program runs in**:

```
west build -t run          # Runs in build/, so the file is build/flash.bin.
./build/zephyr/zephyr.exe  # Runs where you are, and uses a flash.bin there.
```

The file is the whole 2 MiB of the simulated flash. A pristine build, `west build -p`, deletes `build/` and the flash with it: a factory-new device. The command-line options decide what happens to the flash for one run:

Option	Effect
`-flash=<path>`	Use another file.
`-flash_erase`	Erase the flash at startup.
`-flash_rm`	Delete the file when the program ends.
`-flash_in_ram`	Keep the flash in RAM only. Every run starts erased, and the file is left alone.

`strings build/flash.bin` shows the names of the stored settings, `boot/count` for the `lectures/L06/examples/settings` example, because the backend stores each name next to its value. With `CONFIG_SETTINGS_SHELL=y`, the shell shows the same from inside:

```
uart:~$ settings list
thermostat/setpoint
uart:~$ settings read thermostat/setpoint
00000000: 1b 00 00 00                                      |....            |
```

`1b 00 00 00` is 27, as a little-endian `int32_t`: the bytes exactly as the handler wrote them.

6.1.3 Networking on `native_sim`

Two ways onto a network

Zephyr has its own TCP/IP stack, and on a board that is what runs under the socket API. `native_sim` can run it too, over a TAP interface on the host (`CONFIG_ETH_NATIVE_TAP`), but that needs the TAP interface set up on the host, with root privileges.

The alternative is **offloaded sockets**:

```
CONFIG_NETWORKING=y
CONFIG_NET_IPV4=y
CONFIG_NET_SOCKETS=y
CONFIG_NET_DRIVERS=y
CONFIG_NET_SOCKETS_OFFLOAD=y
CONFIG_NET_NATIVE_OFFLOADED_SOCKETS=y
```

With them, every socket call is handed to the host's own socket API. A server listens on a real port of your machine, a client connects to real addresses, and nothing needs setting

up. The price is in the name: Zephyr's TCP/IP stack is not in the program at all, and Linux does the networking. That is what you want while developing the protocol on top, and it is exactly why the same program still has to be tested on a board.

The socket API

Zephyr's socket API is BSD sockets with a prefix:

BSD	Zephyr
`socket()`, `bind()`, `listen()`, `accept()`, `connect()`	`zsock_socket()`, `zsock_bind()`, `zsock_listen()`, `zsock_accept()`, `zsock_connect()`
`send()`, `recv()`, `sendto()`, `recvfrom()`, `close()`	`zsock_send()`, `zsock_recv()`, `zsock_sendto()`, `zsock_recvfrom()`, `zsock_close()`
`setsockopt(s, SOL_SOCKET, SO_RCVTIMEO, ...)`	`zsock_setsockopt(s, ZSOCK_SOL_SOCKET, ZSOCK_SO_RCVTIMEO, ...)`
`struct sockaddr_in`, `AF_INET`, `htons()`	`struct net_sockaddr_in`, `NET_AF_INET`, `net_htons()`

Zephyr 4.4 prefixed the types and constants too. Before it, `struct sockaddr_in` and `AF_INET` were Zephyr's own names, which collided with the C library's and POSIX's. Now the networking headers define `net_`, `NET_` and `ZSOCK_` names, and a compatibility header, enabled by `CONFIG_NET_NAMESPACE_COMPAT_MODE` (on by default), keeps the old names working. New code should use the prefixed ones, as the examples do; code that wants the plain POSIX names includes the POSIX headers and enables Zephyr's POSIX API.

Blocking

A blocking call blocks **the calling thread**, not the program: `zsock_recv()` waits inside the kernel, like `k_sem_take()`, and every other thread runs meanwhile. Underneath, the offloaded sockets driver asks the host which sockets are ready once every simulated millisecond (`CONFIG_NET_NATIVE_OFFLOADED_SOCKETS_EPOLL_WAIT_INTERVAL`).

That holds only while the driver knows the socket's timeouts. **In Zephyr 4.4.2, the driver allocates the socket that `zsock_accept()` returns without initializing its timeouts**, and a blocking `zsock_recv()` on it can go straight to the host's `recv()`, which blocks the whole process, simulated time and every thread with it. The `lectures/L06/examples/sockets` example froze at `Client connected` until it set both timeouts explicitly after accepting:

```
struct zsock_timeval forever = {0};

zsock_setsockopt(client, ZSOCK_SOL_SOCKET, ZSOCK_SO_RCVTIMEO, &forever, sizeof(
    ↪ forever));
zsock_setsockopt(client, ZSOCK_SOL_SOCKET, ZSOCK_SO_SNDTIMEO, &forever, sizeof(
    ↪ forever));
```

A timeout of zero means *wait forever*, as with BSD sockets. Sockets created with `zsock_socket()` are not affected: the driver sets their timeouts when it creates them. The simulator is software, with bugs of its own.

TCP and UDP

The `lectures/L06/examples/sockets` example is a TCP server: `listen()`, `accept()` a connection, then `recv()` and `send()` a stream of bytes, with no message boundaries, until the client closes it. UDP, which Exercise Set 5 uses, has no connections: a server `bind()`s a port and answers each datagram, whole, to the address `recvfrom()` says it came from. Nothing is retransmitted, so a client that waits for an answer should wait with a timeout (`ZSOCK_SO_RCVTIMEO`), not forever.

A program using offloaded sockets can be tested from the host with netcat:

```
nc 127.0.0.1 4242       # TCP, the sockets example.
nc -u 127.0.0.1 4300    # UDP, Exercise Set 5.
```

On `native_sim`, networking also prints `WARNING: Using a test - not safe - entropy source` at boot: the network stack wants random numbers, and `native_sim`'s entropy device is a seeded generator, deterministic by design.

6.1.4 Summary

- The shell is a command line inside the firmware, running in a thread of its own over a backend. On `native_sim`, `CONFIG_UART_NATIVE_PTY_0_ON_STDINOUT` puts it on the terminal.
- A command is a handler and a static entry. The mandatory word count includes the command's name, and the shell checks the count; the handler checks what the words say, with `strtol()`, not `atoi()`.
- Handlers print with `shell_print()` and `shell_error()`, and return `0` or a negative `errno`, which `retval` shows.
- Settings are named key-value pairs, owned by subtree handlers, stored by a backend such as ZMS in the flash partition the devicetree names.
- `settings_load()` calls each handler's `h_set` with the stored length and a function to read the value; nothing is loaded until the application asks.
- `native_sim`'s flash is the file `flash.bin` in the working directory, controlled by `-flash`, `-flash_erase`, `-flash_rm` and `-flash_in_ram`.
- Offloaded sockets put the host's network under Zephyr's socket API, without setup and without Zephyr's IP stack. Zephyr 4.4 prefixes the socket API's names with `zsock_`, `net_` and `NET_`.
- Blocking socket calls block one thread, as long as the socket's timeouts are set: set them explicitly on accepted sockets in Zephyr 4.4.2.

6.2 Debugging: Fatal Errors, Sanitizers, GDB and Coverage

This section covers what happens when a Zephyr program goes wrong, and the tools that find out why. Some of them are Zephyr's own and work the same on a board: fatal errors, the handler that decides what one costs, and assertions. The others are the host's, and are much of the reason to keep code runnable on `native_sim`: AddressSanitizer, UndefinedBehaviorSanitizer, GDB and gcov. By the end you should be able to explain every line the `lectures/L06/examples/debugging` example prints, in each of its configurations.

6.2.1 Fatal errors

What makes an error fatal

A **fatal error** is one the running code cannot continue from. The kernel names the reason with a number:

Reason	Value	Raised by
`K_ERR_CPU_EXCEPTION`	0	The CPU: a bus fault, an illegal instruction, a division by zero, on a board.
`K_ERR_SPURIOUS_IRQ`	1	An interrupt that has no handler.
`K_ERR_STACK_CHK_FAIL`	2	A stack overflow check.
`K_ERR_KERNEL_OOPS`	3	`k_oops()`: *this thread* cannot go on.
`K_ERR_KERNEL_PANIC`	4	`k_panic()`, and every failed assertion: *the system* cannot go on.

On `native_sim` there is no CPU exception to raise, since the code runs natively; a genuine segmentation fault kills the host process like any other Linux program's. Oopses, panics and assertions work exactly as on a board.

`k_oops()` and `k_panic()` are for code that has found something impossible, not for ordinary errors. A sensor that does not answer is an error to return; a reading below absolute zero, from a sensor the code has already validated, is a broken assumption, and an oops is how a thread gives up on it.

What the kernel does

Every fatal error goes through the same path. The kernel logs what happened, and then calls **`k_sys_fatal_error_handler()`** with the reason:

```
[00:00:00.000,000] <err> app: Impossible reading: -300 C
@ WEST_TOPDIR/lectures/L06/exercises/exercise3/src/main.c:29
[00:00:00.000,000] <err> os: >>> ZEPHYR FATAL ERROR 3: Kernel oops on CPU 0
[00:00:00.000,000] <err> os: Current thread: 0x40f6c0 (sampler)
```

- **`@ file:line`** is printed on architectures that have no CPU exception to raise for an oops, `native_sim` among them. On a board, the exception's registers say where instead.
- **`ZEPHYR FATAL ERROR 3: Kernel oops`** gives the reason's number and name. Twister looks for this exact string to tell a crashed test from a failed one.

- **`Current thread`** names the thread, if `CONFIG_THREAD_NAME=y`; otherwise only its address.

All of it is logged, so **without `CONFIG_LOG=y` a fatal error prints nothing** but what the handler prints itself.

The handler

The kernel's own `k_sys_fatal_error_handler()` is a weak function that halts the system, whatever the reason:

```
[00:00:00.000,000] <err> os: Halting system
Exiting due to fatal error
```

On a board, halting means an endless loop with interrupts off, until the watchdog or a person resets the device. On `native_sim`, it means that the process exits with **status 1**, after printing `Exiting due to fatal error` on stderr.

An application that wants a different policy defines the function itself, which replaces the weak one:

```
#include <zephyr/fatal.h>
#include <zephyr/logging/log_ctrl.h>

void k_sys_fatal_error_handler(unsigned int reason, const struct arch_esf *esf)
{
        ARG_UNUSED(esf);

        LOG_PANIC();
        LOG_ERR("Fatal error %u in thread '%s'", reason, k_thread_name_get(↩
    ↪ k_current_get()));

        if (reason == K_ERR_KERNEL_OOPS) {
                return;
        }
        k_fatal_halt(reason);
}
```

What the handler decides is what the error costs:

- **If it returns**, the kernel aborts only the thread that caused the error, and every other thread keeps running. A thread waiting in `k_thread_join()` for the aborted one returns, because an aborted thread counts as finished.
- **If it calls `k_fatal_halt()`**, the system halts, as with the default handler.

Returning from a panic is not allowed. With `CONFIG_ASSERT=y`, the kernel checks for it, and the check is itself an assertion, so it panics again, and calls the same handler, which

returns again. On `native_sim` that recursion ends when the host process runs out of stack, with a segmentation fault and exit status 139:

```
ASSERTION FAIL [reason != K_ERR_KERNEL_PANIC] @ WEST_TOPDIR/deps/zephyr/kernel/
    ↪ fatal.c:135
        Attempted to recover from a kernel panic condition
```

`LOG_PANIC()` switches logging to synchronous. On a board, logging is deferred by default (Chapter 1): the messages wait for a thread of their own, which may never run again after a fatal error. On `native_sim` logging is immediate already, so the call changes nothing you can see there; it belongs in the handler anyway, because the handler is the same code on a board.

A real product's handler usually does more: it saves the reason and the thread's name somewhere that survives a reset, such as a retained RAM area or a setting, and resets the device, so that the next boot can report what happened. The `lectures/L06/examples/debugging` example's handler is `lectures/L06/examples/debugging/src/fatal.c`.

6.2.2 Assertions

An **assertion** states something the code assumes, and checks it at run time:

```
__ASSERT((percent >= 0) && (percent <= 100), "duty cycle %d %% is out of range",
    ↪ percent);
```

`__ASSERT_NO_MSG(condition)` is the same without a message. A failed assertion prints the condition, where it is, and the message, and then panics:

```
ASSERTION FAIL [(percent >= 0) && (percent <= 100)] @ WEST_TOPDIR/lectures/L06/
    ↪ exercises/exercise3/src/main.c:40
        duty cycle 150 % is out of range
@ WEST_TOPDIR/deps/zephyr/lib/os/assert.c:43
[00:00:00.000,000] <err> os: >>> ZEPHYR FATAL ERROR 4: Kernel panic on CPU 0
```

Assertions are compiled out unless `CONFIG_ASSERT=y`, in Zephyr's code and in yours. Without it the condition is not even evaluated, and the program above carries on with a duty cycle of 150 %. With it, CMake warns that `__ASSERT() statements are globally ENABLED`, because every check costs flash and time. That makes assertions a development tool: tests and debug builds enable them, and most products ship without.

Two consequences follow:

- **An assertion must not have side effects.** `__ASSERT(k_sem_take(&sem, K_NO_WAIT) == 0, "...")` takes the semaphore only in builds with assertions.
- **An assertion is not error handling.** Input from a user, a sensor or the network can be wrong in a correct program, and has to be checked with an `if` in every build.

For conditions the compiler can decide, `BUILD_ASSERT(condition, "message")` checks at compile time and costs nothing at run time.

6.2.3 Sanitizers

The **sanitizers** are compiler instrumentation. The compiler adds a check to every memory access or arithmetic operation it can, and a runtime library reports every check that fails, with the source line, the variable, and a stack trace. They find bugs that on a board corrupt memory silently and crash, if at all, somewhere else, much later.

AddressSanitizer

```
west build -p -b native_sim/native/64 lectures/L06/examples/debugging \
    -- -DEXTRA_CONF_FILE=asan.conf
```

`CONFIG_ASAN=y` builds with `-fsanitize=address`. AddressSanitizer surrounds every stack array, heap block and global with poisoned *redzones*, and checks every read and write against them. It finds out-of-bounds reads and writes, use after free, and double frees. Exercise Set 4's battery monitor reads one element past an array on the stack:

```
==1354852==ERROR: AddressSanitizer: stack-buffer-overflow on address 0
    ↪ x78a35ddf4070 ...
READ of size 4 at 0x78a35ddf4070 thread T2
    #0 0x402a6d in average .../exercise4/src/main.c:18

Address 0x78a35ddf4070 is located in stack of thread T2 at offset 48 in frame
    #0 0x402aa0 in main .../exercise4/src/main.c:33

  This frame has 1 object(s):
    [32, 48) 'window' (line 34) <== Memory access at offset 48 overflows this
    ↪ variable
```

Read it from the top: what kind of access (`READ of size 4`), where in the code (line 18, in `average()`), and whose memory it overflowed (`window`, declared on line 34). `T2` is a host thread: every Zephyr thread on `native_sim` is one. The report ends with `SUMMARY: AddressSanitizer: stack-buffer-overflow ... in average`, and the process exits with **status 1** at the first error. (`CONFIG_ASAN_RECOVER`, on by default, compiles the checks so that they *could* continue, but the runtime only does so when `ASAN_OPTIONS` contains `halt_on_error=0`.)

AddressSanitizer exists only on `native_sim`: `CONFIG_ASAN` depends on `ARCH_POSIX`, because the checks call into the host's `libasan`. The Kconfig help also notes that leak detection at exit is off for 64-bit boards built with GCC, so `native_sim/native/64` reports no leaks when the program ends.

UndefinedBehaviorSanitizer

```
west build -p -b native_sim/native/64 lectures/L06/examples/debugging \
    -- -DEXTRA_CONF_FILE=ubsan.conf
```

`CONFIG_UBSAN=y` builds with `-fsanitize=undefined`, which checks for undefined behaviour in arithmetic and pointers: signed integer overflow, shifts by too much, division by zero, misaligned pointers, and out-of-range enum and bool values. Exercise Set 4's power calculation overflows an `int32_t`:

```
.../exercise4/src/main.c:26:32: runtime error: signed integer overflow: 2250000 * ↩
    ↪ 1000 cannot be represented in type 'int'
5000 mV, average 3725 mV, -2044967296 uW
```

Two things differ from AddressSanitizer:

- **The program continues**, with whatever the undefined operation produced, here a wrapped negative number. One run can report several problems.
- **The report goes to stderr**, not to the console's stdout, so a tool that only reads stdout never sees it. Twister's console harness is one such tool.

UndefinedBehaviorSanitizer is not tied to `native_sim` as tightly. Its reports need a runtime library, the host's or picolibc's; on other targets `CONFIG_UBSAN_TRAP` turns undefined behaviour into a CPU exception, without saying what or where.

Both can be enabled at once (`-DCONFIG_ASAN=y -DCONFIG_UBSAN=y`), and a program that is correct prints exactly the same with them as without. That is how Exercise Set 4 is tested.

What a program without them does

The same battery monitor, built without sanitizers, printed this on the machine these notes were written on:

```
3300 mV, average 249333220 mV, 1485000000 uW
5000 mV, average 249333645 mV, -2044967296 uW
```

The out-of-bounds read picked up whatever was next to the array, and the overflow wrapped. Neither crashed. Another build of the same code averaged `261933924 mV` instead, and on another compiler, or with another optimization level, the average could as easily have looked plausible. That is the case for the sanitizers: undefined behaviour does not announce itself.

6.2.4 GDB

A `native_sim` program is an ordinary Linux executable with debug information, so GDB debugs it directly, with no probe and no GDB server:

```
sudo apt install gdb
gdb --args ./build/zephyr/zephyr.exe -no-rt
```

Not verified here. GDB was not installed on the machine these notes were written on, so the commands in this section were not run while writing them. They are GDB's standard commands, and the Kconfig options they rely on were checked in Zephyr's source.

The commands that matter most:

Command	Effect
`break average`, `break main.c:18`	Stop at a function, or at a line.
`break k_sys_fatal_error_handler`	Stop at every fatal error, before the handler decides.
`run`, `continue`	Start the program, or carry on after a stop.
`backtrace`	The calls that led to where the program stopped.
`print window[i]`, `info locals`	Values of variables.
`info threads`, `thread 2`	The host threads, and switching between them.

Two Zephyr options make it easier:

- **`CONFIG_DEBUG_OPTIMIZATIONS=y`** compiles with `-Og`, which the `lectures/L06/examples/debugging` example sets, so that variables are not optimized away and lines run in the order they are written.
- **`CONFIG_ARCH_POSIX_TRAP_ON_FATAL=y`** raises `SIGTRAP` in the halt path, before the process exits, so a debugger stops at a fatal error without a breakpoint.

`native_sim` creates a host thread for every Zephyr thread when the Zephyr thread is created, next to the simulator's own. A thread that is stalled in a busy loop (Chapter 3) is easy to find this way: interrupt the program with `Ctrl+C` in GDB, and look at `info threads` and `backtrace`.

6.2.5 Coverage

Coverage measures which lines and branches of the code a run executed. On `native_sim` it is GCC's own gcov:

```
west build -p -b native_sim/native/64 lectures/L06/exercises/exercise4 \
    -- -DCONFIG_COVERAGE=y
./build/zephyr/zephyr.exe -no-rt -stop_at=1
```

With `CONFIG_COVERAGE=y` the program counts every line it runs, and **writes the counts when it exits**: `-stop_at` is an exit, and so is a fatal error's. They go into a `.gcda` file next to each object file, such as `build/CMakeFiles/app.dir/src/main.c.gcda`, which `gcov` reads:

```
cd build
gcov -o CMakeFiles/app.dir/src/main.c.gcda \
    ../lectures/L06/exercises/exercise4/src/main.c
```

```
Lines executed:100.00% of 14
Creating 'main.c.gcov'
```

(The number of lines depends on how your `exercise4` is written.)

`main.c.gcov` is the source with a count beside every line:

```
       24:   18:        for (size_t i = 0; i < n; i++) {
       18:   19:                sum += window[i];
```

The loop body ran 18 times over the six calls, `1 + 2 + 3 + 4 + 4 + 4`, and its condition was tested 24 times, once more per call, when it failed.

Twister collects coverage over a whole test run, and turns it into an HTML report:

```
sudo apt install lcov
west twister -T lectures/L06/examples/debugging -p native_sim/native/64 \
    --coverage --coverage-tool lcov
```

The report is `twister-out/coverage/index.html`. Twister's default tool is `gcovr` instead, which is installed with `pip install gcovr`.

On a board, coverage is possible too (`CONFIG_COVERAGE_GCOV`), but the counts have to be dumped over a serial line and converted on the host, and the instrumentation needs RAM that small devices do not have. On `native_sim` it is one option.

6.2.6 Summary

- A fatal error has a reason: an oops means a thread cannot go on, a panic that the system cannot. Assertions panic.
- The kernel logs every fatal error, and calls `k_sys_fatal_error_handler()`. The default one halts; on `native_sim` that is exit status 1.
- A handler of your own decides what an error costs. Returning aborts only the faulting thread; a panic must halt, with `k_fatal_halt()`.
- `__ASSERT()` is compiled out unless `CONFIG_ASSERT=y`. Assertions must have no side effects, and are not error handling.
- AddressSanitizer finds out-of-bounds access and use after free, stops at the first error, and exists only on `native_sim`.
- UndefinedBehaviorSanitizer finds undefined arithmetic, reports on stderr, and continues.

- A `native_sim` program is debugged with GDB like any Linux program, one host thread per Zephyr thread.
- `CONFIG_COVERAGE=y` writes gcov data when the program exits; Twister's `--coverage` turns a test run's into a report.

6.3 Review

What you should be able to do now

After this chapter, you should:

- Be able to add shell commands to an application, and use the shell to inspect a running system.
- Be able to store and load settings, and understand where they are stored on `native_sim`.
- Be able to write TCP and UDP servers with Zephyr's socket API, and reach them from the host.
- Understand how Zephyr handles a fatal error, and be able to decide what happens after one.
- Be able to find memory errors and undefined behaviour with the sanitizers.
- Be able to debug a `native_sim` program with GDB, and measure test coverage.

Questions to test yourself

You should be able to explain:

1. What does a shell command handler receive, and how does it report an error?
2. Why can `log enable dbg` not show debug messages from a module compiled at level `INF`?
3. Where does the settings subsystem store its data on `native_sim`, and what makes a run a first boot?
4. What does the offloaded sockets driver do, and why does it need no network setup or root?
5. What is the difference between `k_oops()` and `k_panic()`, and what does `k_sys_fatal_error_handler()` decide?
6. Why does AddressSanitizer work on `native_sim`, and why is it not available when the same code is built for a board?
7. Why does a UBSan report not stop the program, while an ASan report does?
8. What would change, and what would not, if the driver from Chapter 5 and these subsystems moved to a real board?

Looking ahead

This is the last chapter: you have built, configured, described, scheduled, driven, tested and debugged a Zephyr application, and every one of those programs runs on a board as it stands. What is left is the board itself. Moving the same code onto one, flashing it, and testing it there, is the business of two other courses in this track, *Bootloaders and the Boot Path* and *Hardware-in-the-Loop Testing*.

6.4 Exercises

These exercises reinforce the concepts from §6.1 and §6.2.

> **How to check your work.** Write each exercise set as its own application under `lectures/L06/exercises`, with the file names given there, and run `make test` from the repository root. A set's tests switch on as soon as its files exist, and a set you have not started is reported as `SKIP`. Each application grows through its set, and the tests check where it ends up. Shell commands are fed to your program on its standard input, and its output is checked for the lines each exercise shows, in order; the prompt, the echoed commands and anything else in between do not matter. The solutions are published in the companion repository, under `lectures/L06/appendix/solutions`, a few days after the lecture. Try each exercise yourself before you read them.

Remember `source env.sh` in every new terminal. Build and run an exercise from the repository root:

```
west build -p -b native_sim/native/64 lectures/L06/exercises/exercise1
west build -t run
```

EXERCISE SET 1

Shell commands of your own

Create an application named `exercise1` to work in for this exercise set.

Exercise 6.1 The setpoint

CODE

In this exercise you will give a thermostat a shell command that reads and changes its setpoint.

Tasks

- In `prj.conf`, enable the shell, and put the shell's UART on the terminal the program is started from.
- In `src/main.c`, keep the setpoint in a `static int setpoint_c`, initially 21.
- Create a root command `setpoint` with the help text `The thermostat's setpoint.`, no handler of its own, and two subcommands:

  - `get`, which takes no arguments, and prints `Setpoint: <t> C`.
  - `set <celsius>`, which takes exactly one argument, sets the setpoint, and prints `Setpoint set to <t> C`.
- Let the shell check the argument counts: declare them with `SHELL_CMD_ARG()`.
- `main()` prints `Thermostat ready` and returns.

Pipe a session into the program:

```
printf 'setpoint get\nsetpoint set 25\nsetpoint get\n' \
    | ./build/zephyr/zephyr.exe -stop_at=1
```

The output must contain, in this order:

```
Thermostat ready
Setpoint: 21 C
Setpoint set to 25 C
Setpoint: 25 C
```

Reflection

- The program's output also contains `uart:~$ setpoint get`. Where does that line come from?
- Run `setpoint` alone, and then `retval`. What does `retval` print, and why that value?
- `main()` returns at once. Why does the shell keep answering commands?

Exercise 6.2 Wrong input

CODE

Tasks

Make `setpoint set` reject what is not a valid setpoint:

- A word that is not a whole decimal number, such as `warm` or `25x`, prints the error `not a temperature: '<word>'` with `shell_error()` and returns `-EINVAL`.
- A number outside 5 to 30 prints the error `setpoint must be between 5 and 30 C, not <t>` and returns `-ERANGE`. The setpoint does not change.

For the session

```
setpoint get
setpoint set 25
setpoint get
setpoint set 31
retval
setpoint set warm
retval
```

```
setpoint set 25x
setpoint set
```

the output must contain, in this order:

```
Setpoint: 21 C
Setpoint set to 25 C
Setpoint: 25 C
setpoint must be between 5 and 30 C, not 31
-34
not a temperature: 'warm'
-22
not a temperature: '25x'
set: wrong parameter count
```

Hint: `strtol()` from `<stdlib.h>` sets its `end` argument to the first character it did not convert.

Reflection

- Why is `atoi()` not good enough here?
- Which part of the output does the shell print by itself, without your handler being called?
- `setpoint set 99999999999999999999` is a number too. What does your handler do with it, and why?

EXERCISE SET 2

A setpoint that survives a restart

Copy `exercise1` to a new application named `exercise2` to work in for this exercise set.

Exercise 6.3 Storing it

CODE

In this exercise you will store the setpoint with the settings subsystem, on the simulated flash.

Tasks

- In `prj.conf`, enable the settings subsystem with the ZMS backend, and what ZMS needs: flash and the flash map.
- Keep the setpoint in an `int32_t`, stored under the name `thermostat/setpoint`.
- Register a settings handler for the subtree `thermostat` with `SETTINGS_STATIC_HANDLER_DEFINE()`. Its `h_set` callback takes the stored `setpoint`, and rejects a stored value whose length is not that of an `int32_t` with `-EINVAL`.
- In `main()`, initialize the settings subsystem, load the settings, and print `Loaded setpoint: <t> C` instead of `Thermostat ready`.

- `setpoint set` stores the new value with `settings_save_one()` before it changes the setpoint in RAM. If storing fails, it prints `cannot store the setpoint: <error>` with `shell_error()` and returns the error.

Run the program twice, from the same directory:

```
cd build
printf 'setpoint set 25\n' | ./zephyr/zephyr.exe -stop_at=1
printf 'setpoint get\n' | ./zephyr/zephyr.exe -stop_at=1
```

The first run must print `Loaded setpoint: 21 C` and `Setpoint set to 25 C`, and the second `Loaded setpoint: 25 C` and `Setpoint: 25 C`.

Reflection

- Where is the setpoint stored? Find the file, and find the setting's name in it with `strings`.
- Run the second command again from the repository root instead of from `build/`. What does it load, and why?
- Run it with `-flash_in_ram`. What does it load, and has the file changed?
- In `h_set`, why read the value into a local variable and check it, rather than straight into the setpoint?

Exercise 6.4 Forgetting it

CODE

Tasks

Add a subcommand `setpoint reset`, which takes no arguments, deletes the stored setpoint with `settings_delete()`, sets the setpoint back to 21, and prints `Setpoint reset to 21 C`.

In five runs from the same directory, where each run is given the commands shown, the output must contain:

Run	Commands	Output
1	`setpoint set 25`	`Loaded setpoint: 21 C`, then `Setpoint set to 25 C`
2	`setpoint get`	`Loaded setpoint: 25 C`, then `Setpoint: 25 C`
3	none, run with `-flash_in_ram`	`Loaded setpoint: 21 C`
4	`setpoint reset`	`Loaded setpoint: 25 C`, then `Setpoint reset to 21 C`
5	none	`Loaded setpoint: 21 C`

Reflection

- Enable `CONFIG_SETTINGS_SHELL` on the command line, and run `settings list` and `settings read thermostat/setpoint` after setting the setpoint to 27. What do the four bytes it prints mean?
- How would you make every run of the tests start from a factory-new device, without deleting files by hand?

EXERCISE SET 3

Fatal errors

Create an application named `exercise3` to work in for this exercise set.

Exercise 6.5 An impossible reading

CODE

In this exercise you will make a thread give up with `k_oops()`, and see what that costs without a handler of your own.

Tasks

- In `prj.conf`, enable logging and thread names.
- In `src/main.c`, register the log module `app`, and define the readings:

```
static const int readings[] = {21, 22, -300, 23};
```

- Write a thread function `sampler()` that goes through the readings. A reading below -273 logs the error `Impossible reading: <t> C` and calls `k_oops()`; any other logs `Sample <t> C`.
- In `main()`, create the thread with `k_thread_create()`, with a 1024-byte stack from `K_THREAD_STACK_DEFINE()` and priority 5, name it `sampler`, and wait for it with `k_thread_join()`. Then log `main() carries on`, and return.

Build, run, and check the exit status:

```
./build/zephyr/zephyr.exe -stop_at=1; echo "exit status $?"
```

Reflection

- Which lines does the kernel print, and what does each one tell you?
- Does `main() carries on` appear? What is the exit status, and why?
- Build once more with `CONFIG_LOG=n` on the command line. What does a fatal error print now?

Exercise 6.6 A handler of your own

CODE

Tasks

- Create `src/fatal.c`, with a log module named `fatal`, and define `k_sys_fatal_error_handler()` in it. It shall:
  - Switch logging to synchronous with `LOG_PANIC()`.
  - Log the error `Fatal error <reason> in thread '<name>'`.
  - For an oops, log `Aborting the thread` and return.
  - For anything else, log `Halting the system` and call `k_fatal_halt()`.
- Add `src/fatal.c` to the application in `CMakeLists.txt`.

The output must now contain, in this order, and the exit status must be 0:

```
<inf> app: Sample 21 C
<inf> app: Sample 22 C
<err> app: Impossible reading: -300 C
>>> ZEPHYR FATAL ERROR 3: Kernel oops
<err> fatal: Fatal error 3 in thread 'sampler'
<err> fatal: Aborting the thread
<inf> app: main() carries on
```

Reflection

- Why does `Sample 23 C` never appear?
- Why does `k_thread_join()` return, when the thread never returned from its function?
- Remove the check for the oops, so that the handler always returns. Nothing changes in this program's output. Why not, and why is it still wrong?

Exercise 6.7 An assertion

CODE

Tasks

- In `src/main.c`, add a function:

```
static void set_heater_duty(int percent)
{
  __ASSERT((percent >= 0) && (percent <= 100), "duty cycle %d %% is out of ↩
    ↪ range", percent);
  LOG_INF("Heater duty cycle %d %%", percent);
}
```

- At the end of `main()`, after `main() carries on`, call `set_heater_duty(150)`.

Built as usual, the program must also print `<inf> app: Heater duty cycle 150 %` at the end, and exit with status 0. Built with `-DCONFIG_ASSERT=y`, the output must contain, in this order, and the exit status must be 1:

```
<inf> app: main() carries on
duty cycle 150 % is out of range
>>> ZEPHYR FATAL ERROR 4: Kernel panic
<err> fatal: Fatal error 4 in thread 'main'
<err> fatal: Halting the system
```

Reflection

- Why does the same program pass a duty cycle of 150 % in one build and halt in the other?
- With the handler from the last reflection of Exercise 6.6, the one that always returns, build with `-DCONFIG_ASSERT=y` and run. What happens, and what is the exit status?
- Why would `__ASSERT(settings_save_one(...) == 0, "...")` be a bug?

EXERCISE SET 4

Sanitizers

Create an application named `exercise4` to work in for this exercise set.

Exercise 6.8 A battery monitor

CODE

In this exercise you will find two bugs with the sanitizers, and fix them.

Tasks

Create `CMakeLists.txt`, a `prj.conf` with only a comment, and a `src/main.c` holding exactly this:

```
/*
 * SPDX-License-Identifier: MIT
 *
 * Exercise Set 4: a battery monitor, with two bugs.
 */

#include <zephyr/kernel.h>

/* How many of the most recent readings the average is taken over. */
#define WINDOW 4

/* The average of the first n readings in a window. */
static __noinline int32_t average(const int32_t *window, size_t n)
{
        int32_t sum = 0;

        for (size_t i = 0; i <= n; i++) {
                sum += window[i];
        }
        return sum / (int32_t)n;
}

/* The power drawn at a voltage and a current, in microwatts. */
static __noinline int32_t power_uw(int32_t millivolts, int32_t milliamps)
{
        return millivolts * milliamps * 1000;
}

static const int32_t readings_mv[] = {3300, 3310, 3290, 3300, 5000, 5020};
static const int32_t load_ma = 450;

int main(void)
{
        int32_t window[WINDOW] = {0};

        for (size_t i = 0; i < ARRAY_SIZE(readings_mv); i++) {
                window[i % WINDOW] = readings_mv[i];

                printk("%d mV, average %d mV, %d uW\n", readings_mv[i],
                       average(window, MIN(i + 1, WINDOW)), power_uw(readings_mv[i
    ↪ ], load_ma));
        }
        return 0;
}
```

Build and run it as it is.

Reflection

- Which lines of the output are wrong? Would you have noticed if you did not know what to expect?
- The first three lines are right, although the bug in `average()` is there from the first call. Why do they come out right?

Exercise 6.9 AddressSanitizer

CODE

Tasks

Build with `-DCONFIG_ASAN=y`, run, and fix the bug that AddressSanitizer reports, changing as little as possible.

Reflection

- Which line, which variable, and which kind of access does the report name?
- What is the exit status, and why do the lines after the report never appear?

Exercise 6.10 UndefinedBehaviorSanitizer

CODE

Tasks

Build with `-DCONFIG_UBSAN=y`, run, and fix the bug that UndefinedBehaviorSanitizer reports, so that `power_uw()` returns an `int64_t` and computes the power without overflow for any voltage and current that fit in an `int32_t` each.

Built with `-DCONFIG_ASAN=y -DCONFIG_UBSAN=y`, and built without either, the program must print exactly:

```
3300 mV, average 3300 mV, 1485000000 uW
3310 mV, average 3305 mV, 1489500000 uW
3290 mV, average 3300 mV, 1480500000 uW
3300 mV, average 3300 mV, 1485000000 uW
5000 mV, average 3725 mV, 2250000000 uW
5020 mV, average 4152 mV, 2259000000 uW
```

Reflection

- The program did not stop at the report. How is that different from AddressSanitizer?
- Would `return (int64_t)(millivolts * milliamps * 1000);` fix the bug? Try it with UBSan.
- Build the fixed program with `-DCONFIG_COVERAGE=y`, run it, and run `gcov` on `main.c` as in §6.2. How many times did the body of the loop in `average()` run, and why that number?

EXERCISE SET 5

Sockets

Create an application named `exercise5` to work in for this exercise set.

Exercise 6.11 A UDP server

CODE

In this exercise you will write a UDP server that answers every datagram in capitals, and a client for it in the same program, talking through the host's network with offloaded sockets.

Tasks

- In `prj.conf`, enable networking with IPv4 only, sockets, and `native_sim`'s offloaded sockets, as in the `lectures/L06/examples/sockets` example. No logging is needed.
- Define `PORT` as 4300.
- Write a thread `server`, with `K_THREAD_DEFINE()`, a 2048-byte stack and priority 7, which:
  - Creates a UDP socket, and binds it to `PORT` on every address (`NET_INADDR_ANY`).
  - In an endless loop, receives a datagram of up to 64 bytes with `zsock_recvfrom()`, turns it into capitals with `toupper()`, and sends it back with `zsock_sendto()` to the address it came from.
- Use the prefixed names throughout: `struct net_sockaddr_in`, `NET_AF_INET`, `net_htons()`, and so on.

Exercise 6.12 The client

CODE

Tasks

In `main()`:

- Create a UDP socket, and set its receive timeout to one second with `ZSOCK_SO_RCVTIMEO`.
- Sleep 100 ms, so that the server, which has a lower priority, has bound its port.
- For each of the words `hello`, `zephyr` and `native_sim`, send the word to `127.0.0.1` at `PORT`, receive the reply, and print `Sent '<word>', received '<reply>'`. If no reply arrives, print `No reply to '<word>': <errno>` instead.
- Close the socket and print `Done`.

The output must contain, in this order:

```
Sent 'hello', received 'HELLO'
Sent 'zephyr', received 'ZEPHYR'
Sent 'native_sim', received 'NATIVE_SIM'
Done
```

Reflection

- Run the program without `-no-rt` and with `-stop_at=10`, and send it a datagram from another terminal with `printf 'from the host' | nc -u -w1 127.0.0.1 4300`. What comes back, and what does that say about where the port is?
- Why does the client set a receive timeout, when the example's TCP client does not?
- The output also begins with `WARNING: Using a test - not safe - entropy source`. Where does that come from?

APPENDIX A

The Self-Assessment Papers

This appendix and the four after it hold two complete three-hour papers for *Zephyr RTOS*, with worked solutions. Appendices B and C are the papers, questions only; Appendices D and E are their model answers, with marks.

What these are for

The book has **no exam and nothing is marked**. What you get instead is the exercise sets after every chapter, each with a published solution, and the questions in every chapter's review.

These papers do not replace that. **They are here for you to test your own skills and knowledge, nothing more.** They gate nothing, they are not a qualification, and no part of the book requires them. Nothing in the companion repository depends on them, and neither `make examples` nor `make test` knows they exist.

What they check is something the exercises cannot. Every exercise is done at a keyboard, with the chapter open beside you and a build ready to say no. These papers ask what you can reconstruct with none of that in front of you — which of the rules you have absorbed, and which you have merely been looking up.

Take one after the last chapter. Both papers draw on all six chapters, so sitting one partway through examines material you have not read yet, and the result says more about how far you have read than about what you have understood.

The two papers

Both cover the whole book, Chapters 1 to 6, at the same weighting. They share no question. Either can be used alone; use both as a main sitting and a resit, or in alternate years.

Question	Topic	Chapters	Marks
1	The workspace and the build	1	12
2	Kconfig and logging	1	12
3	Devicetree, bindings and overlays	2	13
4	Threads, scheduling and time	3	13
5	Interrupt context, work queues and simulated time	3	12
6	Synchronization and communication	4	13
7	Writing a driver	5	13
8	Subsystems and debugging	6	12
			100

Both papers mix prose with artifacts, because the subject does. Roughly half the marks are for explaining what a build decided and why; the other half are for reading or writing the things a Zephyr application is made of: a `prj.conf`, a devicetree overlay, a binding, a driver, a thread.

The two papers differ in which half they lean on:

- **Paper A leans towards what the build hands you.** Its questions put an artifact in front of you — a build that aborted, a devicetree that produced the wrong macro, a trace of threads in the wrong order, a driver whose device is never ready — and ask what happened and what to change. Naming the cause is half the marks; saying what it does to the running program is the other half.
- **Paper B leans towards what you write.** It asks for a Kconfig menu, a binding and the overlay that instantiates it, a sampling design split between an interrupt and a work queue, the skeleton of a devicetree-instantiated driver, and a ztest suite for it.

Neither paper asks anything that a careful reader could not answer from the chapters. Where a question turns on a rule that is easy to state and easy to get backwards — which file a Kconfig value comes from last, why a log message that was filtered out is not in the binary, which context a timer's expiry function runs in — the solution says so at length, because that is where the marks and the understanding both are.

Conventions the papers assume

Both papers state these in their own rubric, so a candidate never has to have read this appendix.

- **Zephyr 4.4 and the board `native_sim/native/64`**, built with the host's GCC, as in the book.
- **Code is marked on semantics, not syntax.** A missing `#include`, a missing semicolon or a brace in the wrong column costs nothing. A missing `device_is_ready()` check costs everything.

- **The book's house style is expected** where a question asks for it: Zephyr's own, meaning `snake_case`, the kernel's `k_` and `K_` spellings, application options prefixed `APP_`, `IS_ENABLED()` rather than `#ifdef` where both would work, and log modules rather than `printk()` in anything that is not a one-off.
- **The simulator's behaviour may be assumed** without deriving it: 100 ticks per second, a sleep that ends on the tick after the one it asked for, simulated time that only advances when every thread is waiting, immediate logging, and no thread analyzer, stack protection or userspace.
- **"State the consequence" means the consequence.** Naming a cause earns half the marks; saying what it does to the running program earns the other half. "This is wrong" scores nothing.
- **Where a question asks for a stated number of defects**, listing more is not penalised, but only that many are marked, so the strongest answers go first.

Marking

Every solution is written to be marked by somebody who has read the chapters and does not otherwise write Zephyr applications for a living, so each carries the reasoning rather than the answer alone. Marks are shown per part.

Three conventions worth agreeing before a paper is marked:

- **Method carries the marks.** A correct diagnosis with a clumsy fix is worth more than a correct fix with no diagnosis. Several questions feed the next part, so an error should be followed through rather than penalised twice.
- **The named traps are worth full marks on their own.** A number of parts exist entirely to see whether a candidate walks into a specific mistake: a `CONFIG_` option misspelled in C rather than in `prj.conf`, a devicetree node that is `disabled` rather than absent, a `k_sleep()` in a timer's expiry function, a semaphore used as a lock, a driver that reads its configuration from `dev->data`, a busy loop on `native_sim`. Where a solution flags one of these, a candidate who walks into it loses those marks and no others.
- **The discussion parts are not decoration.** "State why" is where the book's actual content is, and a paper marked only on whether the code would build would pass a candidate who has understood none of it.

APPENDIX B

Self-Assessment Paper A

Time: 3 hours. **Closed book.** No compiler and no reference material. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all six chapters, so **it is meant to be taken once you have finished the book**. The model answers are in Appendix D.

Rubric

Zephyr 4.4 and the board `native_sim/native/64` are assumed throughout, built with the host's GCC, as in the book. Headers may be assumed included unless a question asks you to name them.

Code is marked on semantics, not on syntax. A missing `#include`, a missing semicolon or a brace in the wrong column costs nothing. A missing `device_is_ready()` check costs everything.

Where a question says "in the book's style", it means Zephyr's own conventions as the chapters use them:

- `snake_case`, and the kernel's `k_` and `K_` spellings.
- Application options prefixed `APP_`.
- `IS_ENABLED()` rather than `#ifdef` where both would work.
- Log modules rather than `printk()` in anything that is not a one-off.

The simulator's behaviour may be assumed without deriving it: 100 ticks per second, a sleep that ends on the tick after the one it asked for, simulated time that advances only when every thread is waiting, immediate logging, and no thread analyzer, stack protection or userspace.

Where a question says "state the consequence", naming the cause earns half the marks and saying what it does to the running program earns the other half. "This is wrong" scores nothing.

Where a question asks you to find defects, the number of defects is stated. Listing more than the stated number is not penalised, but only the stated number is marked, so put your strongest answers first.

Marks

Question	1	2	3	4	5	6	7	8
Marks	12	12	13	13	12	13	13	12

Question 1 The workspace and the build

12 MARKS

(a) A colleague clones the book's repository, runs `make setup`, waits for it to finish, and then builds:

```
-- Application: /home/dev/zephyr/lectures/L01/examples/hello
-- ZEPHYR_TOOLCHAIN_VARIANT not set, trying to locate Zephyr SDK
CMake Error at .../cmake/modules/FindZephyr-sdk.cmake:160 (find_package):
  Could not find a package configuration file provided by "Zephyr-sdk"
```

Explain what the build is looking for and why it is not there. State the one command that fixes it, what that command sets, and why this book needs no Zephyr SDK at all. *(3 marks)*

(b) With that fixed, a build of a new application of their own fails during configuration:

```
CMake Error at .../cmake/modules/extensions.cmake:3016 (message):
  No prj.conf file(s) was found in the /home/dev/blink folder(s), please read
  the Zephyr documentation on application development.
```

Name the three files every Zephyr application must have and state what each one contributes. Then state which line of `CMakeLists.txt` does most of the work, and why `project()` must come after it. *(3 marks)*

(c) The book's `west.yml` holds:

```
manifest:
  remotes:
    - name: zephyrproject-rtos
      url-base: https://github.com/zephyrproject-rtos

  projects:
    - name: zephyr
      remote: zephyrproject-rtos
      revision: v4.4.2
      path: deps/zephyr
      clone-depth: 1
      west-commands: scripts/west-commands.yml
```

```
  self:
    path: .
```

State what `west update` does with this file, and where Zephyr ends up. State what would change if `west-commands` were removed. Then state the two steps that move the book's workspace to a newer Zephyr release, and why the manifest does not import Zephyr's own. (*4 marks*)

(d) A colleague builds once with an option on the command line:

```
west build -p -b native_sim/native/64 app -- -DCONFIG_LOG_DEFAULT_LEVEL=2
```

Later they build the same application in the same directory without that option, and the level is still 2. Explain why, and give the command that proves what the configuration really is. (*2 marks*)

Question 2 Kconfig and logging

12 MARKS

(a) An application adds `CONFIG_SHELL_LOG_BACKEND=y` to its `prj.conf`. The build stops:

```
warning: SHELL_LOG_BACKEND (defined at subsys/shell/Kconfig:278) was assigned the ↩
    ↪ value 'y' but got
the value 'n'. Check these unsatisfied dependencies: SHELL (=n). ...

error: Aborting due to Kconfig warnings
```

Explain what Kconfig did with the assignment and why Zephyr stops the build rather than carrying on. Give the two ways of making the build succeed, and state where you would read the value the build actually ended up with. (*4 marks*)

(b) An application has a Kconfig option `APP_FAHRENHEIT`. Its `main.c` contains:

```
if (IS_ENABLED(CONFIG_APP_FARENHEIT)) {
        printk("%d F\n", celsius_to_fahrenheit(reading));
}
```

The build succeeds with no warning, and the program prints Celsius whatever `prj.conf` says. Explain what the compiler did with that line, state the consequence, and state why the same misspelling in `prj.conf` would not have survived. Name one thing that would have caught it. (*3 marks*)

(c) A module is registered with `LOG_MODULE_REGISTER(monitor);` and the application logs four messages, at `LOG_ERR`, `LOG_WRN`, `LOG_INF` and `LOG_DBG`. The application is built with:

```
west build -p -b native_sim/native/64 app -- -DCONFIG_LOG_DEFAULT_LEVEL=2
```

State which of the four messages are printed. State whether the others are in the binary, and describe one way to demonstrate your answer without reading the source. *(3 marks)*

(d) `native_sim` builds with `CONFIG_LOG_MODE_IMMEDIATE=y`, while the same application built for a microcontroller gets deferred logging. State one consequence of each mode for the running program. *(2 marks)*

Question 3 Devicetree, bindings and overlays

13 MARKS

(a) Two builds of the same application fail in two different ways. The first:

```
error: '__device_dts_ord_32' undeclared here (not in a function); did you mean '
    ↪ __device_dts_ord_38'?
```

The second:

```
undefined reference to `__device_dts_ord_32'
```

State what `32` is and where you would look it up. Explain what each of the two messages means about the node, and give one change that produces the first and one that produces the second. *(5 marks)*

(b) A binding declares one required property and one optional one:

```
compatible: "qacademy,alarm"
include: base.yaml
properties:
  threshold-c:
    type: int
    required: true
  label-text:
    type: string
```

Two overlays are written against it. The first omits `threshold-c`. The second adds `hysteresis-c = <2>;`, which the binding does not declare, and the C code then reads it with `DT_PROP(DT_NODELABEL(alarm0), hysteresis_c)`.

State what the build does in each case, and at which stage. Then state why a property that the binding does not declare is a different kind of mistake from a misspelled `CONFIG_` macro in C. *(4 marks)*

(c) An overlay describes a button as `gpios = <&gpio0 1 (GPIO_PULL_UP | GPIO_ACTIVE_LOW)>`. State what `GPIO_ACTIVE_LOW` changes about the C code that reads the pin, and state what level `gpio_emul_input_set()` must be given to simulate a press. *(2 marks)*

(d) A node can be reached in C through its node name, its label and an alias. State which of the three a portable sample should use and why, and state what `DT_NODELABEL()` is for. *(2 marks)*

Question 4 Threads, scheduling and time

13 MARKS

(a) The trace below is printed by an application whose threads are named with their priorities in brackets. Every thread waits on a semaphore until it is woken.

```
-- Act 1
main   ( 10)  waking coop_a
coop_a ( -1)  running
coop_a ( -1)  gave coop_b's semaphore: coop_b has a higher priority, but I keep ↩
    ↪ the CPU
coop_b ( -2)  running, because coop_a yielded
coop_a ( -1)  back again, after coop_b returned
-- Act 2
main   ( 10)  waking low
low    (  5)  running
low    (  5)  giving high's semaphore
high   (  2)  running, in the middle of low's k_sem_give()
low    (  5)  back again: high preempted me inside k_sem_give()
```

Explain why `coop_b` does not run at the moment its semaphore is given, while `high` does. State what must have happened between the third and fourth lines of Act 1. State which priorities are cooperative and which are preemptible, and what that means for a critical section. *(6 marks)*

(b) On `native_sim`, a thread that calls `k_msleep(500)` at time 0 wakes at 510 ms, but a one-shot `k_timer` started at time 0 with `K_MSEC(500)` expires at exactly 500 ms. Explain both numbers. State what a periodic timer does about the difference. *(4 marks)*

(c) An application's `main()` prints one line and returns. The program does not exit. Explain what returning from `main()` ends and what it does not, state how you would make the program stop by itself, and state what `-no-rt` changes about the run and what it does not. *(3 marks)*

Question 5 Interrupt context, work queues and simulated time 12 MARKS

(a) The following is submitted for review.

```
static void sample_expiry(struct k_timer *timer)
{
        struct sample sample;

        LOG_INF("sampling");
        k_mutex_lock(&bus_mutex, K_FOREVER);
        read_sensor(&sample);
        k_mutex_unlock(&bus_mutex);
        k_msleep(5);
        store(&sample);
}

K_TIMER_DEFINE(sample_timer, sample_expiry, NULL);
```

Identify **three** defects, state the consequence of each, and describe the standard Zephyr shape this function should have instead. *(5 marks)*

(b) The system work queue runs at a cooperative priority. State what happens to other work items when one of them takes 200 ms, state what happens if a work item blocks forever, and give the one condition under which you would start a work queue of your own. *(3 marks)*

(c) This loop works on a microcontroller and hangs on `native_sim`:

```
static volatile bool ready;

static void isr_handler(const struct device *dev, ...)
{
        ready = true;
}

int main(void)
{
        start_hardware();
        while (!ready) {
        }
        LOG_INF("ready");
        return 0;
}
```

Explain what `native_sim` does here and why, state what the same code costs on real hardware, and give two different rewrites that are correct on both. *(4 marks)*

Question 6 Synchronization and communication 13 MARKS

(a) One application is run twice. Three threads share one CPU: `low` (priority 7) takes a lock and computes for 100 ms, `high` (priority 5) wants the same lock 21 ms later, and `medium` (priority 6) wakes at 31 ms and computes for 500 ms without needing the lock.

With a semaphore as the lock:

```
[   0 ms] low    takes the semaphore
[  21 ms] high   wants the semaphore
[  31 ms] medium starts computing, needs no lock
[ 531 ms] medium is done
[ 600 ms] low    gives the semaphore back
high waited 579 ms for the semaphore
```

With a `k_mutex` instead:

```
[   0 ms] low    takes the mutex
[  21 ms] high   wants the mutex
[ 100 ms] low    (priority 5) gives the mutex back
high waited 79 ms for the mutex
```

Name the phenomenon the first trace shows. Explain both waiting times, arithmetic included. State what the kernel did to `low` in the second run and when it undid it. State one thing the mutex does **not** fix. *(6 marks)*

(b) A producer submits samples to a `k_msgq` with `k_msgq_put(&q, &sample, K_NO_WAIT)` and the call starts returning `-ENOMSG`. State what that means and what happens to the sample. State what `K_FOREVER` would do instead, and which of the two an interrupt handler may use. *(3 marks)*

(c) Three observers are attached to one zbus channel: a listener, a subscriber and a message subscriber. Over a run of several publications, the subscriber prints one reading twice and never prints another, while the message subscriber prints every reading exactly once. Explain the subscriber's behaviour. State in which thread and with what held the listener runs, and what that forbids it from doing. *(4 marks)*

Question 7 Writing a driver 13 MARKS

(a) An application starts with:

```
const struct device *const sensor = DEVICE_DT_GET(DT_NODELABEL(thermo0));

if (!device_is_ready(sensor)) {
```

```
        LOG_ERR("thermo0 is not ready");
        return 0;
}
```

The message is printed on every boot, although the build succeeded and the node is enabled. State the two things `device_is_ready()` reports on, name the most likely thing the driver's init function did, and state what would happen if the application used the device anyway. *(4 marks)*

(b) A driver for an I2C part contains the following. It builds, and one instance works until a second is added to the devicetree.

```
#define DT_DRV_COMPAT qacademy_thermo

struct thermo_data {
        struct i2c_dt_spec i2c;
        int16_t last_sample;
};

static int thermo_init(const struct device *dev)
{
        struct thermo_data *data = dev->data;
        uint8_t id;

        data->i2c.bus = DEVICE_DT_GET(DT_NODELABEL(i2c0));
        data->i2c.addr = 0x48;

        (void)i2c_reg_read_byte_dt(&data->i2c, THERMO_REG_WHO_AM_I, &id);
        return 0;
}
```

Identify **three** defects, state the consequence of each, and give the corrected shape of the affected lines. *(5 marks)*

(c) The driver's Kconfig option is written `depends on DT_HAS_QACADEMY_THERMO_ENABLED`. State where that symbol comes from and what the dependency achieves. State what happens to the driver, and to an application that calls `DEVICE_DT_GET()` on the node, if the node is given `status = "disabled"`. *(2 marks)*

(d) On this build, I2C controllers initialize at priority 50 and sensors at 90, both at the `POST_KERNEL` level. State what the numbers order, and what would go wrong if the sensor's priority were 40. *(2 marks)*

Question 8 Subsystems and debugging 12 MARKS

(**a**) A shell session on a running program:

```
uart:~$ greet Ada 9
times must be between 1 and 5, not '9'
uart:~$ retval
-22
uart:~$ foo
foo: command not found
uart:~$ retval
-8
uart:~$ led
led - control the LED
Subcommands:
  on  off  status
uart:~$ retval
1
```

State where each of the three return values comes from, and what each one means. State what the shell does with a handler's return value, and why a command handler should return a negative errno rather than printing and returning 0. (*3 marks*)

(**b**) A boot counter is stored with the settings subsystem. It counts 1, 2, 3 over three runs started with `west build -t run`, then goes back to 1 after the developer rebuilds with `-p`. Explain where the counter lives on `native_sim` and why a pristine build resets it. Name the command-line option that makes every run a first boot without deleting anything. (*3 marks*)

(**c**) Two runs of the same application end differently:

```
[00:00:00.000,000] <err> os: >>> ZEPHYR FATAL ERROR 3: Kernel oops on CPU 0
[00:00:00.000,000] <err> app: main() is still running, and returns
```

```
ASSERTION FAIL [percent <= 100] @ .../src/main.c:41
[00:00:00.000,000] <err> os: >>> ZEPHYR FATAL ERROR 4: Kernel panic on CPU 0
Exiting due to fatal error
```

Explain the difference between the two reasons, state what the application's `k_sys_fatal_error_handler()` must do in each case, and state what the process exit status is in the second. (*3 marks*)

(**d**) A build with `CONFIG_ASAN=y` prints:

```
==1354852==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x78a35ddf4070
WRITE of size 1 at 0x78a35ddf4070 thread T2
    #0 0x402bbe in fill .../src/main.c:28
```

State what the report tells you that a microcontroller would not have told you at all. State why the program stops here rather than continuing, and state why AddressSanitizer is available on this board and not on a Cortex-M build of the same application. *(3 marks)*

APPENDIX C

Self-Assessment Paper B

Time: 3 hours. **Closed book.** No compiler, no workspace and no reference material. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all six chapters, so **it is meant to be taken once you have finished the book**. The model answers are in Appendix E.

Rubric

Zephyr 4.4 and the board `native_sim/native/64` are assumed throughout, built with the host's GCC, as in the book. Headers may be assumed included unless a question asks you to name them.

Code is marked on semantics, not on syntax. A missing `#include`, a missing semicolon or a brace in the wrong column costs nothing. A missing `device_is_ready()` check costs everything.

Where a question says "in the book's style", it means the conventions used in the chapters:

- Zephyr's own C style: `snake_case`, the kernel's `k_` and `K_` spellings.
- Application options prefixed `APP_`, and read with `IS_ENABLED()` rather than `#ifdef` wherever both would work.
- Log modules rather than `printk()` in anything that is not a one-off.
- Hardware taken from the devicetree, never a pin number written into the C.

The simulator's behaviour may be assumed without deriving it: 100 ticks per second, a sleep that ends on the tick after the one it asked for, simulated time that advances only when every thread is waiting, immediate logging, and no thread analyzer, stack protection or userspace.

Where a question says "state the consequence", naming a cause earns half the marks and saying what it does to the running program earns the other half. "This is wrong" scores nothing.

Where a question asks you to write code, documentation comments are not required and are not marked. Write the code.

Marks

Question	1	2	3	4	5	6	7	8
Marks	12	12	13	13	12	13	13	12

Question 1 The workspace and the build

12 MARKS

(a) Write the `west.yml` of a workspace that pins Zephyr 4.4.2, checks it out into `deps/zephyr`, fetches only the pinned commit rather than the project's history, makes Zephyr's own west commands available, and treats the repository holding the manifest as the workspace itself. Then state what `west update` does, and name one thing it does **not** do. *(4 marks)*

(b) Write the three files of the smallest application that logs `ready` at level INF: `CMakeLists.txt`, `prj.conf` and `src/main.c`. State what `find_package(Zephyr)` does, and why `project()` must come after it. *(4 marks)*

(c) Write the command that builds that application for `native_sim/native/64` from the repository root, discarding any earlier build, and the command that then runs it. Write the command that builds the same application with an extra configuration file `debug.conf` merged on top of `prj.conf`.

Then name three files under `build/` that record what the build decided, and state what each holds. *(4 marks)*

Question 2 Configuring an application

12 MARKS

(a) An application samples a temperature and can warn when it gets too hot. Write its `Kconfig` file, declaring:

- `APP_SAMPLE_PERIOD_MS`, an integer number of milliseconds between samples, between 10 and 10000, 1000 unless set;
- `APP_ALARM`, off unless set, which turns the warning on;
- `APP_ALARM_LIMIT_C`, an integer with the default 30, which exists only when the alarm does.

Write the `prj.conf` that samples every 250 ms with the alarm on. Then state what the build does if a `prj.conf` sets `CONFIG_APP_ALARM_LIMIT_C=35` while leaving the alarm off. *(5 marks)*

(b) Write the C that logs each reading through a log module named `sampler` and, when the alarm is enabled, warns about a reading at or above the limit. The warning lives in `src/alarm.c`, which is compiled only when the alarm is enabled; write the line in `CMakeLists.txt` that arranges that, and write the call site in `src/main.c`.

State why the call site uses `IS_ENABLED()` rather than `#ifdef`, and what the compiler does with the branch that cannot run. (*4 marks*)

(c) Name, in the order the build applies them, four places a Kconfig option's value can come from, and state which one wins.

Then: a build directory is created with `-DCONFIG_APP_SAMPLE_PERIOD_MS=50` on the command line. Later the same directory is built again, without `-p` and without that option. State which period that build uses, and why. (*3 marks*)

Question 3 Describing hardware

13 MARKS

A product has heaters, each switched by one GPIO pin. They are not on a bus, and there is no existing binding for them.

(a) Write the binding `qacademy,heater`. A node shall have a required `control-gpios` giving the pin that switches the heater, a required string `zone` naming the area it warms, and an integer `max-power-percent` which is 100 unless the node says otherwise.

State where in an application the file goes, and what else the build needs before it will accept the vendor prefix `qacademy` without complaint. (*5 marks*)

(b) Write the overlay that adds two of them: `heater_a` in zone `cabin` on `gpio0` pin 2, active high; `heater_b` in zone `tank` on `gpio0` pin 3, active low, limited to 60 %. Give the first the alias `primary-heater`.

State the file name that makes the build pick the overlay up without being told, and state what the overlay does to the board's own devicetree. (*4 marks*)

(c) Write the C that builds a table of every enabled heater **at compile time**, holding for each one its GPIO specification, its zone and its power limit, and then configures each pin as an inactive output. Use the table to log each heater's zone and pin.

State what changes in that table if `heater_b` is given `status = "disabled"`, and state what happens instead if its `zone` property is deleted. (*4 marks*)

Question 4 Threads and time — 13 MARKS

A system has three activities:

1. a control loop that must run every 10 ms and must not be late;
2. a display that is refreshed every 200 ms;
3. a logger that writes to flash, takes up to 50 ms, and may be late.

(a) Choose a priority for each, say for each whether it is cooperative or preemptible, and justify the three choices in terms of what happens when two of them are ready at once. Write the three `K_THREAD_DEFINE()` lines. *(5 marks)*

(b) Write the control loop's body so that it runs every 10 ms without drift, and explain why a loop whose body ends with `k_msleep(10)` does drift. Name a second way to get the same result, and state what makes it equivalent. *(4 marks)*

(c) On `native_sim`, `k_msleep(30)` returns after 40 ms, while a periodic timer started with `k_timer_start(&t, K_MSEC(30), K_MSEC(30))` expires at 30 ms, 60 ms and 90 ms. Explain both.

Then state what `CONFIG_SYS_CLOCK_TICKS_PER_SEC=1000` changes about the first, and name one cost of setting it. *(4 marks)*

Question 5 Interrupts, work and the simulator — 12 MARKS

(a) Write a sampler that takes a reading every `CONFIG_APP_SAMPLE_PERIOD_MS` milliseconds: a `k_timer` whose expiry function hands the reading over to the system work queue, and a work handler that logs it.

Name two things the expiry function must not do, and state for each what would happen if it did. State which thread the work handler runs in. *(5 marks)*

(b) Extend it: if no sample has been taken for 250 ms, log `idle`. Write the code, and state why it uses `k_work_reschedule()` rather than `k_work_schedule()`.

State, to the nearest tick, when the message is logged if the last sample was taken at 500 ms. *(4 marks)*

(c) A colleague waits for the first sample like this, with `ready` set by the timer's expiry function:

```
static volatile bool ready;

while (!ready) {
}
```

State what this program does on `native_sim`, and why. Rewrite the wait correctly. *(3 marks)*

Question 6 Threads that must agree

13 MARKS

(a) A sampler thread produces a `struct sample` every 100 ms; a storage thread takes 250 ms to write each one away. Write the message queue between them, the producer's put and the consumer's get, so that the sampler never waits and a sample that does not fit is counted as dropped.

State what `k_msgq_put()` does with the struct, and why that means the producer's local variable may go out of scope immediately. (*5 marks*)

(b) A shared calibration table is guarded by a semaphore created with `K_SEM_DEFINE(lock, 1, 1)`. Rewrite the take and the give with the primitive the book would use instead, and state the two things it gives you that the semaphore does not.

Then: three threads, at priorities 5, 6 and 7, use the CPU; the one at 7 holds the guard and the one at 5 wants it. Name what the semaphore version suffers from, and state what the replacement bounds the highest-priority thread's wait to. (*4 marks*)

(c) The same sample is now wanted by two independent parts of the application. Write a zbus channel carrying it, with a validator that rejects a reading outside −40 to 125 degrees, one listener and one subscriber.

State in which thread each of the two observers runs, and explain why a subscriber can miss a message that a listener cannot. (*4 marks*)

Question 7 A driver of your own

13 MARKS

Baro is a made-up I2C pressure sensor. Its register map is:

Register	Address	Contents
`WHO_AM_I`	`0x0F`	Reads `0x5B`, always.
`PRESSURE_L`	`0x28`	The low byte of the reading.
`PRESSURE_H`	`0x29`	The high byte. The reading is unsigned, in steps of 10 Pa.

(a) Write the driver, as a devicetree-instantiated driver in Zephyr's device model: the compatible it serves, its configuration and its run-time data, an initialization function that refuses a part that does not identify itself, the two sensor API functions that read the pressure, and the macro that defines one device per devicetree node.

The sensor API reports pressure in kilopascals, so a reading of 10132 counts is `val1` 101 and `val2` 320000. (*6 marks*)

(b) The driver is shipped as a Zephyr module. Write its `zephyr/module.yml`, the Kconfig option that turns the driver on, and the binding `qacademy,baro`.

State why the Kconfig option depends on `DT_HAS_QACADEMY_BARO_ENABLED`, and what that arranges for an application that does not have the part. (*4 marks*)

(c) An emulator for the part provides:

```
void baro_emul_set_raw(const struct emul *target, uint16_t raw);
```

Write a ztest suite with two tests: that the device is ready, and that 10132 counts is read back as 101.32 kPa. Name the file that tells Twister how to run it.

State what `device_is_ready()` returns for a device whose initialization function returned `-ENODEV`, and what the application must therefore do. (*3 marks*)

Question 8 Making it usable

12 MARKS

(a) Write the shell command `setpoint`, with two subcommands: `setpoint get`, which prints the current value, and `setpoint set <celsius>`, which accepts −40 to 125 and rejects anything else with a message.

State what the shell has already checked before your `set` handler runs, and what the handler's return value is used for. (*5 marks*)

(b) Make the setpoint survive a restart. Write the settings handler and the call that stores the value, and name the Kconfig options the application needs.

State where the value is kept on `native_sim`, and name two ways to make the next run behave as a first boot. (*4 marks*)

(c) Write a fatal error handler that aborts only the offending thread when a thread calls `k_oops()`, but halts the system for anything else.

State what the process's exit status is on `native_sim` when it halts, and why returning from a kernel panic is not allowed. (*3 marks*)

APPENDIX D

Model Answers to Paper A

Marks are shown per part. Method carries them: a correct diagnosis with a clumsy fix is worth more than a correct fix with no diagnosis, and several later parts consume earlier answers, so an error should be followed through rather than penalised twice.

Code is marked on semantics. Missing includes, missing semicolons and misplaced braces cost nothing.

Question 1 The workspace and the build — 12 MARKS

(a) — 3 MARKS

What the build is looking for. Zephyr chooses a toolchain before it compiles anything. If `ZEPHYR_TOOLCHAIN_VARIANT` is not set, it assumes the Zephyr SDK — the cross-compilers for real microcontrollers — and asks CMake to find the SDK's package configuration file. Nothing has installed one, so configuration fails before a single source file is read. (*1 mark*)

The fix is to source the environment file in that terminal:

```
source env.sh
```

It activates the Python environment that holds west and Twister, and sets two variables: `ZEPHYR_BASE`, which says where Zephyr is (`deps/zephyr`), and `ZEPHYR_TOOLCHAIN_VARIANT=host`, which tells Zephyr to build with the host's own GCC. (*1 mark*)

Why no SDK is needed. A `native_sim` image is an ordinary Linux executable: the kernel, the drivers and the application are compiled for the host CPU by the host compiler. The SDK exists to cross-compile for a target, and this book never leaves the host. (*1 mark*)

`make setup` having finished is a red herring, and saying so earns the first mark: setup creates `.venv`, writes `.west/config` and fetches Zephyr, all of which persist, whereas `source env.sh` is per terminal and is what the message is complaining about.

(b) 3 MARKS

The three files, and what each contributes:

- **`CMakeLists.txt`** — how to build it: it finds the Zephyr package and adds the application's own sources to the `app` target.
- **`prj.conf`** — how to configure Zephyr for it: a list of Kconfig settings applied on top of the board's defaults. It must exist even if it is empty, which is exactly what this error says.
- **`src/`** — the code, with a `main()`.

(1.5 marks: half for naming all three, half for what they do)

The line that does most of the work is:

```
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
```

It is where Zephyr's build system takes over: it reads the board, processes the devicetree, runs Kconfig, sets Zephyr up as a set of CMake libraries, and creates the `app` target.

(1 mark)

Why `project()` comes after it. Zephyr selects the compiler as part of that call. CMake's `project()` is what makes CMake look for a compiler, so it must not run until the choice has been made. *(0.5 marks)*

(c) 4 MARKS

What `west update` does. It reads the manifest and makes the workspace match it: it clones the `zephyr` project from `https://github.com/zephyrproject-rtos/zephyr` — the remote's `url-base` followed by the project's name — and checks out the revision `v4.4.2`. With `clone-depth: 1` it fetches only that commit rather than Zephyr's history. `path: deps/zephyr` puts it in `deps/zephyr`, inside the repository, and `self: path: .` says the manifest repository is the workspace directory itself. *(2 marks)*

Without `west-commands`. West would know only its own built-in commands. `west build` and `west twister` are extension commands that Zephyr provides, and that line is what tells west where Zephyr declares them. *(1 mark)*

Moving to a newer release is two steps: change `revision:` in `west.yml`, and run `west update`. West never moves a project on its own; pulling the repository changes the manifest, and `west update` is what makes the checkout match it. *(0.5 marks)*

Why Zephyr's own manifest is not imported. It lists some eighty modules — vendor HALs, cryptography libraries, file systems — none of which `native_sim` needs, and together they are several gigabytes. A product would import the few it needs by name. *(0.5 marks)*

(d) 2 MARKS

Why the level is still 2. A `-DCONFIG_...` argument is stored in the build directory's CMake cache, and every later build in that directory keeps using it until the directory is rebuilt pristine or the option is given a new value. The second build inherited it. (*1 mark*)

What proves the configuration. Read it back from the build:

```
grep CONFIG_LOG_DEFAULT_LEVEL build/zephyr/.config
```

`build/zephyr/.config` holds every option and the value the build actually used. Credit `west build -t menuconfig` as an alternative, as long as the candidate says it shows the same file's contents. A pristine build (`west build -p`) is the fix, not the proof. (*1 mark*)

Question 2 Kconfig and logging 12 MARKS

(a) 4 MARKS

What Kconfig did. An assignment in `prj.conf` is a *request*, not a command. `SHELL_LOG_BACKEND` depends on `SHELL`, which is `n`, so the symbol cannot be `y` and Kconfig left it at `n`. The warning says exactly that: assigned `y`, got `n`, because of the unsatisfied dependency `SHELL (=n)`. (*1.5 marks*)

Why the build stops. Zephyr turns that warning into an error. The alternative is an image built without the feature that `prj.conf` plainly asks for, and nobody finds out until the behaviour is missing at run time. Failing the build makes a request that did not take impossible to ignore. (*1 mark*)

Two ways to succeed, either of which is the right answer depending on intent:

- Enable the dependency as well: `CONFIG_SHELL=y` alongside it.
- Remove the assignment, having decided the feature is not wanted.

(*1 mark*)

Where to read the result. `build/zephyr/.config`. The same values reach C through `build/zephyr/include/generated/zephyr/autoconf.h`. (*0.5 marks*)

(b) 3 MARKS

What the compiler did. Nothing at all. `CONFIG_APP_FARENHEIT` is not defined, and an undefined macro inside `IS_ENABLED()` is indistinguishable from an option that is off, so the expression is the constant `0` and the compiler removes the branch. There is no diagnostic, because nothing is wrong as far as C is concerned. (*1 mark*)

The consequence. The feature can never be switched on. `CONFIG_APP_FAHRENHEIT=y` in `prj.conf` takes effect in the configuration, `.config` shows it enabled, and the program still prints Celsius. The build and the configuration both look right; only the behaviour is wrong. (*1 mark*)

Why the same misspelling in `prj.conf` would not survive. Kconfig knows which symbols exist. An assignment to a symbol that does not exist is a warning, and Zephyr aborts the build on it, exactly as in part (a). *(0.5 marks)*

What would have caught it. A test that checks the program's output in both configurations — the book's own exercise suites do this, building each set twice. Credit also Zephyr's compliance check for undefined Kconfig references, and credit a candidate who says that a review looking for `CONFIG_` names not present in `.config` would find it. *(0.5 marks)*

(c) 3 MARKS

What is printed. The error and the warning only. Level 2 is `WRN`, and the levels run `NONE` 0, `ERR` 1, `WRN` 2, `INF` 3, `DBG` 4. A module registered without a level takes `CONFIG_LOG_DEFAULT_LEVEL`, so this module's level is 2. *(1 mark)*

Whether the others are in the binary. No. A module's level is a **compile-time** filter: the `LOG_INF()` and `LOG_DBG()` calls expand to nothing, and their format strings are not in the image either. That is why debug logging can be left in the source for free, and why turning it on needs a rebuild rather than a setting. *(1 mark)*

How to demonstrate it. Look for the format strings in the executable:

```
strings build/zephyr/zephyr.exe | grep "Temperature"
```

The two surviving messages are there; the filtered ones are not. Credit any equivalent: comparing the two builds' sizes, or inspecting the map file. *(1 mark)*

(d) 2 MARKS

- **Immediate** (`native_sim`'s default): the message is formatted and written by the thread that logged it, in order with `printk()` and with everything else. The cost is paid at the call, which on a UART would be milliseconds inside whatever was running.
- **Deferred** (the default on a board): the call only copies the arguments into a buffer, and a low-priority log thread formats and prints later. Logging from an interrupt handler is then cheap, but a message and a `printk()` after it can appear in the opposite order, and messages still in the buffer are lost if the system dies.

(1 mark each; half for naming the modes without a consequence)

Question 3 Devicetree, bindings and overlays

13 MARKS

(a)

5 MARKS

What 32 is. A node's **ordinal**: the number the build gives each devicetree node, in dependency order. `__device_dts_ord_32` is the name of the device object for node 32. Ordinals belong to one build: the same node can be 22 in one application and 32 in another, so the number means nothing until it is looked up in *that* build's `build/zephyr/include/generated/zephyr/devicetree_generated.h`, which lists the ordinal beside the node's path. (*2 marks*)

The first message is a compile error. The device object was never *declared*, which means the build generated nothing for that node: the node is disabled, or absent. `DEVICE_DT_GET()` expanded to the address of a symbol that does not exist. The "did you mean" is noise — 38 is simply the nearest number that does exist. (*1 mark*)

The second is a link error. The name was declared, so the node exists and is enabled, but no translation unit ever *defined* that device: no driver was built for it. The node is there, the driver is not. (*1 mark*)

One change for each. For the first, disable the node:

```
&gpio0 {
        status = "disabled";
};
```

For the second, leave the node enabled and remove the driver, for example by building with `-DCONFIG_GPIO=n`. (*1 mark*)

The pair is worth learning as a pair: *undeclared* means look at the devicetree, *undefined reference* means look at Kconfig.

(b)

4 MARKS

The overlay that omits `threshold-c` fails during configuration, when the tree is checked against its bindings, before any C is compiled:

```
devicetree error: 'threshold-c' is marked as required in 'properties:' in
'.../qacademy,alarm.yaml', but does not appear in <Node /alarm0 in .../app.overlay
    ↪ :12>
```

(*1.5 marks*)

The overlay that adds `hysteresis-c` configures without complaint: the property is in the tree but the binding does not declare it, so no macro is generated for it. The failure comes later, when the C is compiled and `DT_PROP()` expands to a name that does not exist:

```
error: 'DT_N_S_alarm0_P_hysteresis_c' undeclared here (not in a function)
```

(*1.5 marks*)

Why it is a different kind of mistake from a misspelled `CONFIG_` macro. Both are typos that the tool cannot know were meant to be something else, but the devicetree one **fails loudly at compile time**, because a missing property macro is an undeclared identifier. A misspelled `CONFIG_` macro inside `IS_ENABLED()` is silently `0`, and the program builds and runs with the feature off. The devicetree's generated macros are named entities; Kconfig's are conditional compilation. (*1 mark*)

(c) 2 MARKS

What `GPIO_ACTIVE_LOW` changes. Nothing in the C code, which is the point: the flag lives in the devicetree, and the GPIO API's `_dt` functions apply it. The code says *active* and *inactive* — `gpio_pin_get_dt()` returns 1 when the button is pressed, `GPIO_INT_EDGE_TO_ACTIVE` is the interrupt for a press — and the driver turns that into the electrical level the board actually uses. (*1 mark*)

What `gpio_emul_input_set()` needs. The **physical** level, not the logical one. The emulator sits where the pin is, below the flags, so pressing an active-low button means driving the pin to 0 and releasing it means 1. (*1 mark*)

(d) 2 MARKS

A portable sample uses an alias, such as `led0` or `sw0`. An alias is a name in the tree's `aliases` node that points at whatever node plays that role on this board, so the same source builds for any board that defines it. A node label (`&gpio0`) and a node name (`gpio_emul`) are properties of one particular tree, so code that uses them is tied to that board. (*1 mark*)

`DT_NODELABEL()` turns a label into a node identifier, for the cases where you do mean one specific node: an overlay's own node, or a peripheral an application is written against. (*1 mark*)

Question 4 Threads, scheduling and time 13 MARKS

(a) 6 MARKS

Why `coop_b` does not run when its semaphore is given. `coop_a` has a cooperative priority (negative). A cooperative thread cannot be preempted: once it has the CPU it keeps it until it gives it up by blocking, sleeping or yielding. Giving a semaphore makes `coop_b` *ready*, and ready is not running. `coop_b` has the higher priority of the two, so it is first in the queue, but nothing takes the CPU away from `coop_a` to hand it over. (*2 marks*)

Why `high` does. `low` is preemptible (priority 5, non-negative). The moment `k_sem_give()` makes a higher-priority thread ready, the scheduler switches — inside the call, before it returns. The trace shows that directly: `high`'s line is printed between `low`'s "giving" line and its "back again" line. (*1.5 marks*)

What happened between the third and fourth lines of Act 1. `coop_a` gave up the CPU deliberately: it blocked, slept or called `k_yield()`. The fourth line says "because coop_a

yielded", so the answer expected is `k_yield()`; credit any of the three if the candidate says the switch cannot happen without one of them. (*1 mark*)

Which priorities are which, and what it means for a critical section. Negative priorities are cooperative and non-negative ones are preemptible. A cooperative thread's critical section is protected from every other thread by the fact that it runs, but not from interrupts, and it delays every higher-priority thread for as long as it runs — which is why cooperative threads are for short work, and why a cooperative thread that blocks gives all of that protection away at the moment it blocks. (*1.5 marks*)

(b) 4 MARKS

Why the sleep ends at 510 ms. Timeouts are kept in ticks, and `native_sim` has 100 ticks per second, so 500 ms is 50 ticks. A thread calling `k_msleep()` cannot know how much of the current tick has already passed, so the kernel adds one tick to be sure the sleep is never *shorter* than asked: 51 ticks, 510 ms. A sleep lasts at least the time requested, and up to one tick longer. (*2 marks*)

Why the timer fires at exactly 500 ms. `k_timer_start()` compensates: it subtracts that extra tick from the duration before handing it to the timeout machinery, precisely so that a timer expires when the application said it should rather than one tick late. (*1 mark*)

What a periodic timer does about it. Nothing accumulates: the period is measured from the scheduled expiry, not from when the handler ran, so a periodic timer does not drift — 100, 200, 300, even if a handler is late. (*1 mark*)

(c) 3 MARKS

What returning from `main()` ends. The `main` thread, and nothing else. `main()` is just the entry point of one thread; the kernel, every other thread and every timer keep running. With nothing else to do, the idle thread runs. A microcontroller has nothing to return *to*, so there is no notion of the system exiting. (*1.5 marks*)

Making it stop by itself. Run it with `-stop_at=<seconds>`, which stops the program after that much simulated time. Credit `Ctrl+C` as the interactive answer. (*1 mark*)

What `-no-rt` changes. How fast simulated time advances relative to the host's clock: without it the simulator paces itself to real time, with it the run finishes as fast as the host can manage. What it does not change is a single thing the program sees — the uptimes, the order of events and the output are identical, because everything the program observes is simulated time. (*0.5 marks*)

Question 5 Interrupt context, work queues and simulated time

12 MARKS

(a)

5 MARKS

A `k_timer` expiry function runs in **interrupt context**, from the system clock's interrupt. Three defects follow from that:

1. **`k_msleep(5)`.** Sleeping in an ISR is illegal: there is no thread to suspend. With `CONFIG_ASSERT=y` the kernel catches it and panics; without assertions it is undefined behaviour in the scheduler. (*1.5 marks*)
2. **`k_mutex_lock(&bus_mutex, K_FOREVER)`.** A mutex has an owner and may block, so it may not be taken from an ISR at all; a timeout other than `K_NO_WAIT` is doubly illegal there. (*1.5 marks*)
3. **The work itself.** `read_sensor()` and `store()` are bus traffic and storage in interrupt context: everything in the system, including higher-priority threads, waits for them. `LOG_INF()` belongs to the same complaint — cheap under deferred logging, not free, and on `native_sim` immediate. (*1 mark*)

The shape it should have. The expiry function does one thing: it submits a work item and returns.

```
static void sample_work_handler(struct k_work *work)
{
        struct sample sample;

        k_mutex_lock(&bus_mutex, K_FOREVER);
        read_sensor(&sample);
        k_mutex_unlock(&bus_mutex);
        store(&sample);
}

K_WORK_DEFINE(sample_work, sample_work_handler);

static void sample_expiry(struct k_timer *timer)
{
        k_work_submit(&sample_work);
}
```

The handler runs in a work queue's thread, where sleeping, locking and logging are all allowed. (*1 mark*)

(b) 3 MARKS

A 200 ms work item. The work queue is one thread running items one after another, so every other item queued on it waits 200 ms. Nothing else on that queue can overtake it, whatever its urgency. (*1 mark*)

An item that blocks forever. The system work queue is dead: no further item on it ever runs. Anything in Zephyr that uses the system queue behind your back stops working too. (*1 mark*)

When to start your own queue. When the work must not be delayed by — or must not delay — other users of the system queue: work that blocks, work that takes long, or work that needs a priority of its own. One condition stated clearly earns the mark. (*1 mark*)

(c) 4 MARKS

What `native_sim` does. It hangs, permanently. Code runs in zero simulated time, and the simulator advances the clock and delivers interrupts only when every Zephyr thread is waiting. A loop that never calls into the kernel never lets that happen, so the interrupt that would set `ready` can never arrive, and no other thread runs either. `-stop_at` cannot end it, because the simulated time it is waiting for never arrives. (*2 marks*)

What it costs on real hardware. It works, and it is still bad: the CPU spins at full power until the interrupt arrives, delaying nothing but wasting everything. The interrupt is real there, so it does preempt the loop. (*1 mark*)

Two correct rewrites. Any two of:

- A semaphore: the ISR calls `k_sem_give(&ready_sem)` and `main()` calls `k_sem_take(&ready_sem, K_FOREVER)`.
- `k_poll()` on a `k_poll_signal` raised by the ISR, which generalises to waiting for several things.
- As a last resort, keeping the loop but letting the CPU idle inside it, with `k_cpu_idle()` or a short `k_busy_wait()`; the book calls this the workaround rather than the fix.

(*1 mark for two*)

Question 6 Synchronization and communication 13 MARKS

(a) 6 MARKS

The phenomenon is priority inversion, and the unbounded kind: the highest-priority thread in the system waits for a thread that is not even using the lock. (*1 mark*)

579 ms, with the semaphore. `high` asks at 21 ms and gets the lock at 600 ms. `low` had done 31 ms of its 100 ms of work when `medium` woke at 31 ms and preempted it — `medium` has the higher priority and needs no lock — and ran to 531 ms. Only then did `low` get the CPU back to finish its remaining 69 ms, reaching 600 ms, where it released the lock. So `high`'s wait is `low`'s critical section *plus the whole of medium's unrelated work*: $600 - 21 = 579$. (*2 marks*)

79 ms, with the mutex. `high` asks at 21 ms and gets the lock at 100 ms, which is exactly the rest of `low`'s 100 ms critical section: $100 - 21 = 79$. `medium` never gets in. (*1 mark*)

What the kernel did to `low`. Priority inheritance. When `high` blocked on the mutex at 21 ms, the kernel raised the owner's priority to the waiter's, so `low` ran at priority 5 — the trace shows it, `low (priority 5)` — and `medium` at 6 could no longer preempt it. The kernel restored `low`'s own priority at the unlock, which is why `medium` then runs and `low` finishes last. (*1.5 marks*)

What the mutex does not fix. It does not remove the wait: `high` still waits for the whole of `low`'s critical section, and keeping that short is still the designer's job. Inheritance bounds the wait by the critical section, it does not eliminate it. Credit equally: it does nothing for a semaphore used as a lock, it does not prevent deadlock, and it cannot help a lock held across a blocking call. (*0.5 marks*)

(b) 3 MARKS

What `-ENOMSG` means here. The queue is full and the call was told not to wait, so nothing was queued: the sample is dropped. The producer is being told that the consumer is not keeping up. (*1 mark*)

What `K_FOREVER` would do. Block the producer until a slot appears. Nothing is lost, and the producer is throttled to the consumer's rate — which is right for a pipeline and wrong for anything that must keep sampling on time. (*1 mark*)

Which an interrupt handler may use. Only `K_NO_WAIT`. An ISR may not block, so the only choice in interrupt context is to accept that a full queue means a dropped message — and, ideally, to count the drops. (*1 mark*)

(c) 4 MARKS

The subscriber's behaviour. A subscriber is notified through a queue of channel *references*, not of copies: it is told which channel changed, and reads the channel's current value when its thread eventually runs. If two messages are published before it runs, both notifications are waiting, and both reads return the second message — so it prints the newer reading twice and never sees the one that was overwritten. That is not a bug: for something like a display, only the latest value matters. A message subscriber is queued a *copy* of each message, so it sees every one exactly once, at the cost of the buffer the copy lives in. (*2.5 marks*)

Where a listener runs. Inside `zbus_chan_pub()`, in the **publisher's** thread, with the channel's lock held. It therefore must not block, must not sleep, and must not publish to the same channel; it should copy or flag and return. That is also why a listener never misses a message: it is called before the publish returns. (*1.5 marks*)

Question 7 Writing a driver

13 MARKS

(a)

4 MARKS

What `device_is_ready()` reports on. Two things: that the device exists in this build and was initialized by the kernel, and that its init function reported success. It is a query of the device's initialization state, not a probe of the hardware. (*1.5 marks*)

What the driver most likely did. Its init function returned a negative errno — the usual cause being the identity check: the driver read the part's `WHO_AM_I` register over I2C and got something other than the expected value, and returned `-ENODEV`. The kernel marks the device not ready and carries on booting. (*1.5 marks*)

If the application used it anyway. It would call into a driver whose state was never set up: `dev->data` holds whatever init left, the bus may never have been configured, and the reads return stale or nonsense values, or fault. The whole point of returning the error at init is that the check is the one place to catch it, so a user that skips the check turns a clean "not ready" into a misbehaving system. (*1 mark*)

(b)

5 MARKS

1. The devicetree values are hard-coded (`DEVICE_DT_GET(DT_NODELABEL(i2c0))` and `0x48`) rather than taken from the instance being defined. The second node in the tree gets the first one's bus and address, so two instances talk to the same chip, and the driver silently ignores what the devicetree says. The corrected shape takes them from the instance:

```
.i2c = I2C_DT_SPEC_INST_GET(inst),
```

filled in by the instance macro that defines each device. (*2 marks*)

2. The `i2c_dt_spec` lives in `dev->data`. It never changes, so it belongs in a `const` config struct reached through `dev->config`, which on a real target lives in flash. As written, every instance spends RAM on constants, and nothing stops the driver from modifying what the devicetree said:

```
struct thermo_config {
        struct i2c_dt_spec i2c;
};
```

with `dev->data` left for the part that does change, such as `last_sample`. (*1.5 marks*)

3. The identity read's result is discarded and init returns 0 regardless. `(void)` on `i2c_reg_read_byte_dt()` throws away the bus error, and `id` is never compared with anything, so a missing, broken or wrong part reports itself ready and the application trusts it. Init must check both:

```
ret = i2c_reg_read_byte_dt(&config->i2c, THERMO_REG_WHO_AM_I, &id);
if (ret < 0) {
        return ret;
}
if (id != THERMO_WHO_AM_I) {
        return -ENODEV;
}
```

(*1.5 marks*)

Credit also, without extra marks, a candidate who notes that init should check the bus itself with `i2c_is_ready_dt(&config->i2c)` before using it, that `DT_DRV_COMPAT` is only useful with the `DT_INST_*` macros and `DT_INST_FOREACH_STATUS_OKAY()`, and that nothing here would define more than one device in the first place.

(c) 2 MARKS

Where the symbol comes from. The build generates one `DT_HAS_<COMPATIBLE>_ENABLED` symbol per compatible that appears on an enabled node in the final devicetree, and Kconfig can depend on it because Kconfig runs after devicetree. (*1 mark*)

What it achieves, and what disabling the node does. The driver's option can only be on when the hardware is in the tree, so a build that has no such part cannot accidentally include the driver, and one that does gets it by default. With `status = "disabled"`, the symbol is not generated, the option goes to `n`, the driver is not compiled — and the application's `DEVICE_DT_GET()` fails at compile time with the `__device_dts_ord_N` undeclared error of Question 3. (*1 mark*)

(d) 2 MARKS

What the numbers order. Initialization within the same level: at `POST_KERNEL`, every device with priority 50 is initialized before every device with priority 90. The pair exists so that a device is initialized after whatever it depends on. (*1 mark*)

If the sensor were 40. The sensor's init would run before the I2C controller's, so its identity read would go through a bus that is not yet initialized. It would fail, init would return an error, and the device would be not ready for the rest of the run — a boot-order bug that looks exactly like broken hardware. (*1 mark*)

Question 8 Subsystems and debugging

12 MARKS

(a)

3 MARKS

- **`-22` is `-EINVAL`, returned by the command's own handler.** The handler validated its argument, printed why it was refused, and returned the errno.
- **`-8` is `-ENOEXEC`, returned by the shell itself** for a command it does not know.
- **`1` is `SHELL_CMD_HELP_PRINTED`**, returned when a command with subcommands is run without one: all it did was print its help.

(*1.5 marks, half each*)

What the shell does with the value. It does not print it, but it keeps it, and the built-in `retval` command prints the previous command's return value. (*0.5 marks*)

Why return a negative errno. Because the value is what a script, a test, or anything driving the shell can act on: a handler that prints an error and returns 0 reports success to everything except a human reading the screen. Returning the errno makes the failure machine-readable and keeps the shell's own conventions intact. (*1 mark*)

(b)

3 MARKS

Where the counter lives. In the settings subsystem's storage, on the board's flash — and on `native_sim` that flash is a file, `flash.bin`, in the directory the program runs in. `west build -t run` runs the program in the build directory, so the file is `build/flash.bin`. (*1.5 marks*)

Why a pristine build resets it. `west build -p` deletes the build directory and everything in it, `flash.bin` included. The next run therefore starts with erased flash — a factory-new device — and the counter starts again at 1. (*1 mark*)

Making every run a first boot without deleting anything. `-flash_in_ram`, which keeps the simulated flash in RAM only and leaves the file alone. Credit `-flash_erase`, which erases at startup, as an answer that also achieves it; credit `-flash=<path>` only if the candidate explains that it selects a different file. (*0.5 marks*)

(c)

3 MARKS

The difference. Reason 3 is a **kernel oops**: an error attributed to the thread that caused it, raised by `k_oops()` or an equivalent. Reason 4 is a **kernel panic**, which a failed `__ASSERT()` raises: an error the system as a whole cannot continue from. (*1 mark*)

What the handler must do. For the oops it may **return**, and the kernel then aborts only the offending thread — which is what the first trace shows, `main()` carrying on afterwards. For the panic it must **not** return; it has to halt or reboot the system, because there is no defined state to go back to. (*1 mark*)

The exit status. On `native_sim`, halting means the process exits with **status 1**, which is also why a test that ends this way is reported as a failure rather than as output. (*1 mark*)

(d)

3 MARKS

What the report tells you. The kind of error (a write past the end of a stack buffer), the address, the thread, and the exact source line that did it — with a frame pointing at `fill()` in `main.c:28`. On a microcontroller the same write would simply have landed on whatever variable was next in memory, and the symptom would have appeared somewhere else entirely, later, as corrupted data. (*1.5 marks*)

Why it stops. AddressSanitizer halts the program at the first error rather than letting it continue on memory it has proved is wrong; the process exits with status 1. (*0.5 marks*)

Why only on this board. AddressSanitizer is the host toolchain's: it instruments the program and links it against the host's runtime, which exists because a `native_sim` image is an ordinary Linux program. There is no such runtime in a Cortex-M image, and `CONFIG_ASAN` depends on the POSIX architecture accordingly. Credit a candidate who adds that UndefinedBehaviorSanitizer is not in the same position — it can trap on a target through `CONFIG_UBSAN_TRAP` — and that this is a large part of why it pays to keep code runnable on `native_sim`. (*1 mark*)

APPENDIX E

Model Answers to Paper B

Marks are shown per part. Method carries them: a correct design with a clumsy fix is worth more than a correct fix with no reasoning, and several later parts consume earlier answers, so an error should be followed through rather than penalised twice.

Code is marked on semantics. Missing includes, missing semicolons and misplaced braces cost nothing.

Question 1 The workspace and the build — 12 MARKS

(a) — 4 MARKS

```
manifest:
  remotes:
    - name: zephyrproject-rtos
      url-base: https://github.com/zephyrproject-rtos

  projects:
    - name: zephyr
      remote: zephyrproject-rtos
      revision: v4.4.2
      path: deps/zephyr
      clone-depth: 1
      west-commands: scripts/west-commands.yml

  self:
    path: .
```

(*2 marks*)

One mark for a manifest with a remote and a pinned project, one for `path`, `clone-depth` *and* `west-commands` *together.* `self: path: .` *earns its mark below.*

Credit each of these where the candidate says it:

- **`revision`** may be a tag, a branch or a commit hash; a product pins a tag or a hash, so that the same manifest checks out the same code next year.
- **`clone-depth: 1`** fetches only the pinned commit, which is most of the download; the cost is that `git log` in `deps/zephyr` shows one commit.
- **`west-commands`** is what makes `west build` and `west twister` exist. Without it west has only its own commands, because the book's manifest does not import Zephyr's.
- **`self: path: .`** says the manifest repository *is* the workspace directory.

What `west update` does: it checks out every project in the manifest at the revision the manifest names, cloning it first if it is not there. *(1 mark)*

What it does not do: it does not update the manifest. Nothing fetches a newer Zephyr by itself: somebody edits `revision`, or a `git pull` of the companion repository brings a new manifest in, and only then does `west update` make the workspace match it. Accept also: it does not build anything, and it does not touch projects already at the right revision.

(1 mark)

(b)

4 MARKS

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ready)

target_sources(app PRIVATE src/main.c)
```

```
CONFIG_LOG=y
```

```
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>

LOG_MODULE_REGISTER(app, LOG_LEVEL_INF);

int main(void)
{
        LOG_INF("ready");
        return 0;
}
```

(2 marks)

One mark for the three CMake lines plus `target_sources`, one for the module registration and the log call. `prj.conf` must exist, and `CONFIG_LOG=y` must be in it; an application with no `prj.conf` fails to configure with `No prj.conf file(s) was found`.

What `find_package(Zephyr)` does. It is where Zephyr's build system takes over: it finds Zephyr through `ZEPHYR_BASE`, reads the board, processes the devicetree, runs Kconfig, sets Zephyr up as a set of CMake libraries, and creates the library target `app` that the application's own sources are added to. (*1 mark*)

Why `project()` comes after it. Zephyr selects the toolchain during `find_package`, and CMake's `project()` is what makes a compiler current. Calling `project()` first would have CMake look for a host compiler before Zephyr had said which compiler to use. (*1 mark*)

(c) 4 MARKS

```
west build -p -b native_sim/native/64 app
west build -t run
```

(*1 mark*)

`-p` is pristine: it deletes the build directory first. Accept `./build/zephyr/zephyr.exe` *in place of* `west build -t run`.

```
west build -p -b native_sim/native/64 app -- -DEXTRA_CONF_FILE=debug.conf
```

(*1 mark*)

Everything after `--` goes to CMake. `CONF_FILE` in place of `EXTRA_CONF_FILE` is wrong and earns nothing: it replaces `prj.conf` rather than merging on top of it.

Three files, any three of:

File	What it holds
`build/zephyr/.config`	Every Kconfig option and the value the build settled on.
`build/zephyr/include/generated/zephyr/autoconf.h`	The same configuration as C macros, included into every source file.
`build/zephyr/zephyr.dts`	The final devicetree, overlays applied.
`build/zephyr/include/generated/zephyr/devicetree_generated.h`	The devicetree as C macros.
`build/zephyr/zephyr.exe`	The program itself, on `native_sim`.
`build/zephyr/zephyr.map`	The linker map: where every symbol ended up.
`build/compile_commands.json`	Every compiler command line, for editors.

(*2 marks*)

Question 2 Configuring an application

12 MARKS

(a)

5 MARKS

```
menu "Application"

config APP_SAMPLE_PERIOD_MS
        int "Milliseconds between samples"
        range 10 10000
        default 1000

config APP_ALARM
        bool "Warn when a reading reaches the limit"

config APP_ALARM_LIMIT_C
        int "Temperature at which the alarm warns, in degrees Celsius"
        depends on APP_ALARM
        default 30

endmenu

source "Kconfig.zephyr"
```

(*3 marks*)

One mark for the two typed options with prompt, range and defaults; one for `depends on APP_ALARM` *on the limit; one for* `source "Kconfig.zephyr"` *at the end. An application's* `Kconfig` *replaces Zephyr's top-level one, so without that last line not one of Zephyr's own options exists and the build fails on the board's own defaults, with* `attempt to assign the value 'y' to the undefined symbol CONSOLE`.

```
CONFIG_APP_SAMPLE_PERIOD_MS=250
CONFIG_APP_ALARM=y
```

(*1 mark*)

With the alarm off, setting the limit aborts the build. `APP_ALARM_LIMIT_C` depends on `APP_ALARM`, so with the alarm off the symbol does not exist and the assignment cannot take. Kconfig says so, and Zephyr treats the warning as an error:

```
warning: APP_ALARM_LIMIT_C (defined at Kconfig:11) was assigned the value '35' but
    ↪ got the
value ''. Check these unsatisfied dependencies: APP_ALARM (=n) ...

error: Aborting due to Kconfig warnings
```

The alternative, a build that silently ignores the line, is what the abort exists to prevent.
(*1 mark*)

(b) 4 MARKS

```
target_sources_ifdef(CONFIG_APP_ALARM app PRIVATE src/alarm.c)
```

(*1 mark*)

Kconfig has run by the time `find_package(Zephyr)` returns, so every option is a CMake variable too; this is how nearly all of Zephyr is built.

```
LOG_MODULE_REGISTER(sampler, LOG_LEVEL_INF);

/* ... */
        LOG_INF("Reading %d C", celsius);

        if (IS_ENABLED(CONFIG_APP_ALARM)) {
                alarm_check(celsius);
        }
```

with `alarm.c` holding

```
LOG_MODULE_DECLARE(sampler);

void alarm_check(int celsius)
{
        if (celsius >= CONFIG_APP_ALARM_LIMIT_C) {
                LOG_WRN("Reading %d C is at or above the limit of %d C", celsius,
                        CONFIG_APP_ALARM_LIMIT_C);
        }
}
```

(*2 marks*)

One mark for the module registration and the guarded call, one for `LOG_MODULE_DECLARE` in the second file. A second `LOG_MODULE_REGISTER` of the same name is the mistake here; the module is registered once and declared everywhere else.

Why `IS_ENABLED()`. It expands to `1` or `0`, so the call is compiled and type-checked in every configuration, and the compiler removes the branch that cannot run — call included, which is why the program still links when `alarm.c` is not compiled. Code behind `#ifdef` is not even parsed when the option is off, so it rots unnoticed until somebody builds the other configuration. (*1 mark*)

(c) 3 MARKS

In order:

1. the option's own `default` in the `Kconfig` file;
2. the board's defaults, from its `_defconfig`;
3. the application's `prj.conf`, and any board-specific `.conf` beside it;
4. extra configuration files given with `-DEXTRA_CONF_FILE=`;
5. `-DCONFIG_...` on the `west build` command line.

The last one applied wins, so the command line beats everything else. Accept any four of these five in the right order. (*2 marks*)

The second build still uses 50 ms. A `-DCONFIG_...` option is stored in the build directory's CMake cache, and every later build in that directory keeps using it until the directory is rebuilt with `-p` or the option is given a new value. This is one of the ways a build directory goes stale, and why a pristine build is the only way to be sure that nothing but the files and the command line decide the configuration. (*1 mark*)

Question 3 Describing hardware 13 MARKS

(a) 5 MARKS

```
description: |
  A heater driven by one GPIO pin.

compatible: "qacademy,heater"

properties:
  control-gpios:
    type: phandle-array
    required: true
    description: The pin that switches the heater on.

  zone:
    type: string
    required: true
    description: The name of the zone this heater warms.

  max-power-percent:
    type: int
    default: 100
    description: The highest duty cycle this heater may be driven at.
```

(*3 marks*)

One mark for `compatible` and the three property names, one for the types, one for `required: true` on two and `default: 100` on the third. A `default` on a required property is a contradiction and loses the last mark.

Where it goes: `dts/bindings/` in the application directory — the application's own directory is searched for bindings, alongside Zephyr's `dts/bindings`. The file name does not matter to the build, but the convention is the compatible: `dts/bindings/qacademy,heater.yaml`. *(1 mark)*

The vendor prefix. `qacademy` is not one of the prefixes Zephyr knows, so the build warns `unknown vendor prefix 'qacademy'`. A `dts/bindings/vendor-prefixes.txt` beside the binding, with the line `qacademy` and a tab and a description, registers it. This matters more than the warning suggests: Twister builds with `--edtlib-Werror`, where the same warning is an error. *(1 mark)*

(b) 4 MARKS

```
/ {
        heater_a: heater-a {
                compatible = "qacademy,heater";
                control-gpios = <&gpio0 2 GPIO_ACTIVE_HIGH>;
                zone = "cabin";
        };

        heater_b: heater-b {
                compatible = "qacademy,heater";
                control-gpios = <&gpio0 3 GPIO_ACTIVE_LOW>;
                zone = "tank";
                max-power-percent = <60>;
        };

        aliases {
                primary-heater = &heater_a;
        };
};
```

(3 marks)

One mark for two nodes with the right compatible at the root, one for the `gpios` cells with their flags, one for the alias. `max-power-percent` must be left out of `heater_a`, which is what makes the binding's default do its work.

The file name is `app.overlay`, in the application directory: the build picks that up with no argument and no CMake line. Accept `boards/native_sim_native_64.overlay` with the note that a board-named overlay *replaces* `app.overlay` rather than adding to it.

What it does: the overlay is merged on top of the board's devicetree before anything is generated from it, adding nodes, changing properties, or disabling nodes the board declared.

The board's own file is not edited; the result is what `build/zephyr/zephyr.dts` shows.

(1 mark)

(c) 4 MARKS

```
struct heater {
        struct gpio_dt_spec control;
        const char *zone;
        int max_power_percent;
};

#define HEATER_ENTRY(node)                                                  ↩
    ↪ \
        {                                                                   ↩
    ↪ \
                .control = GPIO_DT_SPEC_GET(node, control_gpios),           ↩
    ↪ \
                .zone = DT_PROP(node, zone),                                ↩
    ↪ \
                .max_power_percent = DT_PROP(node, max_power_percent),      ↩
    ↪ \
        },

static const struct heater heaters[] = {DT_FOREACH_STATUS_OKAY(qacademy_heater, ↩
    ↪ HEATER_ENTRY)};

int main(void)
{
        for (size_t i = 0; i < ARRAY_SIZE(heaters); i++) {
                const struct heater *h = &heaters[i];

                if (!gpio_is_ready_dt(&h->control)) {
                        LOG_ERR("%s: controller not ready", h->zone);
                        continue;
                }
                gpio_pin_configure_dt(&h->control, GPIO_OUTPUT_INACTIVE);
                LOG_INF("%-6s pin %u, at most %d %%", h->zone, h->control.pin,
                        h->max_power_percent);
        }
        return 0;
}
```

(3 marks)

One mark for `DT_FOREACH_STATUS_OKAY` with the compatible spelled in C, commas and dashes replaced by underscores, and a macro that expands to one initializer including its trailing comma;

one for `GPIO_DT_SPEC_GET` and `DT_PROP`; one for the readiness check and configuring the pin as an inactive output. The table is `static const`: it is built by the compiler and lives in flash. Nothing here is a search at run time.

With `status = "disabled"` the node is not "status okay", so `DT_FOREACH_STATUS_OKAY` does not expand it: the table has one entry, `ARRAY_SIZE` is 1, and nothing else changes. It still builds.

With `zone` deleted the build stops during configuration, before any C is compiled:

```
devicetree error: 'zone' is marked as required in 'properties:' in
'.../dts/bindings/qacademy,heater.yaml', but does not appear in <Node /heater-b in
'.../app.overlay'>
```

That is the binding doing its job: a required property is a promise the devicetree must keep. (*1 mark*)

Question 4 Threads and time

13 MARKS

(a)

5 MARKS

A defensible answer, and the one the book leads to:

Activity	Priority	Kind	Why
Control loop	`-1`	Cooperative	It must not be late. A cooperative thread cannot be preempted at all, so once it is running nothing else runs until it blocks — which it does, on its next period.
Display	`5`	Preemptible	It must not delay the control loop, and it does not mind being interrupted part-way through a refresh.
Logger	`7`	Preemptible	It is the one that may be late. At the lowest priority it runs only when the other two are waiting, and its 50 ms of flash work delays nothing.

```
K_THREAD_DEFINE(control, 1024, control_thread, NULL, NULL, NULL, -1, 0, 0);
K_THREAD_DEFINE(display, 1024, display_thread, NULL, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(logger, 2048, logger_thread, NULL, NULL, NULL, 7, 0, 0);
```

(*5 marks*)

*Three marks for the three choices with a reason each; two for the three definitions. In Zephyr a **lower number is a higher priority**, and negative priorities are the cooperative ones: getting that backwards costs the marks for the choices. Any consistent set of preemptible priorities is fine — what is marked is control above display above logger, and the reasoning.*

Credit as an addition: the logger is the thread to give the larger stack, because it is the one that calls into a file system or flash; and a cooperative control loop is only safe because its body is short and ends in a wait.

(b) 4 MARKS

```
static K_TIMER_DEFINE(control_tick, NULL, NULL);

static void control_thread(void)
{
        k_timer_start(&control_tick, K_MSEC(10), K_MSEC(10));

        while (true) {
                k_timer_status_sync(&control_tick);   /* Waits for the next expiry
    ↪ . */
                control_step();
        }
}
```

(*2 marks*)

`k_timer_status_sync()` blocks until the timer has expired at least once and returns how many times it has, so a slow iteration shows up as a count above 1 rather than as drift.

Why `k_msleep(10)` drifts. It sleeps for 10 ms *from the moment it is called*, which is after the body has run. Every iteration is therefore the body plus 10 ms plus the rounding to the next tick, and the error accumulates: the loop's period is never 10 ms and the tenth iteration is late by ten roundings. A periodic timer measures from the previous expiry, so its errors do not add up. (*1 mark*)

A second way: absolute timeouts. Keep a deadline and sleep until it, rather than for a duration:

```
int64_t next = k_uptime_get() + 10;

while (true) {
        k_sleep(K_TIMEOUT_ABS_MS(next));
        control_step();
        next += 10;
}
```

This is equivalent because the wake-up time, not the sleep's length, is what is specified: the body's duration comes out of the wait rather than being added to it. (*1 mark*)

(c) 4 MARKS

`k_msleep(30)` returns after 40 ms. The kernel keeps timeouts in ticks, and `native_sim` has 100 of them a second, so 30 ms is three ticks. The thread calls at some point *within* the current tick, and the kernel cannot know how much of it is left, so it waits the three ticks the duration needs plus the remainder of the one in progress: four tick boundaries, 40 ms. A sleep is a *minimum*. (*2 marks*)

The timer expires at 30, 60 and 90. `k_timer_start()` compensates for exactly that added tick, and a periodic timer's next expiry is measured from the previous one rather than from when the handler ran, so a periodic timer does not drift. (*1 mark*)

With 1000 ticks per second, a tick is 1 ms, so the same sleep returns after 31 ms: the rounding is still there but is ten times smaller. The cost is that the kernel's clock bookkeeping happens ten times as often; on a board that is more timer interrupts, more power, and less time asleep. (*1 mark*)

Question 5 Interrupts, work and the simulator 12 MARKS

(a) 5 MARKS

```
static void sample_work_handler(struct k_work *work);

static K_WORK_DEFINE(sample_work, sample_work_handler);

static void sample_timer_expired(struct k_timer *timer)
{
        ARG_UNUSED(timer);
        k_work_submit(&sample_work);          /* All an ISR should do: hand the ↪
    ↪ work on. */
}

static K_TIMER_DEFINE(sample_timer, sample_timer_expired, NULL);

static void sample_work_handler(struct k_work *work)
{
        ARG_UNUSED(work);
        LOG_INF("sample %d", read_sensor());
}

int main(void)
{
        k_timer_start(&sample_timer, K_MSEC(CONFIG_APP_SAMPLE_PERIOD_MS),
                      K_MSEC(CONFIG_APP_SAMPLE_PERIOD_MS));
```

```
        return 0;
}
```

(*3 marks*)

One mark for the timer with a period, one for the expiry function doing nothing but submitting, one for the work item and its handler.

Two things the expiry function must not do:

- **Sleep or block** — `k_sleep()`, `k_mutex_lock()` with a timeout, `k_msgq_get(..., K_FOREVER)`. A timer's expiry function runs in the system clock's interrupt handler, and there is no thread to put to sleep. With `CONFIG_ASSERT=y` the kernel catches it and panics; without, the system misbehaves.
- **Take long** — the expiry function delays every interrupt behind it and every thread. Anything that reads a sensor over a bus, writes to flash, or formats a long log line belongs in the work handler.

(*1 mark*)

Which thread the handler runs in: the system work queue's thread, `sysworkq`, a cooperative thread shared by every subsystem. That is why it may sleep and log, and why it should still be quick: everybody else's work is queued behind it. A queue of your own, started with `k_work_queue_start()`, is the answer when the work is slow. (*1 mark*)

(b) 4 MARKS

```
static void idle_work_handler(struct k_work *work)
{
        ARG_UNUSED(work);
        LOG_INF("idle");
}

static K_WORK_DELAYABLE_DEFINE(idle_work, idle_work_handler);
```

and, at the end of the sample handler:

```
        k_work_reschedule(&idle_work, K_MSEC(250));
```

(*2 marks*)

Why `k_work_reschedule()`. It *moves* an already scheduled item to the new time; `k_work_schedule()` leaves an already scheduled item alone. With `schedule`, the timeout would keep the time set by the first sample and fire 250 ms after *that*, whatever happened since. Pushing the deadline back on every sample is exactly what "no sample for 250 ms" means. (*1 mark*)

When it fires. 250 ms is 25 ticks, plus the remainder of the tick in progress: 26 ticks, so 260 ms after the last sample. With the last sample at 500 ms, the message is logged at **760 ms**. Accept "at 750 ms, give or take a tick" with the rounding named. (*1 mark*)

(c) 3 MARKS

It hangs, and takes the whole system with it. On `native_sim` the code runs in zero simulated time and interrupts arrive only when the CPU is idle, which it never becomes while a thread spins. The timer's expiry function is never called, `ready` is never set, the clock never advances, and even `-stop_at` cannot end the run, because that too is a simulated-time event. On a real microcontroller the same loop would end — the timer interrupt would preempt it — but it would burn the CPU for the whole wait. *(2 marks)*

The fix is to wait on something rather than to look at it:

```
static K_SEM_DEFINE(ready, 0, 1);

static void sample_timer_expired(struct k_timer *timer)
{
        k_sem_give(&ready);
}

/* In the thread: */
        k_sem_take(&ready, K_FOREVER);
```

(1 mark)

Accept `k_poll`, `k_timer_status_sync()`, or a `k_msgq` or zbus wait. A `k_msleep()` inside the loop is worth half: it lets time pass, so it works, but it is polling.

Question 6 Threads that must agree 13 MARKS

(a) 5 MARKS

```
struct sample {
        uint32_t seq;
        int32_t celsius;
};

K_MSGQ_DEFINE(samples, sizeof(struct sample), 4, alignof(struct sample));

/* The sampler, every 100 ms: */
        struct sample s = {.seq = ++seq, .celsius = read_sensor()};

        if (k_msgq_put(&samples, &s, K_NO_WAIT) != 0) {
                dropped++;
        }

/* The storage thread: */
```

```
        struct sample s;

        while (k_msgq_get(&samples, &s, K_FOREVER) == 0) {
                store(&s);
        }
```

(*4 marks*)

One mark for `K_MSGQ_DEFINE` *with the message size, the depth and the alignment; one for* `K_NO_WAIT` *and counting the failure; one for* `K_FOREVER` *in the consumer; one for checking both return values. A producer that passes a timeout, or ignores the result of* `k_msgq_put()`*, has silently dropped the sample instead of counting it.*

What `k_msgq_put()` does with the struct: it copies the bytes into the queue's own storage, which was allocated when the queue was defined. Nothing is shared and nothing is owned, so the producer's local may go out of scope the moment the call returns — and a reader gets a snapshot, not a reference to something that may have changed since. (*1 mark*)

(b)

4 MARKS

```
static K_MUTEX_DEFINE(calibration_lock);

        k_mutex_lock(&calibration_lock, K_FOREVER);
        /* ... the critical section ... */
        k_mutex_unlock(&calibration_lock);
```

(*1 mark*)

Two things a mutex gives you that a binary semaphore does not:

- **Ownership.** A mutex is held by a thread, and only that thread can unlock it; it is also recursive for its owner. A semaphore has no owner: any thread can give it, including one that never took it, so an unbalanced give goes unnoticed.
- **Priority inheritance.** While a higher-priority thread waits for the mutex, the kernel raises the holder's priority to the waiter's, and restores it at the unlock.

Credit also: a mutex may not be used in an ISR, which is a useful restriction, since an ISR has no priority to inherit and no identity to own with. (*2 marks*)

What the semaphore version suffers from: unbounded priority inversion. The thread at 7 holds the guard; the one at 5 blocks on it; the one at 6, which never touches the guard, preempts 7 and runs for as long as it likes. The highest-priority thread in the system waits for the middle one, and its wait is bounded by nothing anybody designed.

With the mutex, 7 runs at 5's priority while it holds the lock, so 6 cannot preempt it. The highest-priority thread's wait is bounded by the length of the critical section — which is the thing the designer controls, and the reason to keep it short. (*1 mark*)

(c) 4 MARKS

```
static bool sample_is_valid(const void *msg, size_t msg_size)
{
        const struct sample *s = msg;

        ARG_UNUSED(msg_size);
        return (s->celsius >= -40) && (s->celsius <= 125);
}

static void alarm_callback(const struct zbus_channel *chan)
{
        const struct sample *s = zbus_chan_const_msg(chan);

        LOG_INF("alarm saw #%u", s->seq);
}

ZBUS_LISTENER_DEFINE(alarm, alarm_callback);
ZBUS_SUBSCRIBER_DEFINE(display, 4);

ZBUS_CHAN_DEFINE(sample_chan, struct sample, sample_is_valid, NULL,
                 ZBUS_OBSERVERS(alarm, display), ZBUS_MSG_INIT(0));

static void display_thread(void)
{
        const struct zbus_channel *chan;
        struct sample s;

        while (zbus_sub_wait(&display, &chan, K_FOREVER) == 0) {
                zbus_chan_read(chan, &s, K_FOREVER);
                LOG_INF("display saw #%u", s.seq);
        }
}
```

(*3 marks*)

One mark for the channel with its message type and validator, one for the listener, one for the subscriber and its wait-then-read loop. The publisher calls `zbus_chan_pub(&sample_chan, &s, ...)`*; a rejected message gives it* `-ENOMSG`*.*

Where each one runs. The **listener** is a callback run inside `zbus_chan_pub()`, in the publisher's own thread and with the channel locked — so it must be quick and must not block, like an ISR. The **subscriber** runs in its own thread, whenever the scheduler gets to it.

Why a subscriber can miss a message. It is not handed the message: it is handed a notification that the channel changed, and reads the channel later, by which time the channel holds whatever was published last. Two publications before the subscriber runs leave it

reading the second one twice, or seeing only the newer value. The listener cannot miss one, because it is called during the publication, before `zbus_chan_pub()` returns. Credit the message subscriber, `ZBUS_MSG_SUBSCRIBER_DEFINE`, as the observer that keeps a copy of every message and so misses none. *(1 mark)*

Question 7 A driver of your own

13 MARKS

(a)

6 MARKS

```c
#define DT_DRV_COMPAT qacademy_baro

#include <zephyr/device.h>
#include <zephyr/drivers/i2c.h>
#include <zephyr/drivers/sensor.h>
#include <zephyr/logging/log.h>

#include "baro_reg.h"

LOG_MODULE_REGISTER(baro, CONFIG_SENSOR_LOG_LEVEL);

struct baro_config {
        struct i2c_dt_spec i2c;
};

struct baro_data {
        uint16_t raw;
};

static int baro_sample_fetch(const struct device *dev, enum sensor_channel chan)
{
        const struct baro_config *config = dev->config;
        struct baro_data *data = dev->data;
        uint8_t buf[2];
        int ret;

        if ((chan != SENSOR_CHAN_ALL) && (chan != SENSOR_CHAN_PRESS)) {
                return -ENOTSUP;
        }

        ret = i2c_burst_read_dt(&config->i2c, BARO_REG_PRESSURE_L, buf, sizeof(buf
    ↪ ));
        if (ret < 0) {
```

```
                return ret;
        }

        data->raw = (uint16_t)buf[0] | ((uint16_t)buf[1] << 8);
        return 0;
}

static int baro_channel_get(const struct device *dev, enum sensor_channel chan,
                            struct sensor_value *val)
{
        const struct baro_data *data = dev->data;

        if (chan != SENSOR_CHAN_PRESS) {
                return -ENOTSUP;
        }

        /* One count is 10 Pa; the sensor API's pressure channel is in kilopascals
    ↪ . */
        val->val1 = data->raw / 100;
        val->val2 = (data->raw % 100) * 10000;
        return 0;
}

static DEVICE_API(sensor, baro_api) = {
        .sample_fetch = baro_sample_fetch,
        .channel_get = baro_channel_get,
};

static int baro_init(const struct device *dev)
{
        const struct baro_config *config = dev->config;
        uint8_t id;
        int ret;

        if (!i2c_is_ready_dt(&config->i2c)) {
                return -ENODEV;
        }

        ret = i2c_reg_read_byte_dt(&config->i2c, BARO_REG_WHO_AM_I, &id);
        if (ret < 0) {
                return ret;
        }

        if (id != BARO_WHO_AM_I_VALUE) {
```

```
                LOG_ERR("%s: WHO_AM_I is 0x%02x, expected 0x%02x", dev->name, id,
                        BARO_WHO_AM_I_VALUE);
                return -ENODEV;
        }

        return 0;
}

#define BARO_DEFINE(inst)                                                       ↩
    ↪ \
        static struct baro_data baro_data_##inst;                               ↩
    ↪ \
        static const struct baro_config baro_config_##inst = {                  ↩
    ↪ \
                .i2c = I2C_DT_SPEC_INST_GET(inst),                              ↩
    ↪ \
        };                                                                      ↩
    ↪ \
        SENSOR_DEVICE_DT_INST_DEFINE(inst, baro_init, NULL, &baro_data_##inst,  ↩
    ↪ \
                                     &baro_config_##inst, POST_KERNEL,          ↩
    ↪ \
                                     CONFIG_SENSOR_INIT_PRIORITY, &baro_api);

DT_INST_FOREACH_STATUS_OKAY(BARO_DEFINE)
```

(*6 marks*)

One mark each for:

1. **`DT_DRV_COMPAT`** spelled with the comma turned into an underscore, before the includes. Every `DT_INST_` macro in the file counts instances of that compatible only.
2. **The config/data split**: what the devicetree says is `const` and lives in `dev->config`; what changes at run time lives in `dev->data`. A driver that keeps its `i2c_dt_spec` in `data`, or that makes the config non-const, loses this mark — on a board the config is in flash.
3. **`baro_init` checking the bus and the identity**, and returning `-ENODEV` rather than 0 when the part answers with the wrong value.
4. **`sample_fetch`** reading both bytes in one burst and storing the raw value, with the channel checked.
5. **`channel_get`** converting to `sensor_value` correctly: 10132 counts gives `val1` 101 and `val2` 320000, because `val2` is millionths.
6. **`DT_INST_FOREACH_STATUS_OKAY(BARO_DEFINE)`** with `I2C_DT_SPEC_INST_GET`, so that the driver serves every node with that compatible rather than one.

A note for the marker: `SENSOR_DEVICE_DT_INST_DEFINE` and `DEVICE_DT_INST_DEFINE` are both acceptable, as is a plain `static const struct sensor_driver_api` in place of `DEVICE_API`.

(b) 4 MARKS

```
name: qacademy-baro
build:
  cmake: .
  kconfig: Kconfig
  settings:
    dts_root: .
```

(*1 mark*)

`dts_root: .` is what makes the module's `dts/bindings` visible to the build; without it the binding is not found and every node with that compatible has no binding.

```
config BARO
        bool "Baro I2C pressure sensor"
        default y
        depends on SENSOR
        depends on DT_HAS_QACADEMY_BARO_ENABLED
        select I2C
```

(*1 mark*)

```
description: Baro, a made-up I2C pressure sensor reading in steps of 10 Pa.

compatible: "qacademy,baro"

include: [sensor-device.yaml, i2c-device.yaml]
```

(*1 mark*)

`i2c-device.yaml` is what requires `reg`, the bus address, and makes the node an I2C device; `sensor-device.yaml` brings the properties every sensor may have.

Why the dependency. `DT_HAS_QACADEMY_BARO_ENABLED` is generated from the devicetree: it is defined exactly when some node has that compatible and status okay. Tying the driver to it means the driver is built when, and only when, the hardware is described — which is why enabling a driver in Zephyr usually means adding a devicetree node rather than setting an option. An application without the part gets none of the driver's code, and `default y` costs it nothing. (*1 mark*)

(c) 3 MARKS

```
#define BARO_NODE DT_NODELABEL(baro)

static const struct device *const baro = DEVICE_DT_GET(BARO_NODE);
static const struct emul *const baro_emul = EMUL_DT_GET(BARO_NODE);

ZTEST_SUITE(baro, NULL, NULL, NULL, NULL, NULL);

ZTEST(baro, test_device_is_ready)
{
        zassert_true(device_is_ready(baro));
}

ZTEST(baro, test_reading_converts_to_kilopascals)
{
        struct sensor_value pressure;

        baro_emul_set_raw(baro_emul, 10132);
        zassert_ok(sensor_sample_fetch(baro));
        zassert_ok(sensor_channel_get(baro, SENSOR_CHAN_PRESS, &pressure));
        zassert_equal(pressure.val1, 101);
        zassert_equal(pressure.val2, 320000);
}
```

(*2 marks*)

One mark for the suite and the readiness test, one for driving the emulator and asserting on both halves of the `sensor_value`*.*

The file is `testcase.yaml`, beside the test application, naming the platform and `harness: ztest`. `west twister -T <dir> -p native_sim/native/64` then builds and runs it.

`device_is_ready()` returns false for a device whose init returned a negative error: the kernel records the failure and the device stays unusable for the life of the program. The application must therefore check it before the first call — that is what the check is for, and a driver that fails its identity check is exactly the case it catches. (*1 mark*)

Question 8 Making it usable

12 MARKS

(a)

5 MARKS

```
static int32_t setpoint_c = 20;

static int cmd_setpoint_get(const struct shell *sh, size_t argc, char **argv)
{
        ARG_UNUSED(argc);
        ARG_UNUSED(argv);

        shell_print(sh, "setpoint %d C", setpoint_c);
        return 0;
}

static int cmd_setpoint_set(const struct shell *sh, size_t argc, char **argv)
{
        char *end;
        long value = strtol(argv[1], &end, 10);

        ARG_UNUSED(argc);

        if ((*end != '\0') || (value < -40) || (value > 125)) {
                shell_error(sh, "setpoint must be -40..125 degrees Celsius, not '%
    ↪ s'", argv[1]);
                return -EINVAL;
        }

        setpoint_c = (int32_t)value;
        shell_print(sh, "setpoint %d C", setpoint_c);
        return 0;
}

SHELL_STATIC_SUBCMD_SET_CREATE(setpoint_cmds,
                               SHELL_CMD(get, NULL, "Print the setpoint.",
    ↪ cmd_setpoint_get),
                               SHELL_CMD_ARG(set, NULL, "Set the setpoint: set <
    ↪ celsius>.",
                                             cmd_setpoint_set, 2, 0),
                               SHELL_SUBCMD_SET_END);

SHELL_CMD_REGISTER(setpoint, &setpoint_cmds, "The temperature setpoint.", NULL);
```

(4 marks)

One mark for the handler signature and `shell_print` to the shell that ran the command rather than `printk`; one for the subcommand set and its `SHELL_SUBCMD_SET_END`; one for `SHELL_CMD_ARG` with two mandatory words; one for validating the text as well as the range — `strtol` alone accepts `abc` as 0, which is why the end pointer is checked.

What the shell has already checked: the argument *count*. `SHELL_CMD_ARG(..., 2, 0)` declares two mandatory words and no optional ones, so `setpoint set` on its own never reaches the handler; the shell rejects it with `set: wrong parameter count` and prints the command's help. What is left to the handler is what the words *say*.

The return value is 0 for success or a negative errno; the shell reports a failure to the user, and it is what a script or a test driving the shell sees. *(1 mark)*

(b)

4 MARKS

```
static int setpoint_set_cb(const char *name, size_t len, settings_read_cb read_cb,
    void *cb_arg)
{
        const char *next;

        if (settings_name_steq(name, "celsius", &next) && (next == NULL)) {
                if (len != sizeof(setpoint_c)) {
                        return -EINVAL;
                }
                ssize_t read = read_cb(cb_arg, &setpoint_c, sizeof(setpoint_c));

                return (read < 0) ? (int)read : 0;
        }
        return -ENOENT;
}

SETTINGS_STATIC_HANDLER_DEFINE(setpoint, "setpoint", NULL, setpoint_set_cb, NULL,
    NULL);
```

with, at start-up, `settings_subsys_init()` then `settings_load()`, and in the `set` handler:

```
        settings_save_one("setpoint/celsius", &setpoint_c, sizeof(setpoint_c));
```

(2 marks)

One mark for a handler registered for the subtree and reading through `read_cb` rather than expecting a pointer to the value, one for `settings_save_one()` with the full name and `settings_subsys_init()`/`settings_load()` at boot.

The Kconfig options: `CONFIG_SETTINGS=y`, a storage back-end — `CONFIG_ZMS=y` or `CONFIG_NVS=y` — and the flash underneath it, `CONFIG_FLASH=y` and `CONFIG_FLASH_MAP=y`. *(1 mark)*

Where it is kept: in the flash partition labelled `storage_partition` in the board's device-tree. On `native_sim` that flash is a file, `flash.bin`, in the directory the program is run from — the build directory when it is started with `west build -t run`.

Two ways to make the next run a first boot, any two of: delete `flash.bin`; start the program with `-flash_erase`; start it with `-flash_in_ram`, so the flash never touches the disk; or build with `-p`, which deletes the build directory and the file with it. *(1 mark)*

(c) 3 MARKS

```
void k_sys_fatal_error_handler(unsigned int reason, const struct arch_esf *esf)
{
        ARG_UNUSED(esf);

        LOG_PANIC();
        LOG_ERR("fatal error %u in thread '%s'", reason, k_thread_name_get(↩
    ↪ k_current_get()));

        if (reason == K_ERR_KERNEL_OOPS) {
                return;                 /* The kernel aborts this thread only. */
        }

        k_fatal_halt(reason);
}
```

(2 marks)

One mark for replacing the weak handler with one that returns on `K_ERR_KERNEL_OOPS`, one for halting otherwise. `LOG_PANIC()` earns credit as an addition: it puts logging into synchronous mode, so that what has been logged is actually printed before the system goes down — which matters on a board, where logging is deferred to a thread that may never run again.

The exit status is 1. Halting on `native_sim` ends the process, and the harness sees a failure; that is also why Twister treats such a run as failed whatever it printed.

Why returning from a panic is not allowed: an oops means one thread has found something impossible, and the kernel can abort just that thread and carry on. A panic means the kernel itself cannot continue — a failed assertion inside it, a corrupted structure, a fault with no thread to blame — so there is nothing to return *to*. The kernel's own handler halts for exactly that reason. *(1 mark)*